

Mobile Robotics

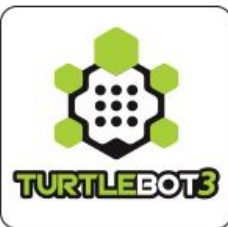
Bachelor's
Degree in
Robotic
Intelligence



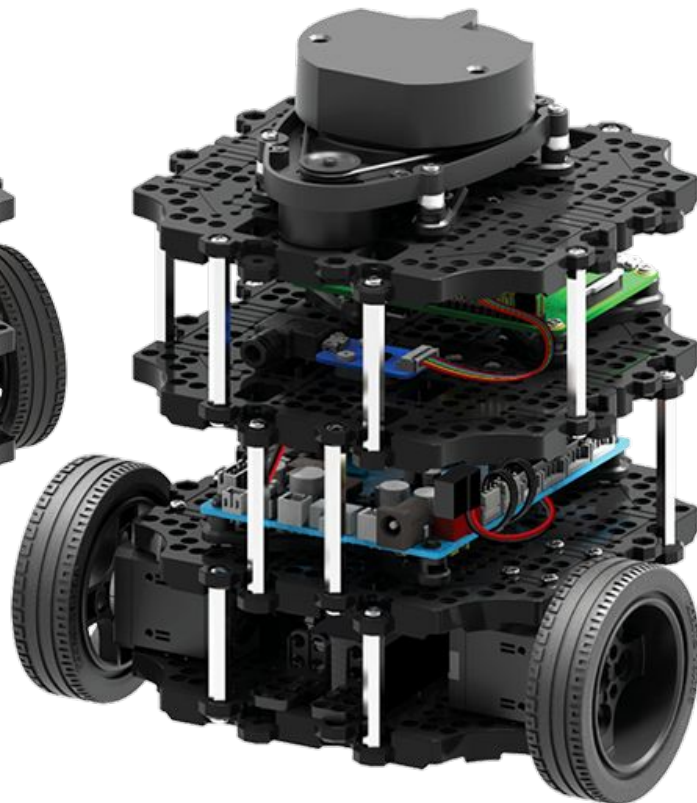
What is a mobile robot?

Examples of mobile robots

Waffle Pi



Burger



• Collaboration with  open
robotics



TurtleBot2

What is a TurtleBot?

TurtleBot is a low-cost, personal robot kit with open-source software. It was created at [Willow Garage](#) in November 2010.

With TurtleBot, you'll be able to build a robot that can drive around your house, see in 3D, and have enough horsepower to create exciting applications.

For more information, see an interview with TurtleBot creators, Tully Foote and Melonee Wise: [TurtleBot Inventors Tell Us Everything About the Robot \(IEEE Spectrum, By Evan Ackerman, 26 Mar 2013\)](#).

There are two generations of TurtleBots in production now: the [TurtleBot4](#) and the [TurtleBot3](#).

TurtleBot Family

There are now 4 generations of TurtleBots.

Find out more about the history of the Turtlebot [here](#).

There is a long story between turtles and robots, dating back to the [20th century](#)...

Original TurtleBot (Discontinued)



TurtleBot 2 Family (Discontinued)



TurtleBot 2



TurtleBot 2i



TurtleBot 2e



TurtleBot Euclid

TurtleBot 3 Family

Burger



Waffle



Waffle Pi



TurtleBot 4 Family

NEW



Lite



Standard

TurtleBot concept

The TurtleBot kit consists of a mobile base, 2D/3D distance sensor, laptop computer or SBC (Single Board Computer), and the TurtleBot mounting hardware kit.

In addition to the kit, users can download the TurtleBot SDK from the ROS website.

TurtleBot is designed to be easy to buy, build, and assemble, using off the shelf consumer products and parts that easily can be created from standard materials.

As an entry level mobile robotics platform, TurtleBot has many of the capabilities of larger robotics platforms.



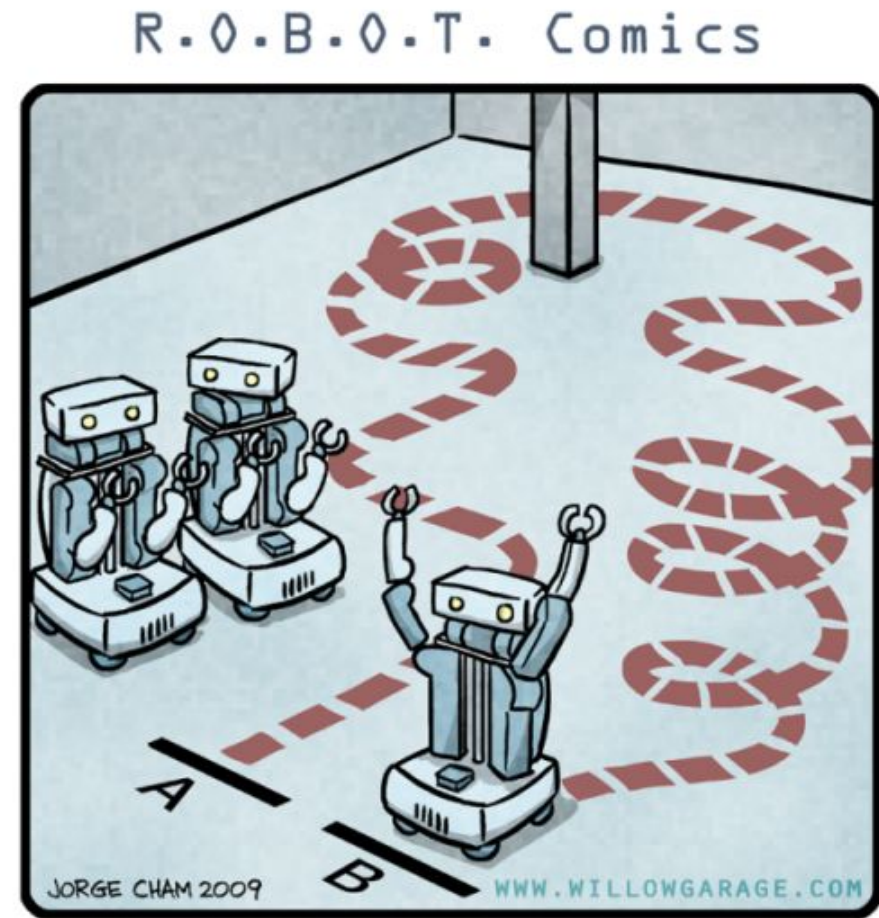
Core Technology

The TurtleBot's core technology is SLAM (simultaneous localization and mapping) and Navigation, making it suitable for home service robots.

The TurtleBot can run SLAM algorithms to build a map and can drive around your room.

Also, it can be controlled remotely from a laptop, joypad or Android-based smart phone.

The TurtleBot can also follow a person's legs as they walk in a room.



"HIS PATH-PLANNING MAY BE SUB-OPTIMAL, BUT IT'S GOT FLAIR."

Aim of this course



References

[TurtleBot](#)

[ROS Documentation](#)

[A Concise Introduction to Robot Programming with ROS2](#)

[Introduction to Autonomous Mobile Robots](#)

[Autonomous Mobile Robots - Online Course](#)

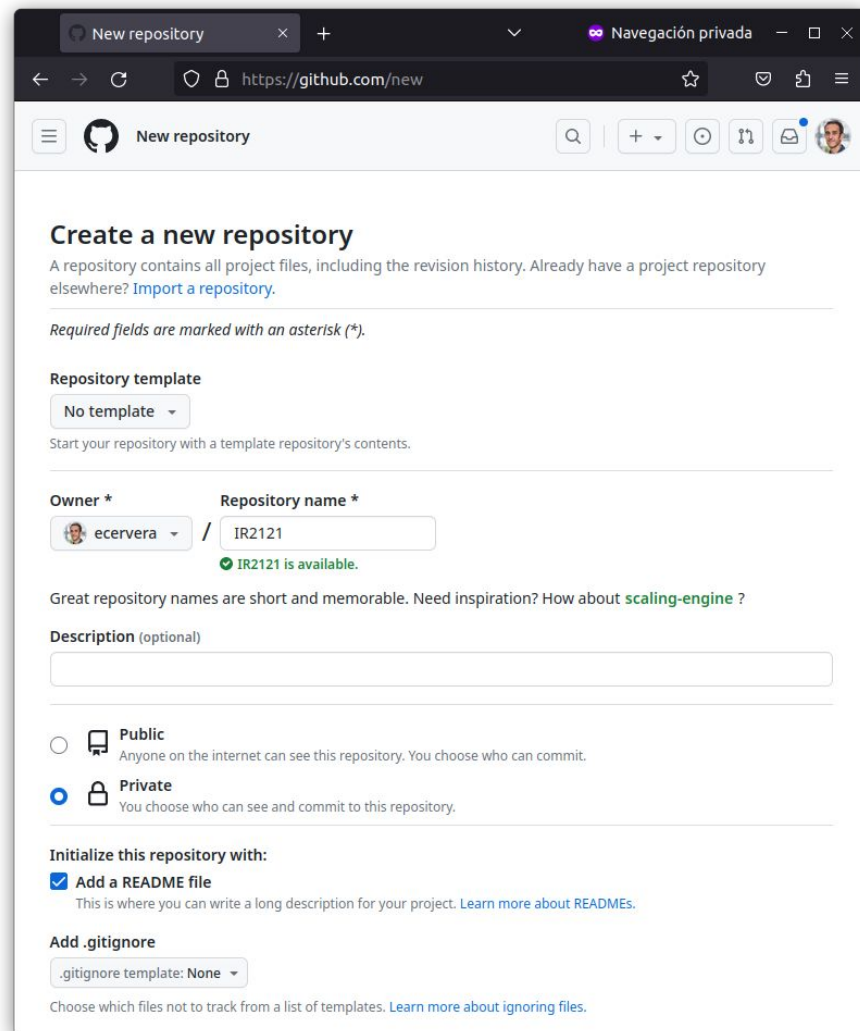
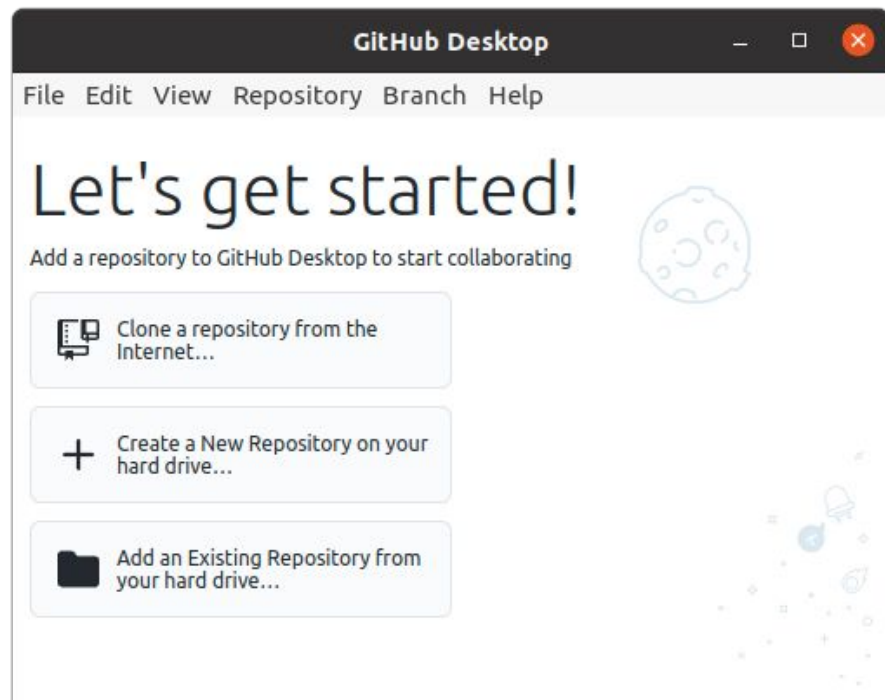
Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence



Preliminary setup

Create a GitHub repository
named "IR2121"



Actividades

GitHub Desktop

Carpetas personal Documentos GitHub IR2121

Nombre	Tamaño	Modificación	Destacar
.git	11 elementos	10:59	☆
README.md	8 bytes	10:59	☆

GitHub Desktop

File Edit View Repository Branch Help

Current repository IR2121 Current branch main Fetch origin Never fetched

Changes History

0 changed files

No local changes

There are no uncommitted changes in this repository. Here are some friendly suggestions for what to do next.

Open the repository in your external editor
Select your editor in [Options](#)
Repository menu or `Ctrl Shift A` [Open in GEdit](#)

View the files of your repository in your File Manager
Repository menu or `Ctrl Shift F` [Show in your File Manager](#)

Open the repository page on GitHub in your browser
Repository menu or `Ctrl Shift G` [View on GitHub](#)

Summary (required)
Description
Commit to main

6 de sep 11:01

ecervera/IR2121

https://github.com/ecervera/IR2121

ecervera / IR2121

<> Code Issues Pull requests Actions Projects Security Insights Settings

IR2121 Private

Unwatch 1 Fork 0 Star 0

main

Go to file Add file <> Code

About

No description, website, or topics provided.

ecervera Initial commit 3 minutes ago 1

README.md Initial commit 3 minutes ago

README.md

IR2121

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

© 2023 GitHub, Inc.

Grant permissions

Please grant permissions for viewing to:

- `ecervera@uji.es`

The screenshot shows the GitHub repository settings page for 'ecervera/IR2121'. The browser address bar displays 'https://github.com/ecervera/IR2121/settings/access'. The page has a sidebar on the left with navigation links: General, Access (selected), Code and automation, Security, and Integrations. Under 'Access', 'Collaborators' is selected. The main content area is titled 'Who has access' and shows two sections: 'PRIVATE REPOSITORY' and 'DIRECT ACCESS'. The 'PRIVATE REPOSITORY' section states 'Only those with access to this repository can view it.' and has a 'Manage' link. The 'DIRECT ACCESS' section states '0 collaborators have access to this repository. Only you can contribute to this repository.' Below these sections is a 'Manage access' section with a message 'You haven't invited any collaborators yet' and a green 'Add people' button. The footer contains links for Terms, Privacy, Security, Status, Docs, Contact GitHub, Pricing, API, Training, Blog, and About, along with the GitHub logo and copyright notice '© 2023 GitHub, Inc.'.

ecervera/IR2121

https://github.com/ecervera/IR2121/settings/access

ecervera / IR2121

<> Code Issues Pull requests Actions Projects Security Insights Settings

General

Access

Collaborators

Code and automation

Branches

Tags

Rules

Actions

Webhooks

Codespaces

Pages

Security

Code security and analysis

Deploy keys

Secrets and variables

Integrations

GitHub Apps

Email notifications

Who has access

PRIVATE REPOSITORY

Only those with access to this repository can view it.

Manage

DIRECT ACCESS

0 collaborators have access to this repository. Only you can contribute to this repository.

Manage access

You haven't invited any collaborators yet

Add people

Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

© 2023 GitHub, Inc.

https://github.com/ecervera/IR2121/settings/branches

Assignment

Submit the URL of your repository through the Virtual Classroom.

The screenshot shows a web browser window with the URL <https://aulavirtual.uji.es/mod/assign/view.php?id=5115791&>. The page header includes the Universitat Jaume I logo, 'Aula Virtual', and navigation links 'Inici' and 'Més'. A search bar and user profile are also present.

The main content area displays the assignment title 'URL of repository' under the 'TASCA' category. Below the title is a text input field with the placeholder 'Submit the URL of your GitHub repository.' and a blue button labeled 'Afegeix la tramesa'.

Below the input field is a section titled 'Estat de la tramesa' containing a table with the following data:

Estat de la tramesa	Encara no s'ha fet cap tramesa
Estat de la qualificació	Sense qualificació
Darrera modificació	-
Comentaris de la tramesa	Comentaris (0)

A help icon (?) is located in the bottom right corner of the page.

Policy for Use of AI in Teaching and Learning¹

[Recomanacions sobre l'ús de la IA generativa en la docència i l'avaluació - CENT](#)



Gemini



deepseek



¹[University Policy Workgroup for AI in Teaching and Learning - National University of Singapore](#)

Policy for Use of AI in Teaching and Learning¹

Recomanacions sobre l'ús de la IA generativa en la docència i l'avaluació - CENT



¹[University Policy Workgroup for AI in Teaching and Learning - National University of Singapore](#)

Acknowledging your use of AI

En els treballs de l'estudiantat, **l'ús literal de respostes generades per IA com si foren originals es pot considerar una acció fraudulenta**, segons l'article 11 de la [normativa d'avaluació de grau i màster de l'UJI](#).

If you completed any work with the aid of an AI tool, **you should always acknowledge the use**.

One way this can be done is in a tabular form as shown below:

AI Tool Used	Prompt and output	How the output is used in the assignment

You are Responsible for Your Work

Remember the limitations of current generative AI tools:

- Output's quality is dependent on the quality of the users' prompts.
- Output may be out of date, as they are dependent on the available training data.
- Output may not be accurate (e.g., they don't always present information that is true, the 'citations' they may generate may be made-up and point to non-existent sources).
- Output may present dominant values and opinions as truth not because other views are incorrect, but simply because dominant claims are more common in the training data.

Policy for Use of AI in this course

- Allowed in homework
 - ➔ always acknowledge the use.
- Not recommended in laboratory work.
- Not allowed in partial and final examinations.

Connecting to the TurtleBot 3

1. Insert a battery
2. Switch on the robot
3. Wait for one minute or so
4. Open a terminal in the PC and execute:

```
ssh ubuntu@192.168.0.xxx
```

If the following message appears just type “yes”+Enter:

```
The authenticity of host '192.168.0.xxx (192.168.0.xxx)' can't be established.  
ECDSA key fingerprint is SHA256:kb8u86vfMPZ1wCmym+sCeWE62gS4X5mftT3mQXjsiME.  
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

The password is “turtlebot”

Bringup

Robots 101-110:

```
source /opt/ros/humble/setup.bash  
export TURTLEBOT3_MODEL=burger  
export LDS_MODEL=LDS-01  
export ROS_DOMAIN_ID=1 # 2, 3, 4... segons el robot  
ros2 launch turtlebot3_bringup robot.launch.py
```

Robots 111-112:

```
source /opt/ros/humble/setup.bash  
export TURTLEBOT3_MODEL=waffle_pi  
export LDS_MODEL=LDS-01  
export ROS_DOMAIN_ID=11 # o 12 segons el robot  
ros2 launch turtlebot3_bringup robot.launch.py
```

Robots 202, 203, 204, 205, 206:

```
source /opt/ros/humble/setup.bash  
export TURTLEBOT3_MODEL=burger  
export LDS_MODEL=LDS-02  
export ROS_DOMAIN_ID=22 # 23, 24, 25, 26 segons el robot  
ros2 launch turtlebot3_bringup robot.launch.py
```

Shutting down the robot

1. Stop the bringup command in the first terminal with Ctrl+C
2. Run the command:

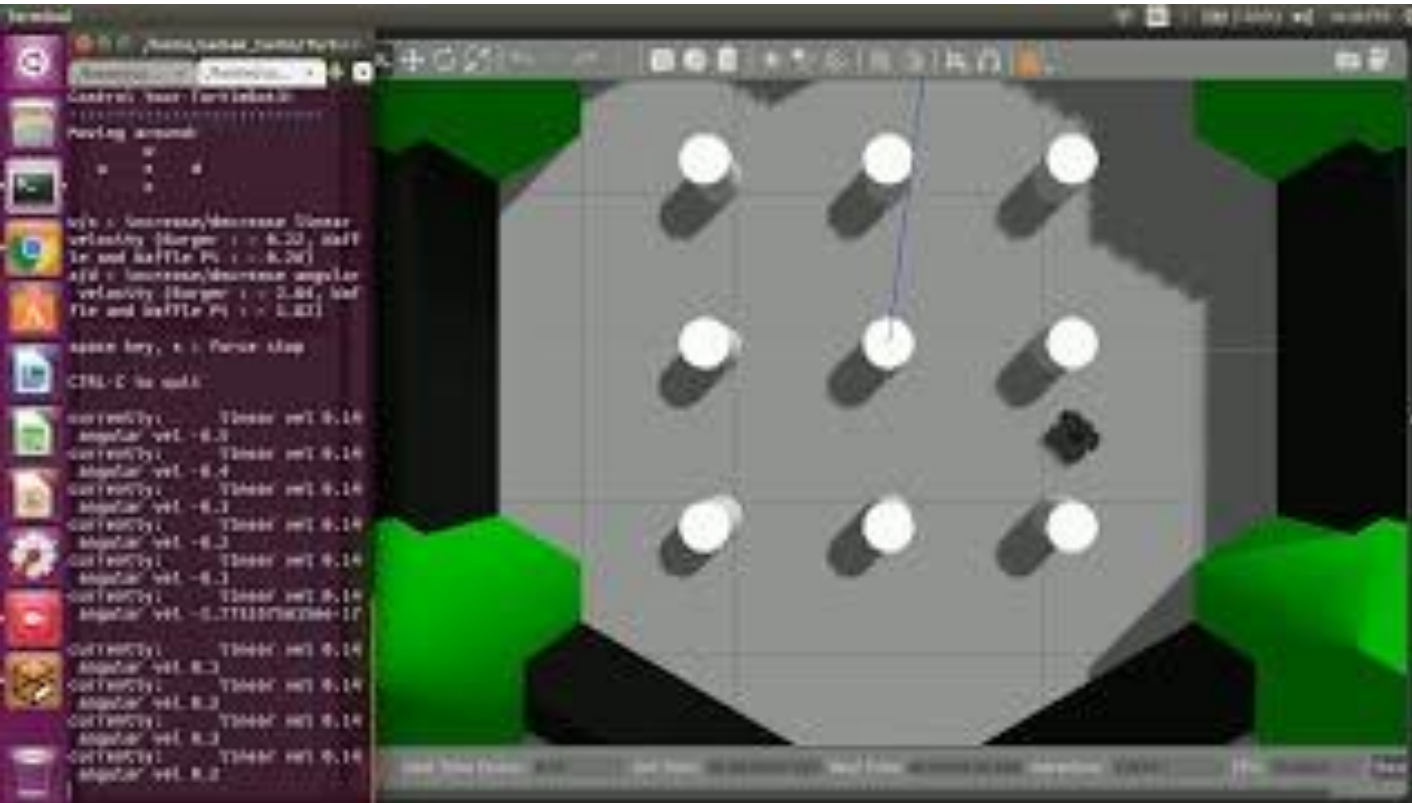
```
sudo halt -p
```

3. Wait for the yellow LED to stop blinking completely
4. Power off the robot
5. Remove the battery and put it in a charger

Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence

Teleoperation



Teleoperation of TurtleBot 3 in ROS Humble and Gazebo

1. Create a folder named “Teleoperation” in your repository.
2. Create a folder named “scripts” inside the “Teleoperation” folder.
3. Create a BASH script named “launch_tb3_house_sim.sh” with the following lines
4. Change the script properties (permission to execute).
5. Commit and push the repository.
6. Test the script.



The screenshot shows a terminal window titled "launch_tb3_house_sim.sh" with the following content:

```
1#!/bin/bash
2source /opt/ros/humble/setup.bash
3export ROS_LOCALHOST_ONLY=1
4export TURTLEBOT3_MODEL=burger
5ros2 launch turtlebot3_gazebo \
6    turtlebot3_house.launch.py
7|
```

The terminal window has a dark theme and includes standard window controls (Abrir, Guardar, etc.) and a status bar at the bottom showing "sh", "Anchura del tabulador: 4", "Ln 7, Col 1", and "INS".

Teleoperation of TurtleBot 3 in ROS Humble and Gazebo

The screenshot displays a desktop environment with a dark theme. On the left, a vertical dock contains icons for various applications, including a file manager, terminal, and web browser. The main area is divided into three windows:

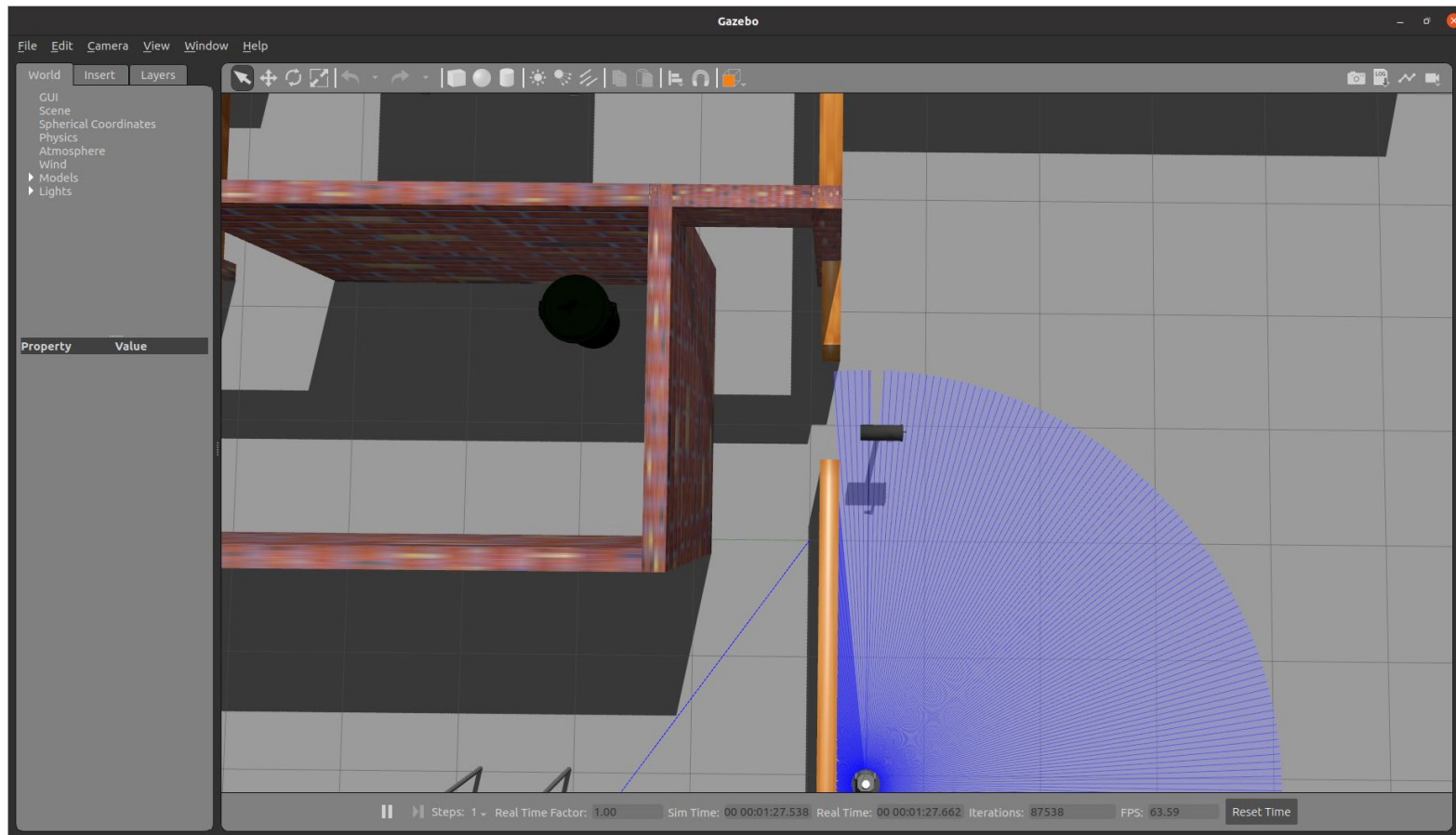
- File Manager (Firefox):** Shows a directory listing for 'Teleoperation' with a file named 'launch_tb3_house_sim.bash' (168 bytes, modified 11:44).
- Terminal (Git Desktop):** Displays the 'Create script' dialog for a new repository named 'IR2121'. The 'Current branch' is 'main'. The 'Fetch origin' button is visible. The 'Changes' tab shows a list of files: 'Teleop.../launch_tb3_house_sim.bash', 'turtletbot3_house.launch.py', and 'turtletbot3_house.launch.py'. The 'Initial commit' is shown as 'Enric Cervera • 1 hour ago'.
- Web Browser (Firefox):** Displays the GitHub repository page for 'ecervera/IR2121'. The repository is private and has 0 stars, 0 forks, and 1 watching. The 'About' section states 'No description, website, or topics provided.' The 'Releases' section shows 'No releases published'. The 'Languages' section shows 'Shell 100.0%'. The 'README.md' file is visible, containing the text 'IR2121'.

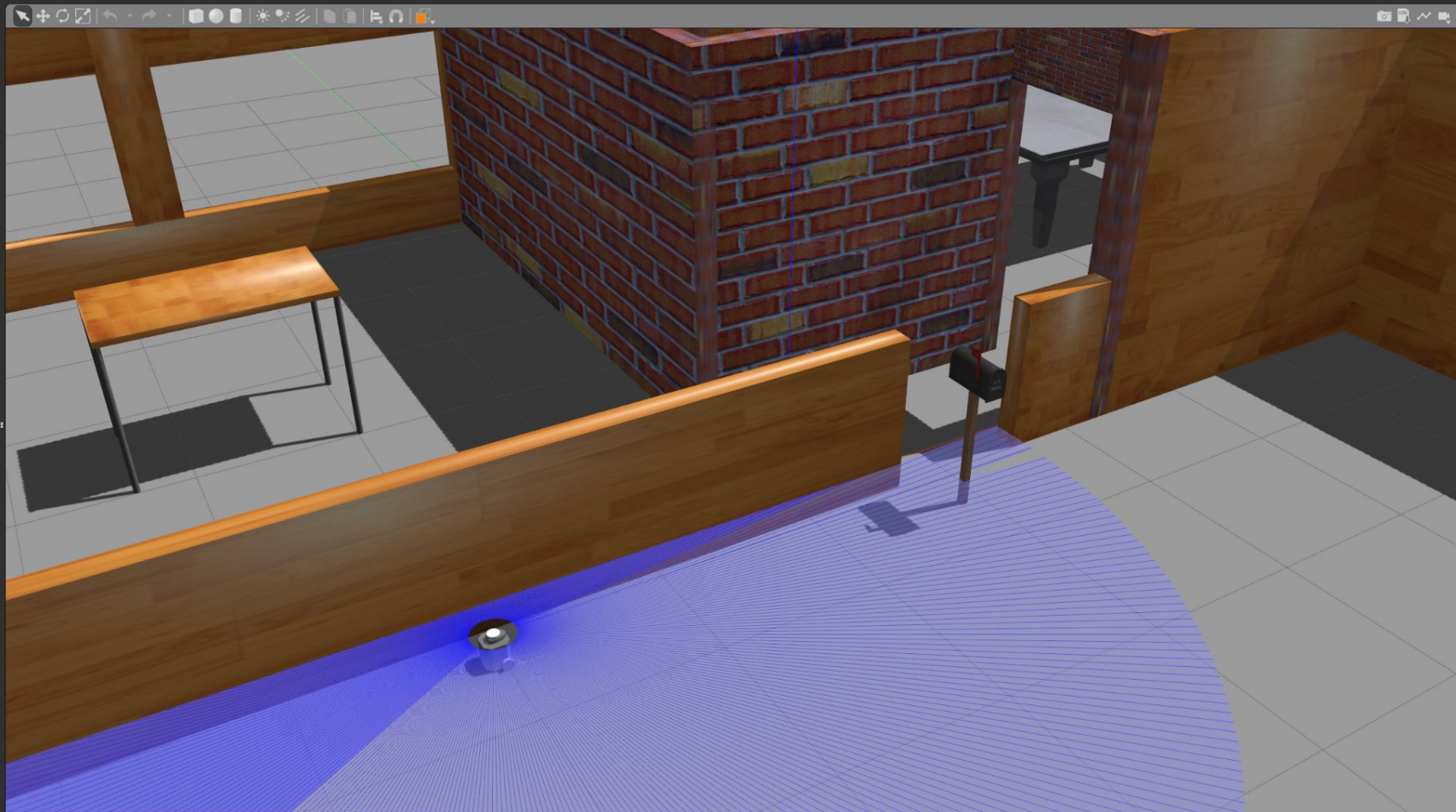
The bottom of the screen shows the system tray with icons for network, volume, and power. The date and time are displayed as '6 de sep 11:58'.

Teleoperation of TurtleBot 3 in ROS Humble and Gazebo

```
usuario@LABROB06: ~/Documentos/GitHub/IR2121/Teleoperation/scripts
usuario@LABROB06:~/Documentos/GitHub/IR2121/Teleoperation/scripts$ ./launch_tb3_house_sim.bash
[INFO] [launch]: All log files can be found below /home/usuario/.ros/log/2023-09-06-11-52-00-845637-LABROB06-9392
[INFO] [launch]: Default logging verbosity is set to INFO
urdf_file_name : turtlebot3_burger.urdf
[INFO] [gzserver-1]: process started with pid [9394]
[INFO] [gzclient -2]: process started with pid [9397]
[INFO] [robot_state_publisher-3]: process started with pid [9400]
[robot_state_publisher-3] [WARN] [1693993921.175503986] [robot_state_publisher]: No robot_description parameter, but
command-line argument available. Assuming argument is name of URDF file. This backwards compatibility fallback wi
ll be removed in the future.
[robot_state_publisher-3] Parsing robot urdf xml string.
[robot_state_publisher-3] Link base_link had 5 children
[robot_state_publisher-3] Link caster_back_link had 0 children
[robot_state_publisher-3] Link imu_link had 0 children
[robot_state_publisher-3] Link base_scan had 0 children
[robot_state_publisher-3] Link wheel_left_link had 0 children
[robot_state_publisher-3] Link wheel_right_link had 0 children
[robot_state_publisher-3] [INFO] [1693993921.176444524] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1693993921.176455327] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1693993921.176458825] [robot_state_publisher]: got segment base_scan
[robot_state_publisher-3] [INFO] [1693993921.176461749] [robot_state_publisher]: got segment caster_back_link
[robot_state_publisher-3] [INFO] [1693993921.176464663] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-3] [INFO] [1693993921.176467477] [robot_state_publisher]: got segment wheel_left_link
[robot_state_publisher-3] [INFO] [1693993921.176470331] [robot_state_publisher]: got segment wheel_right_link
[gzserver-1] Warning [parser.cc:833] XML Attribute[version] in element[sdf] not defined in SDF, ignoring.
[gzserver-1] [INFO] [1693993922.638846323] [turtlebot3_imu]: <initial_orientation_as_reference> is unset, using defa
ult value of false to comply with REP 145 (world as orientation reference)
[gzserver-1] [INFO] [1693993922.664157108] [turtlebot3_diff_drive]: Wheel pair 1 separation set to [0.160000m]
[gzserver-1] [INFO] [1693993922.664202167] [turtlebot3_diff_drive]: Wheel pair 1 diameter set to [0.066000m]
[gzserver-1] [INFO] [1693993922.664953963] [turtlebot3_diff_drive]: Subscribed to [/cmd_vel]
[gzserver-1] [INFO] [1693993922.666052752] [turtlebot3_diff_drive]: Advertise odometry on [/odom]
[gzserver-1] [INFO] [1693993922.666004940] [turtlebot3_diff_drive]: Publishing odom transforms between [odom] and [b
ase_footprint]
[gzserver-1] [INFO] [1693993922.673107578] [turtlebot3_joint_state]: Going to publish joint [wheel_left_joint]
[gzserver-1] [INFO] [1693993922.673139798] [turtlebot3_joint_state]: Going to publish joint [wheel_right_joint]
```

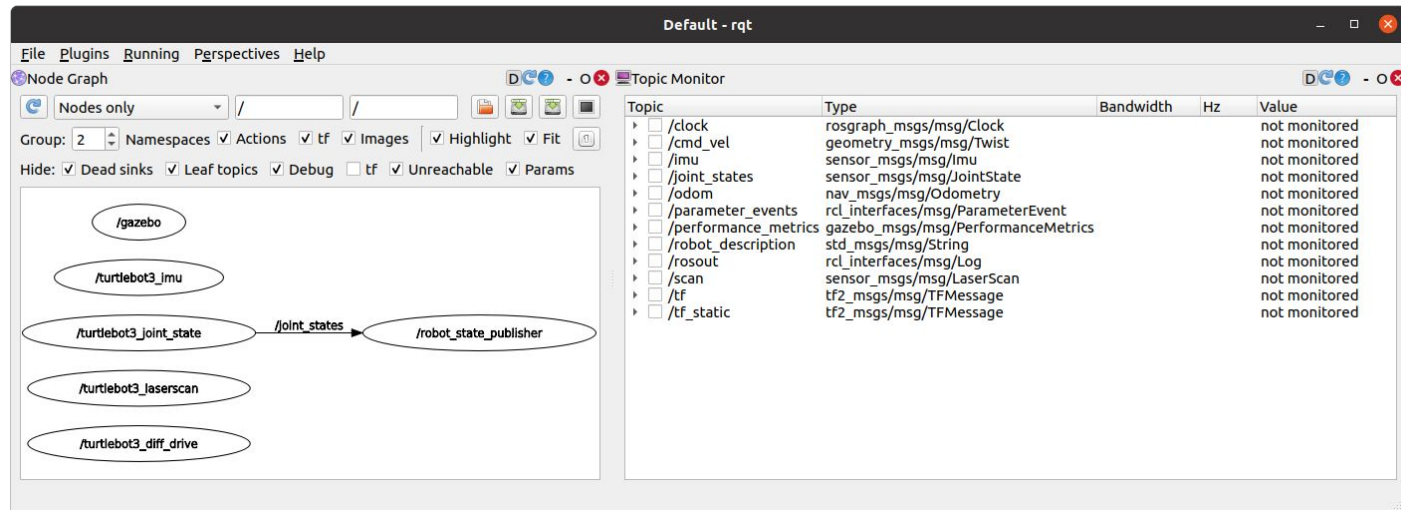
Teleoperation of TurtleBot 3 in ROS Humble and Gazebo



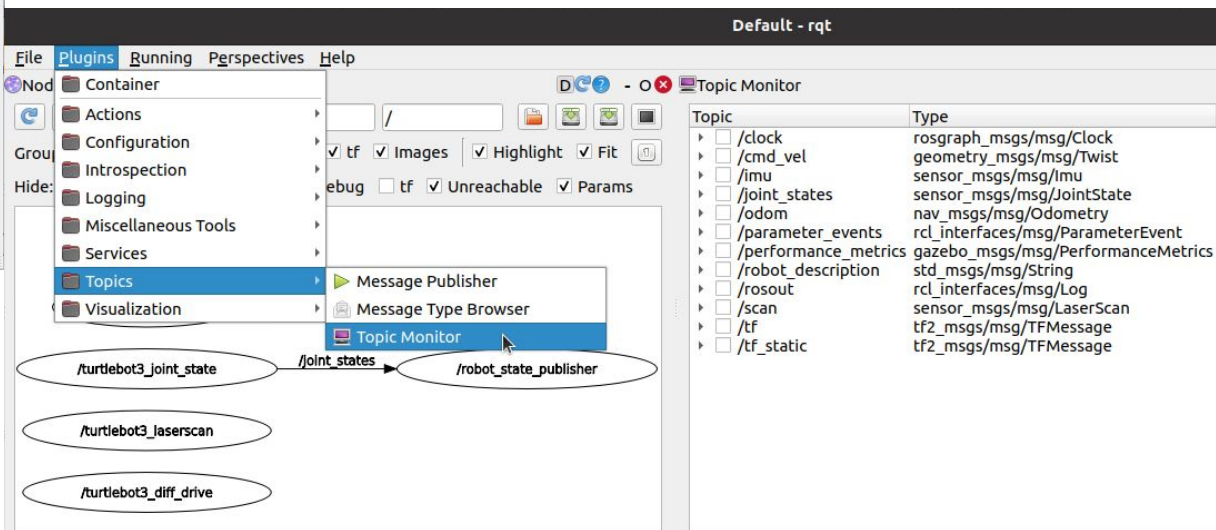
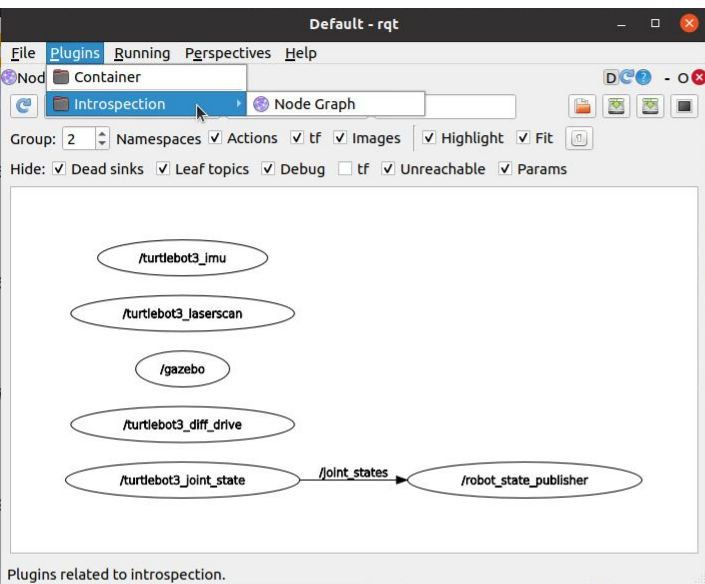


Visualization

1. Create another BASH script named “`launch_rqt.sh`” in the scripts folder.
2. Change the script properties (permission to execute).
3. Commit and push the repository.
4. Test the script.



Visualization

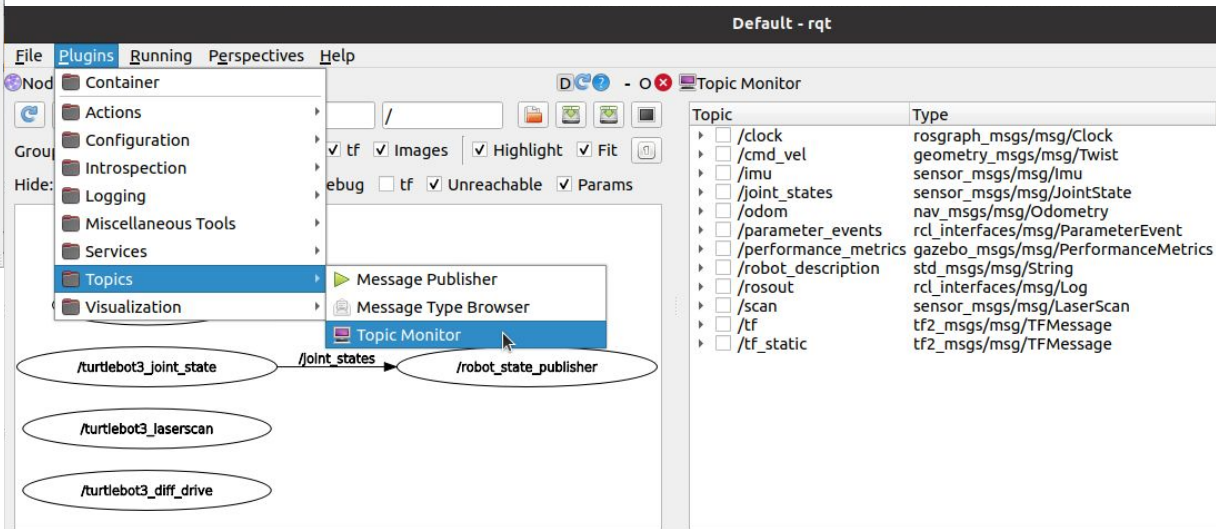
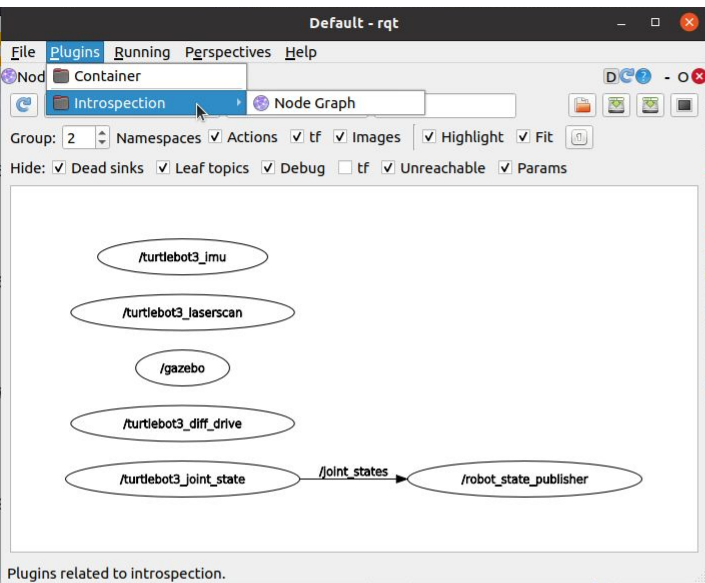


Missing rqt plugins

```
sudo apt update
```

```
sudo apt install \
ros-humble-rqt-common-plugins
```

Launch rqt with **--force-discover** option.



Odometry topic

Node Graph

Nodes only / /

Group: 2 Namespaces ☒ Actions ☒ tf ☒ Images ☒ Highlight ☒ Fit ☐

Hide: ☒ Dead sinks ☒ Leaf topics ☒ Debug ☐ tf ☒ Unreachable ☒ Params

```
graph LR; gazebo((/gazebo)) --> turtlebot3_joint_state((/turtlebot3_joint_state)); turtlebot3_joint_state -- /joint_states --> robot_state_publisher((/robot_state_publisher)); turtlebot3_laserscan((/turtlebot3_laserscan)); turtlebot3_imu((/turtlebot3_imu)); turtlebot3_diff_drive((/turtlebot3_diff_drive));
```

Topic Monitor

Topic	Type	Bandwidth	Hz	Value
☐ /clock	roscpp_msgs/msg/Clock			not monitored
☐ /cmd_vel	geometry_msgs/msg/Twist			not monitored
☐ /imu	sensor_msgs/msg/Imu			not monitored
☐ /joint_states	sensor_msgs/msg/JointSt...			not monitored
☒ /odom	nav_msgs/msg/Odometry	unknown	28.91	'base_footpr...
child_frame_id	string			
header	std_msgs/Header			
▼ pose	geometry_msgs/PoseWit...			
covariance	double[36]			array([1.e-05,...
orientation	geometry_msgs/Quaternion			0.e+00, 0....
▼ position	geometry_msgs/Point			0.e+00, 0....
x	double			0.e+00, 0...
y	double			-1.99989524...
z	double			-0.50000107...
twist	geometry_msgs/TwistWit...			0.008530375...
☐ /parameter_events	rcl_interfaces/msg/Param...			not monitored

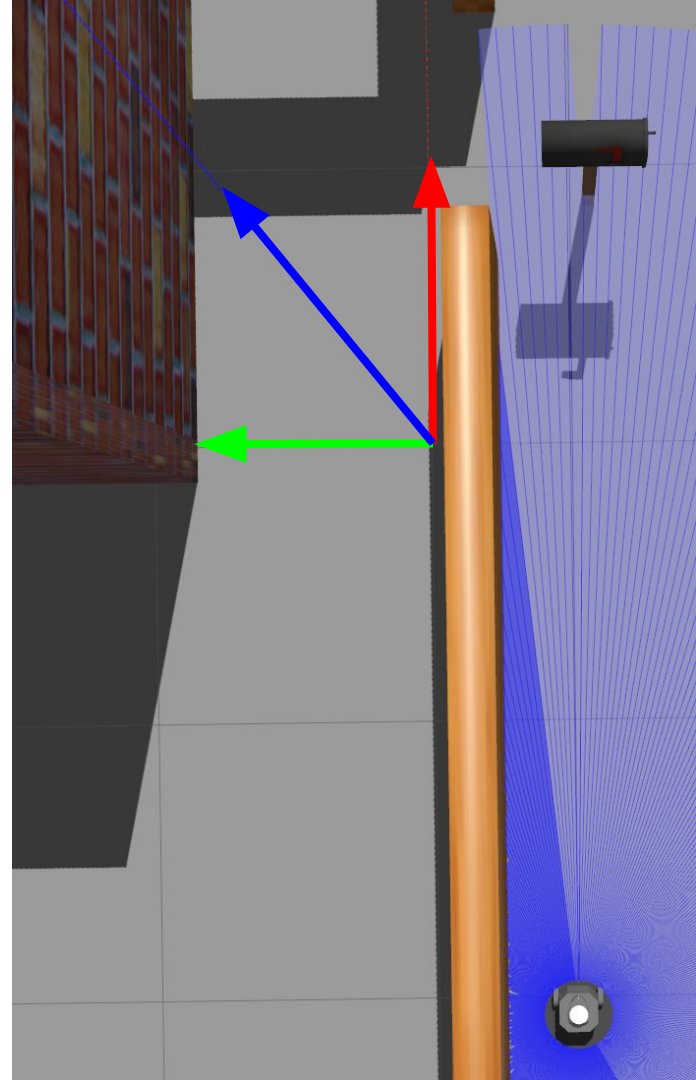
Origin of coordinates

Pose of robot

x -1.9998

y -0.5000

z 0.0085



Teleoperation

1. Create another BASH script named “`launch_tb3_teleop.sh`” in the scripts folder.
2. Change the script properties (permission to execute).
3. Commit and push the repository.
4. Test the script.

Control Your TurtleBot3!

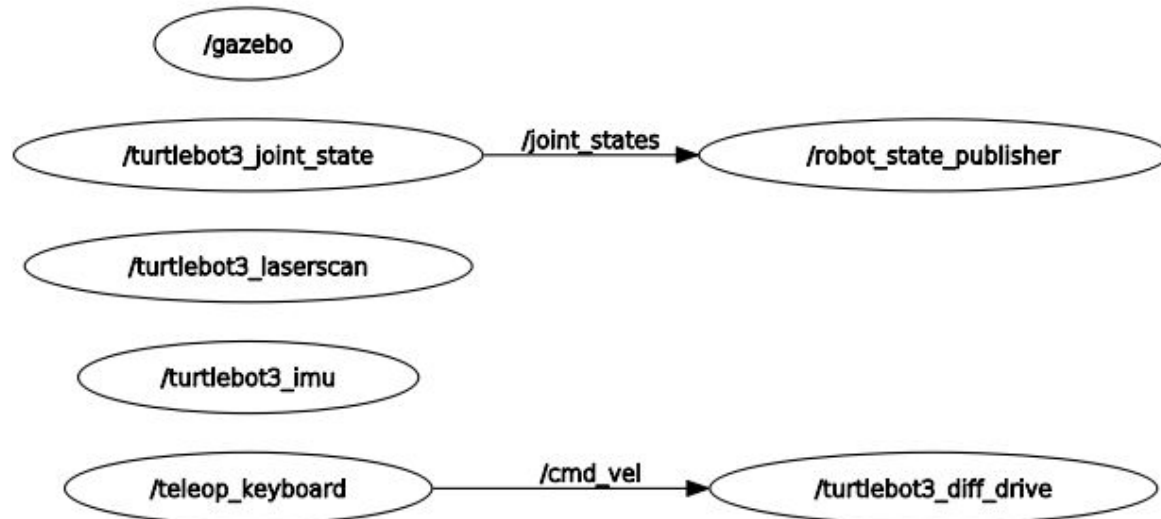
Moving around:

 w
a s d
 x

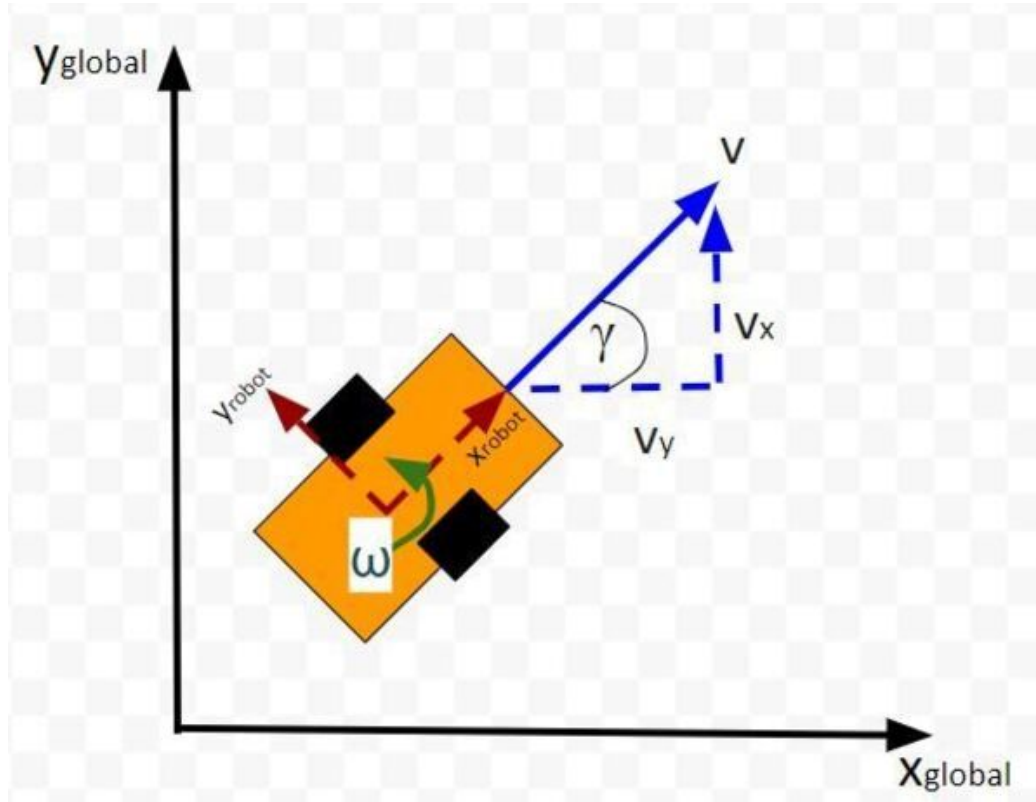
w/x : increase/decrease linear velocity
(Burger : ~ 0.22, Waffle Pi : ~ 0.26)
a/d : increase/decrease angular velocity
(Burger : ~ 2.84, Waffle Pi : ~ 1.82)

space key, s : force stop

CTRL-C to quit



Linear and angular velocities



Task

Teleoperate the robot

Visit all the rooms

Finish at the same initial location

Record topics with `ros2 bag record`

Plot trajectory in RViz

ROS bags

Record

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
ros2 bag record /clock /odom /tf /tf_static
```

Play

```
ros2 bag play rosbag2_2026_...
```

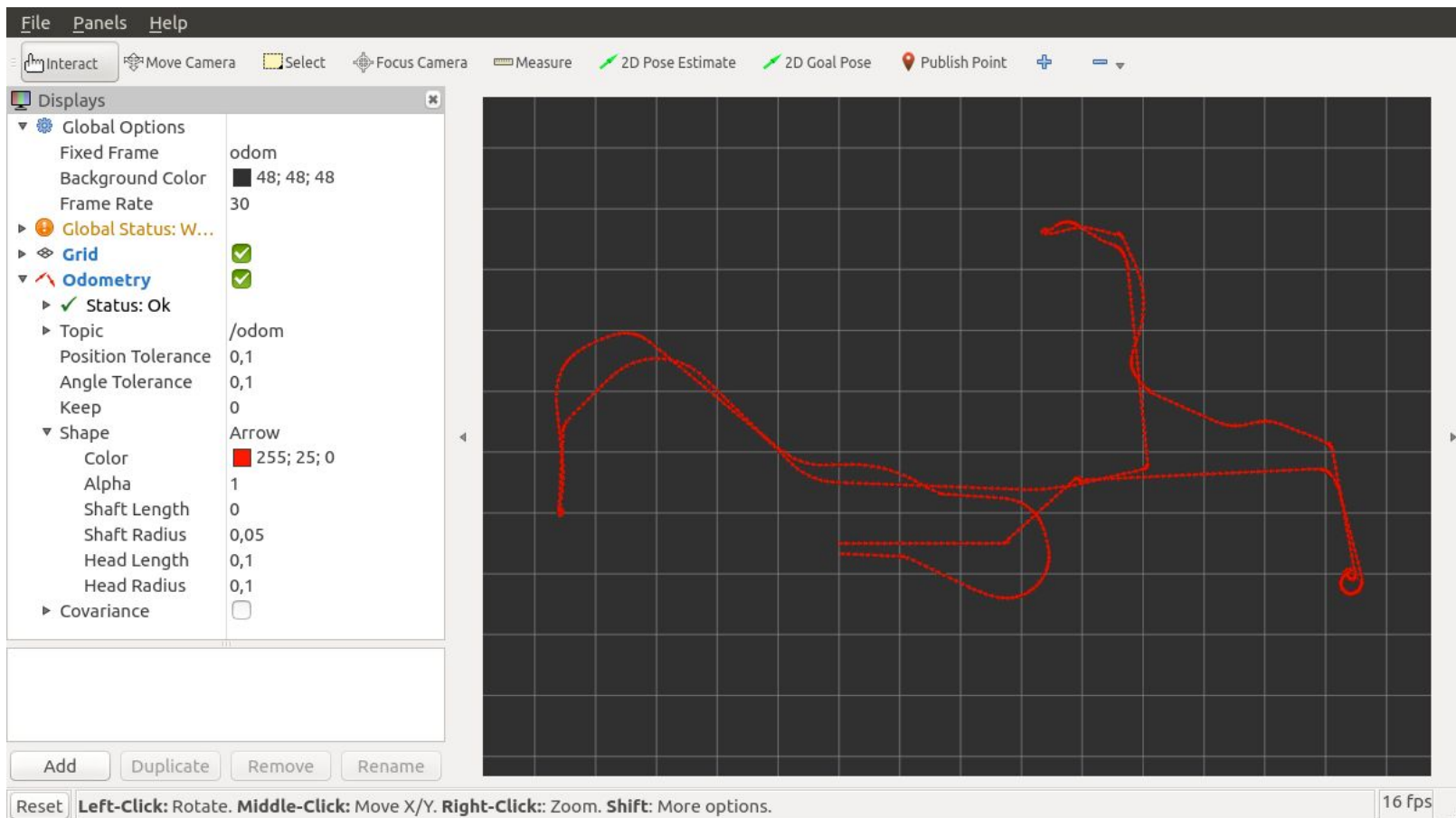
Analyze bag files

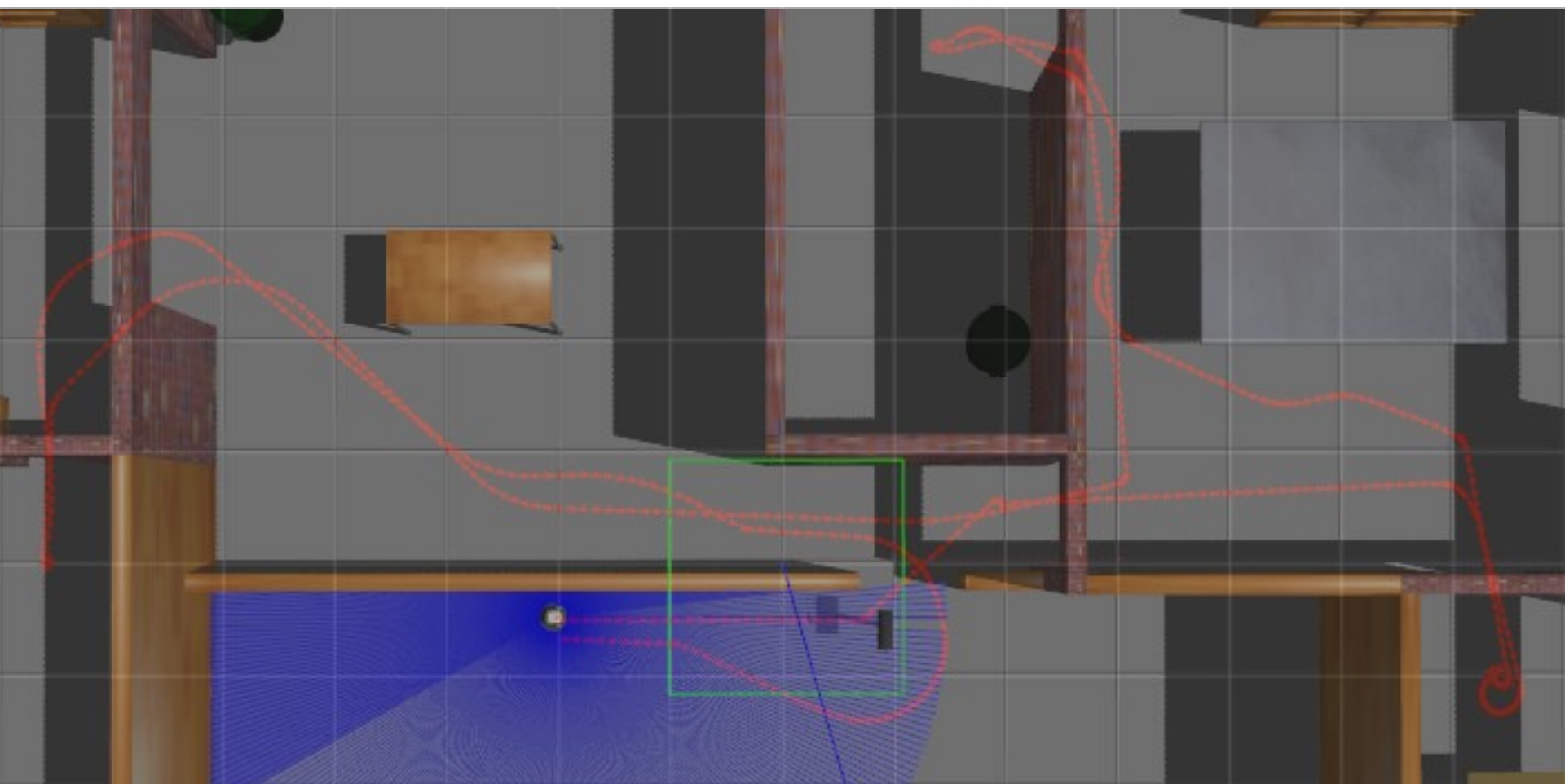
```
ros2 bag info <bagfile>
```

- Bag size
- Duration
- Number of messages
- Topic information:
 - topic
 - type
 - count

RViz

`rviz2 -d config.rviz`





Assignment

Submit to the Aula Virtual the following files:

- ZIP archive of bag folder
- script for launching ros2 bag play
- script for launching rviz
- rviz config file
- picture of the robot's trajectory on top of a simulator image

Recording and Playing

Recording

Gazebo

Teleop

ros2 bag **record** ...

rviz2

Playing

~~Gazebo~~

~~Teleop~~

ros2 bag **play** ...

rviz2

Mobile Robotics



Bachelor's
Degree in
Robotic
Intelligence

Teleoperation

TurtleBot3

https://emanual.robotis.com/docs/en/platform/turtlebot3/basic_operation/#teleoperation

ROBOTIS e-Manual

DYNAMIXELDYNAMIXEL SYSTEMEDUCATIONAL KITSSOFTWAREPARTSFAQ

Enter Search Terms

KineticMelodicNoeticDashingFoxyHumbleWindows

Edit on GitHub

TurtleBot3

1. Overview

2. Features

3. Quick Start Guide

3.1. PC Setup

3.2. SBC Setup

3.3. OpenCR Setup

3.4. Hardware Assembly

3.5. Bringup

3.6. Basic Operation

3.6.1. Teleoperation

3.6.1.1. Topic Monitor

4. SLAM

5. Navigation

6. Simulation

7. Manipulation

8. Autonomous Driving

9. Machine Learning

10. Examples

11. Friends(Locomotion)

12. Learn

3. 6. Basic Operation

3. 6. 1. Teleoperation

WARNING: Make sure to run the [Bringup](#) from the TurtleBot3 SBC before teleoperation. Teleoperate the robot, and be careful when testing the robot on the table as the robot might fall.

The TurtleBot3 can be teleoperated by various remote controllers. Make sure that the necessary ROS packages are supported for your SBC and ROS version.



TurtleBot3 08 Teleoperation Example

Share

Watch on YouTube

ROBOTIS RC100

3. 6. 1. 1. Keyboard

1. Open a terminal on **Remote PC**.

2. Run teleoperation node. Replace the `{${TB3_MODEL}}` with `burger` or `waffle` or `waffle_p1`, if TURTLEBOT3_MODEL parameter is not predefined.

```
$ export TURTLEBOT3_MODEL=${TB3_MODEL}
$ ros2 run turtlebot3_teleop teleop_keyboard
```

Task

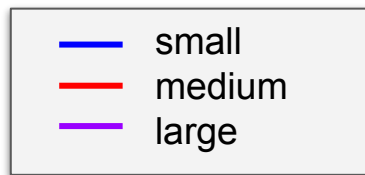
Teleoperate the robot
(monitor the /scan topic with rviz2)

Small / medium / large loop

Finish at the same initial location

Record /odom /tf /tf_static topics
with ros2 bag record

Plot trajectory in RViz



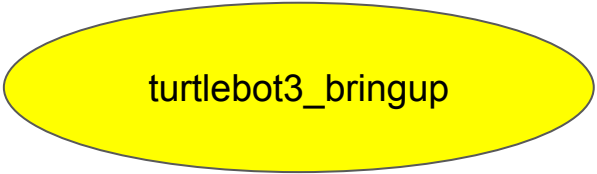
Types of connection to robot

Desktop PC in the laboratory (WiFi and router)

Laptop with native Ubuntu 22.04 via WiFi directly

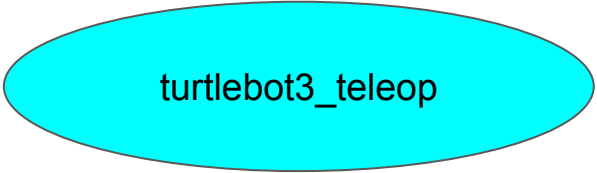
Laptop with native Ubuntu 22.04 via Ethernet cable

ROS nodes



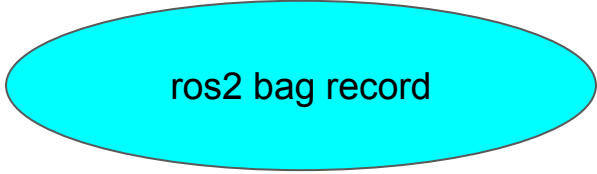
turtlebot3_bringup

Always runs in robot (ssh)



turtlebot3_teleop

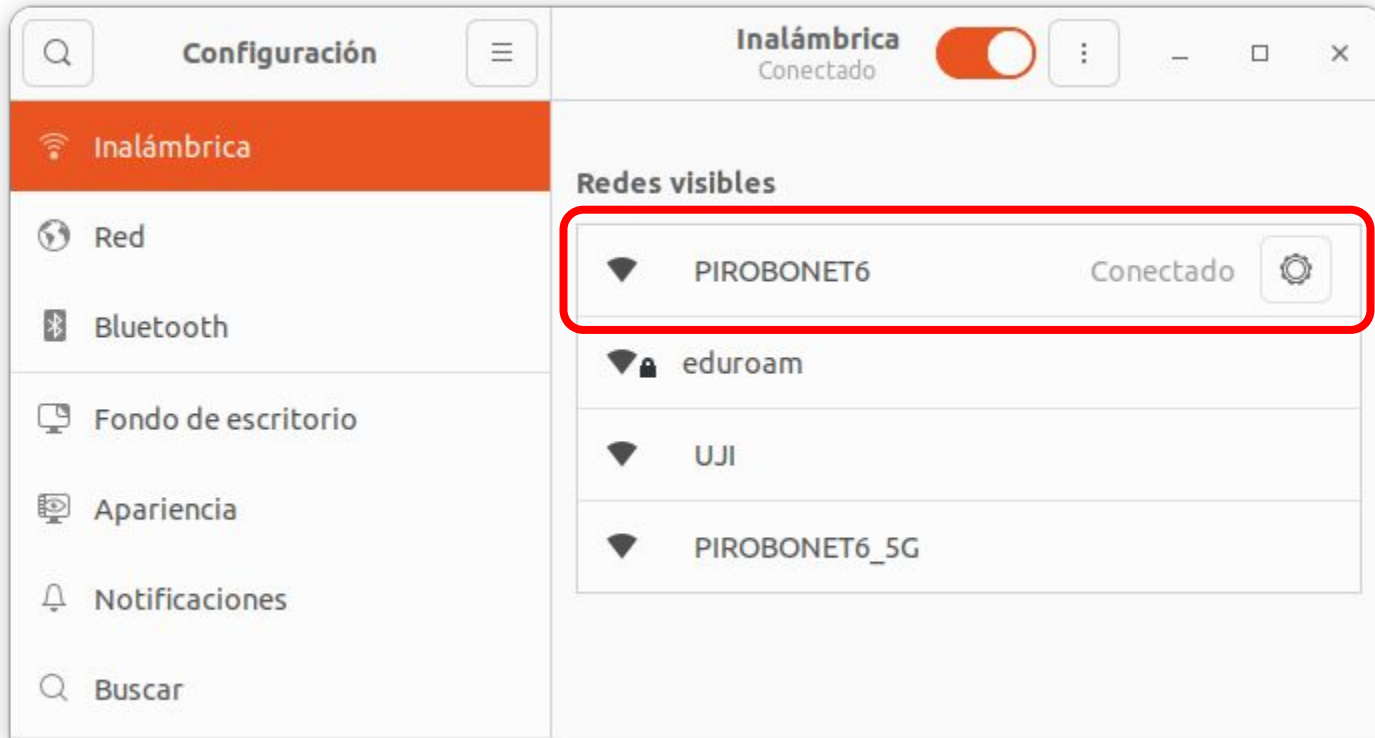
Can run in robot (ssh) or PC



ros2 bag record

Can run in robot (ssh) or PC

TurtleBot 3: WiFi connection from desktop PC



TurtleBot 3: WiFi connection from desktop PC

Open three terminals in the desktop PC.

Connect with ssh to the robot in the first terminal

Terminal 1 (robot): bring up the robot

Terminal 2 (PC): run teleop node

Terminal 3 (PC): record bag

Remember to set the appropriate **ROS_DOMAIN_ID** in every terminal.

Bringing up the TurtleBot 3

```
ssh ubuntu@192.168.0.xxx (password: turtlebot)
```

```
source /opt/ros/humble/setup.bash
```

```
export ROS_DOMAIN_ID=...
```

```
export TURTLEBOT3_MODEL=burger
```

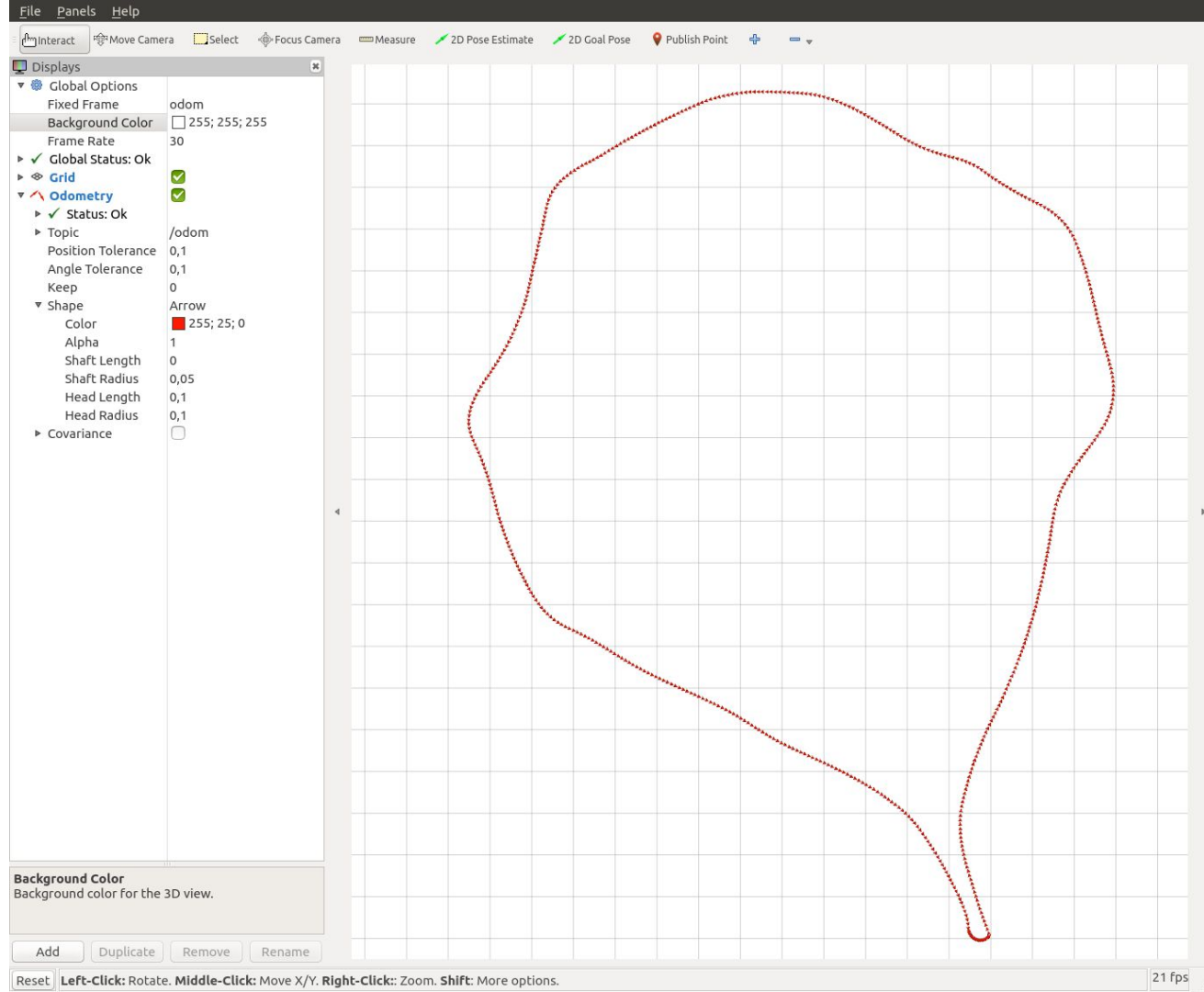
```
export LDS_MODEL=LDS-01 (in new ones LDS-02)
```

```
ros2 launch turtlebot3_bringup robot.launch.py
```

Monitor the /scan topic with rviz2

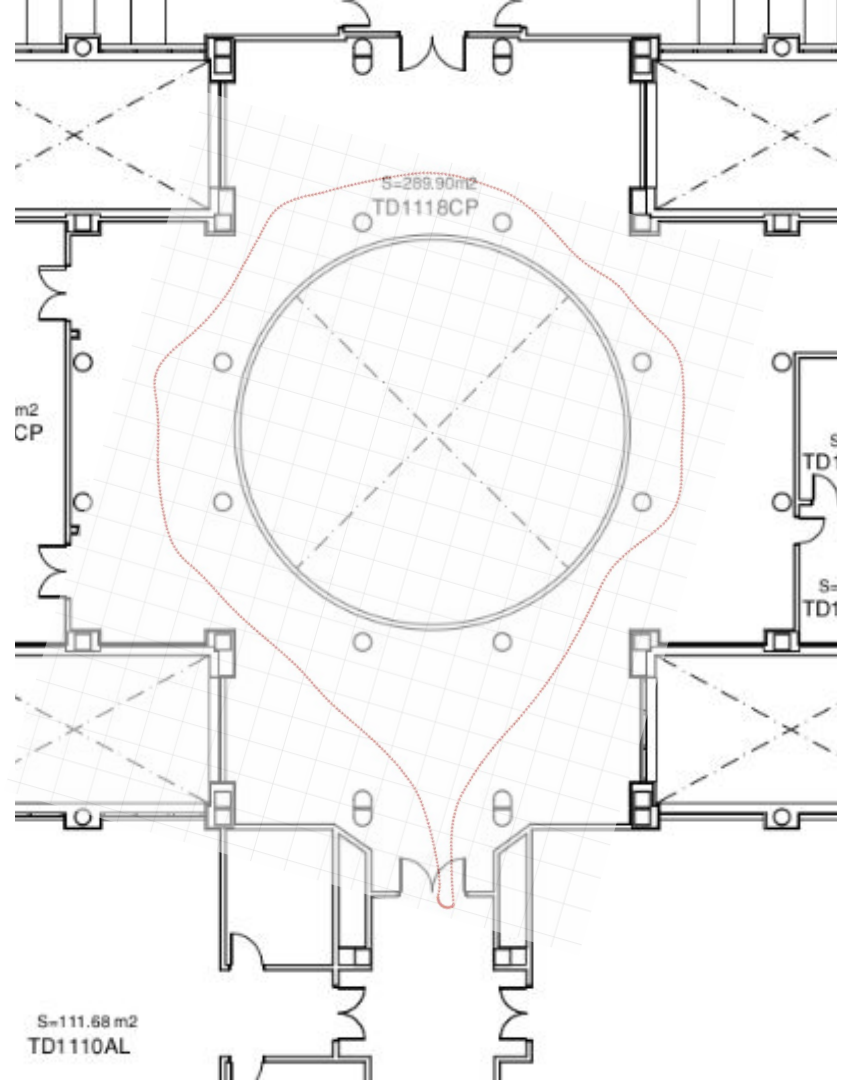
Launch rviz2 in your PC for detecting obstacles with the laser

Display the robot trajectory in RViz



Plot the trajectory on top of the map

You can get the maps
of the ESTCE buildings [here](#).



TurtleBot 3 + laptop: Ethernet wired connection

Configure the Ethernet connection of your laptop as [“Link-local”](#)

Switch off the WiFi connection of your laptop

Connect the Ethernet cable between the TurtleBot and your laptop

Find out the Ethernet IP address of the TurtleBot:

Connect with ssh from desktop PC to the public IP address of the robot

```
ssh ubuntu@192.168.0.XXX
```

Display all the network addresses of the robot

```
ip addr
```

Identify the Ethernet IP address of the robot (169.254.XXX.YYY)

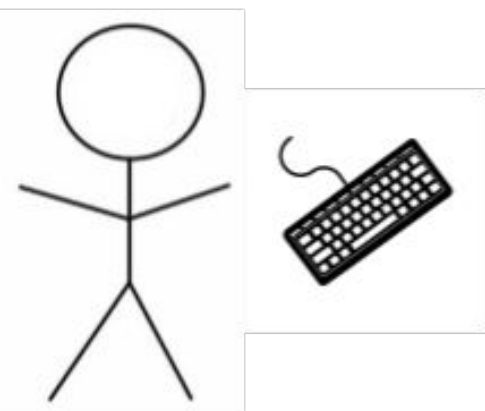
TurtleBot 3 + laptop: Ethernet wired connection

Connect with ssh from your laptop to the TurtleBot (Ethernet)

```
ssh ubuntu@169.254.XXX.YYY
```

Do the work!





/cmd_vel



/scan

/odom

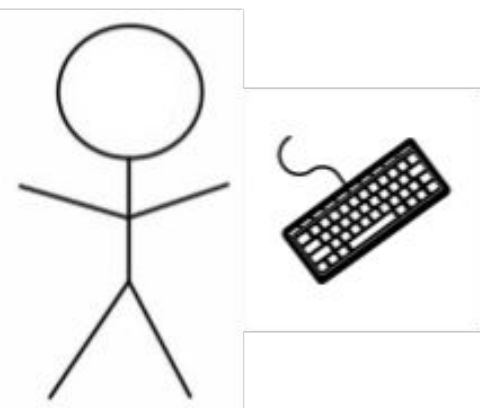
RQt

RViz

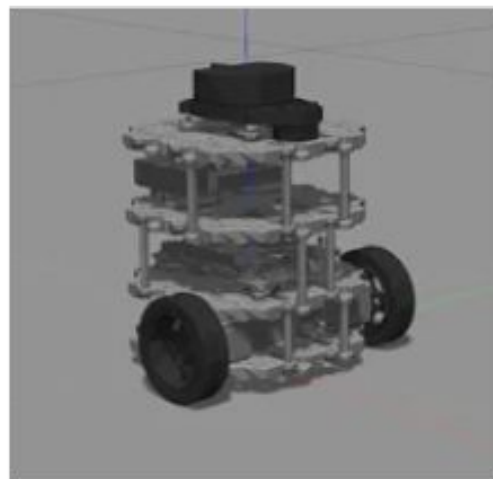
ros2 bag
record

```
ros2 run  
turtlebot3_teleop  
teleop_keyboard
```

```
ros2 launch  
turtlebot3_bringup  
robot.launch.py
```



/cmd_vel



/odom

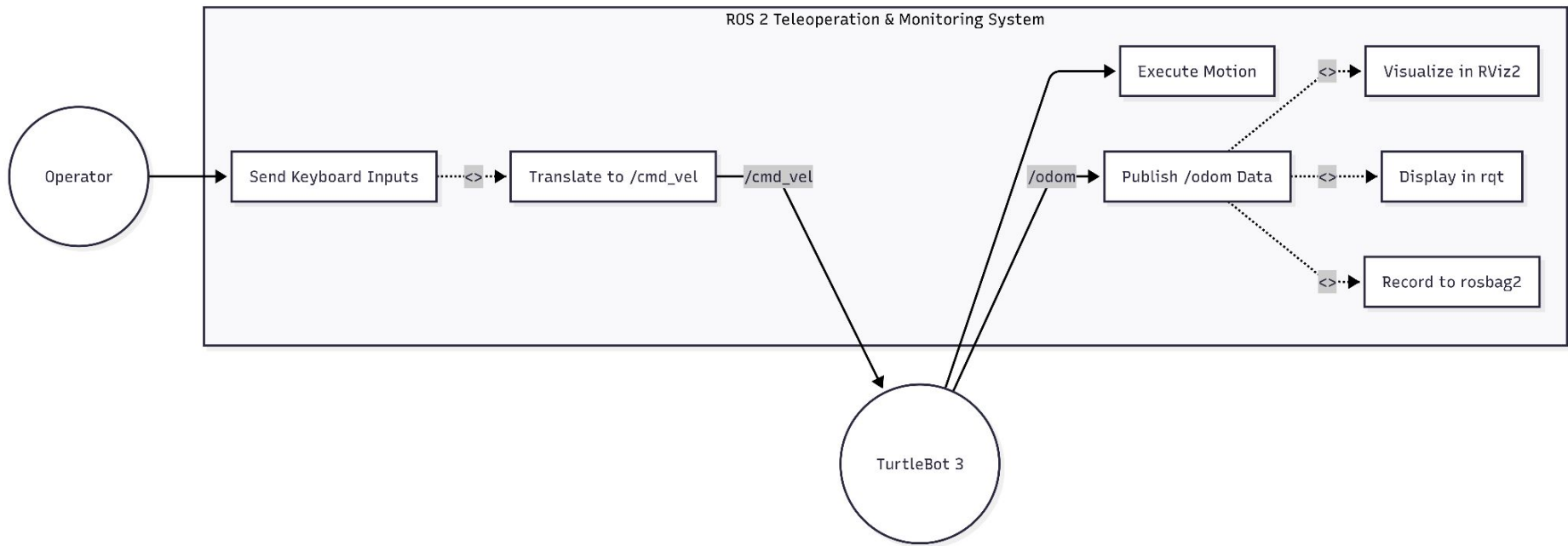
RQt

RViz

ros2 bag
record

```
ros2 run  
turtlebot3_teleop  
teleop_keyboard
```

```
ros2 launch  
turtlebot3_gazebo  
turtlebot3_house.launch.py
```



Extra Bonus: Use the TurtleBot 2

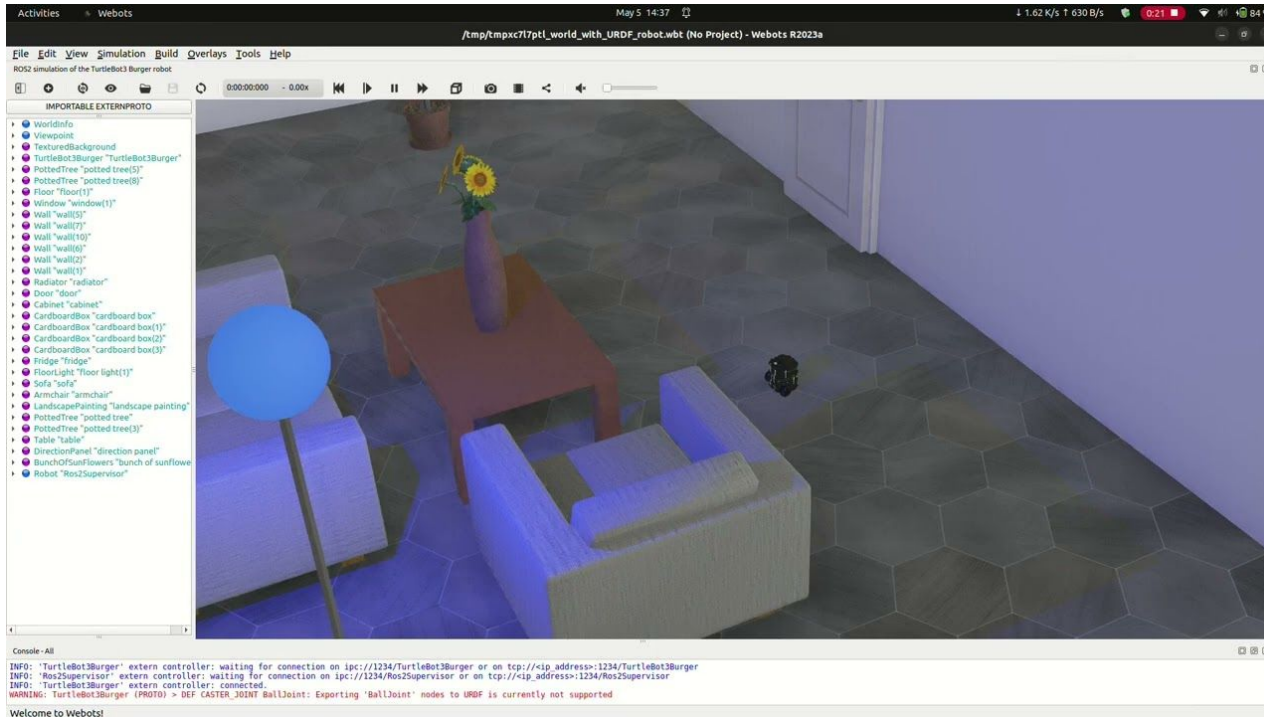
Use the [TurtleBot 2](#)
for the medium
or large tours.



Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence

Teleoperation
in Webots



Teleoperation of TurtleBot 3 in Webots

Prerequisites:

[Webots R2025a \(Linux tar.bz2\)](#) - [Installation instructions for the “tarball” package](#)

```
sudo apt install ros-humble-webots-ros2
```

Make a new script named `launch_tb3_webots.bash` with the following commands:

```
source /opt/ros/humble/setup.bash
export ROS_LOCALHOST_ONLY=1
export WEBOTS_HOME=$HOME/webots-R2025a/webots
ros2 launch webots_ros2_turtlebot robot_launch.py
```

Carpeta... rsonal / webots-R2025a / webots

★ Favoritos

Carpeta personal

Escritorio

Descargas

Documentos

Imágenes

Música

Videos

Papelera

+ Otras ubicaciones

Nombre	Tamaño	Modificación	
 bin	3 elementos	4 Feb 2025	☆
 docs	1 elemento	4 Feb 2025	☆
 include	4 elementos	4 Feb 2025	☆
 lib	2 elementos	4 Feb 2025	☆
 projects	14 elementos	4 Feb 2025	☆
 resources	23 elementos	4 Feb 2025	☆
 scripts	1 elemento	4 Feb 2025	☆
 src	1 elemento	4 Feb 2025	☆
 webots	2.8 kB	4 Feb 2025	☆
 webots-controller	35.5 kB	4 Feb 2025	☆
 .nautilus-metafile.xml	498 bytes	4 Feb 2025	☆

Teleoperation of TurtleBot 3 in Webots

Prerequisites:

[Webots R2025a](#) - [Installation instructions for the “tarball” package](#)

```
sudo apt install ros-jazzy-webots-ros2
```

Make a new script named `launch_tb3_webots.bash` with the following commands:

```
source /opt/ros/humble/setup.bash
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
export WEBOTS_HOME=$HOME/webots-R2025a
ros2 launch webots_ros2_turtlebot robot_launch.py
```

Homework

Teleoperate the robot

Visit the room

Finish at the same initial location

Record topics with `ros2 bag record`

Plot trajectory in RViz

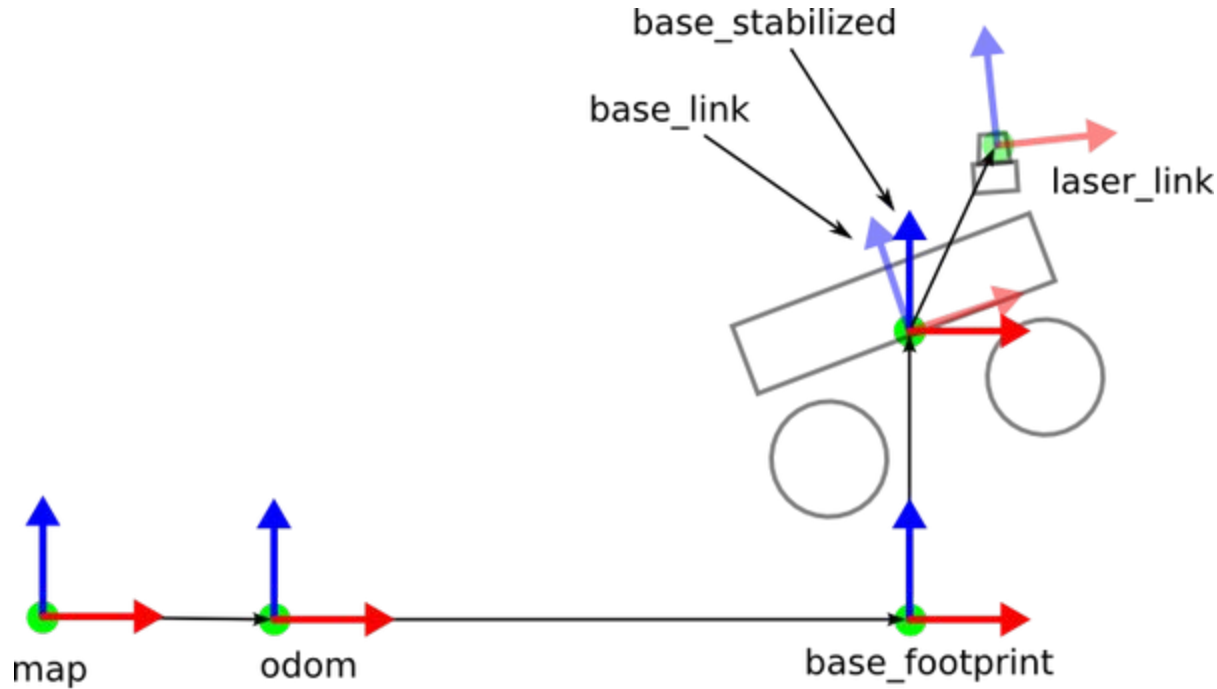
Assignment

Submit to the Virtual Classroom the following files:

- ZIP archive of bag folder
- script for launching ros2 bag play
- script for launching rviz
- rviz config file
- picture of the robot's trajectory on top of a simulator image

Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence



Frames

Odometry

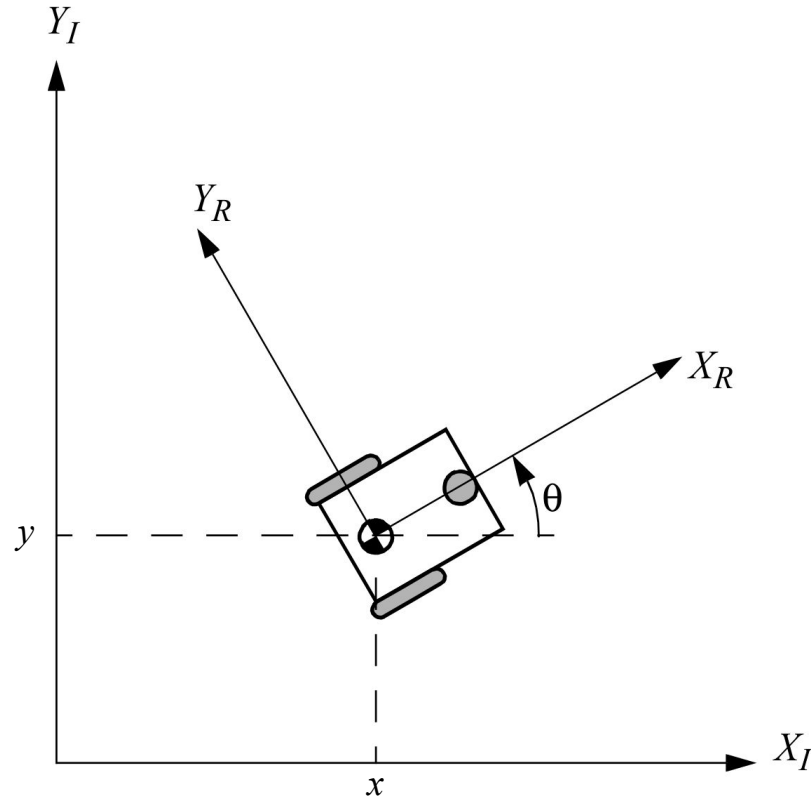
Odometry is the use of data from motion sensors to estimate change in position over time.

It is used by some legged or wheeled robots to estimate their position relative to a starting location.

This method is sensitive to errors due to the integration of velocity measurements over time to give position estimates.

[\[Wikipedia\]](#)

Pose: position + orientation wrt a frame of reference



Topic	Type	Bandwidth	Hz	Value
▼  /odom	nav_msgs/msg/Odometry	unknown	28.99	
child_frame_id	string			'base_footprint'
▼ header	std_msgs/Header			
frame_id	string			'odom'
▶ stamp	builtin_interfaces/Time			
▶ pose	geometry_msgs/PoseWithCovariance			
▶ twist	geometry_msgs/TwistWithCovariance			

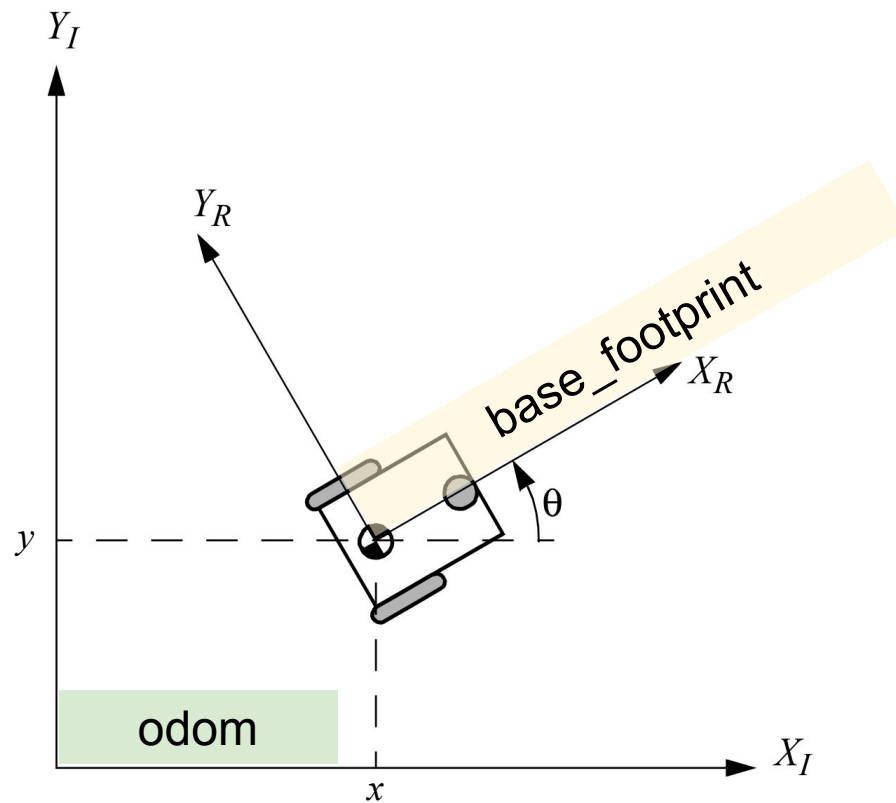
2. The nav_msgs/Odometry Message

The nav_msgs/Odometry message stores an estimate of the position and velocity of a robot in free space:

```
# This represents an estimate of a position and velocity in free space.
# The pose in this message should be specified in the coordinate frame given by header.frame_id.
# The twist in this message should be specified in the coordinate frame given by the child_frame_id
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

The pose in this message corresponds to the estimated position of the robot in the odometric frame along with an optional covariance for the certainty of that pose estimate. The twist in this message corresponds to the robot's velocity in the child frame, normally the coordinate frame of the mobile base, along with an optional covariance for the certainty of that velocity estimate.

ROS frames of reference



Specification - [REP 105](#)

Coordinate Frames

base_link

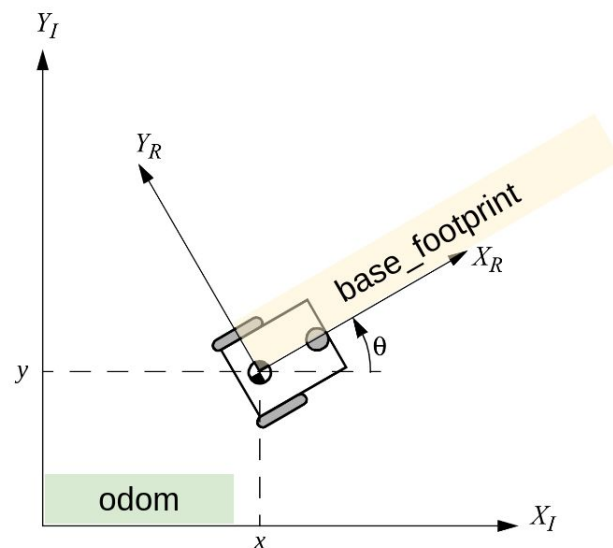
The coordinate frame called `base_link` is rigidly attached to the mobile robot base. The `base_link` can be attached to the base in any arbitrary position or orientation; for every hardware platform there will be a different place on the base that provides an obvious point of reference. Note that REP 103 [1] specifies a preferred orientation for frames.

odom

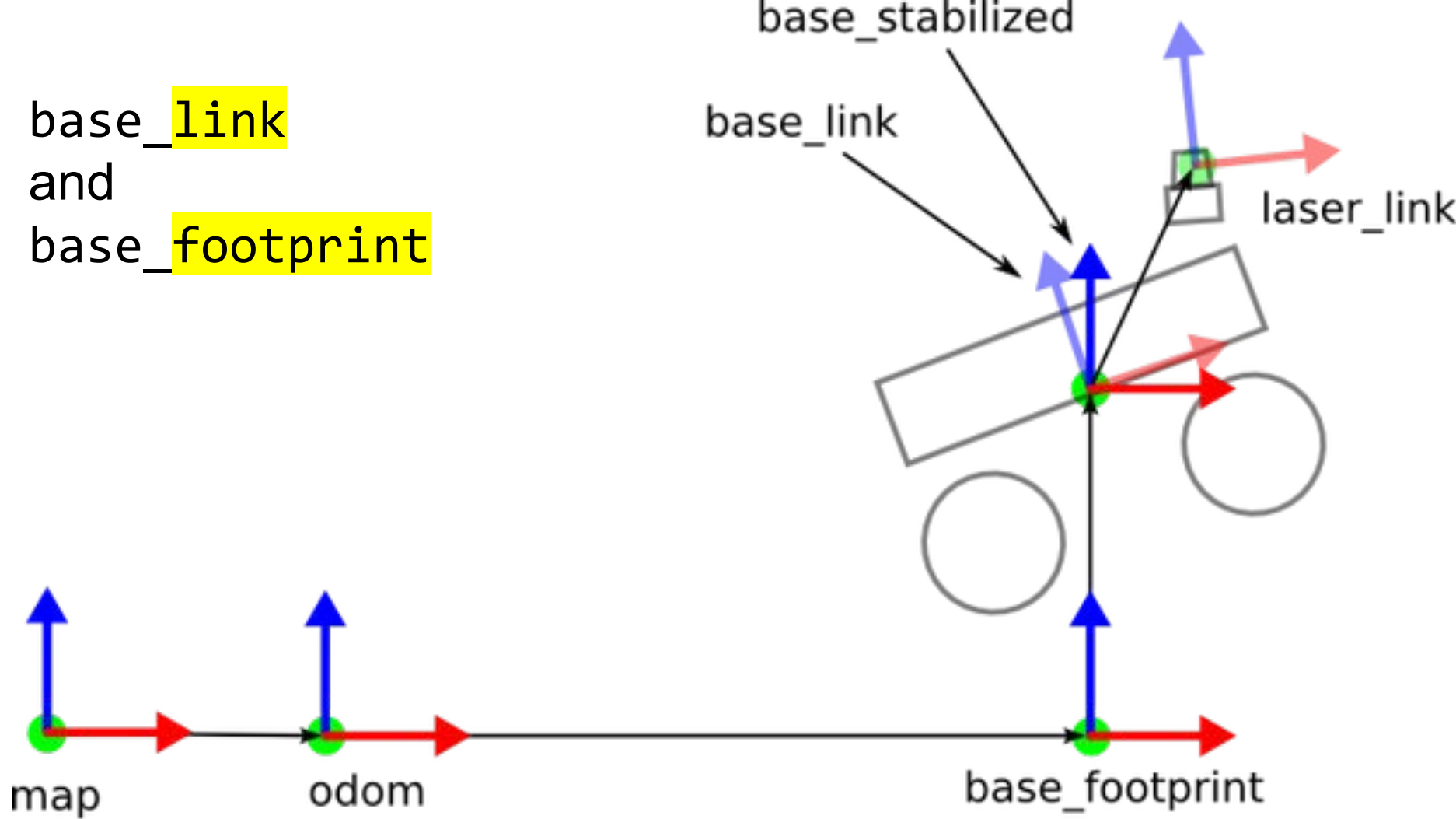
The coordinate frame called `odom` is a world-fixed frame. The pose of a mobile platform in the `odom` frame can drift over time, without any bounds. This drift makes the `odom` frame useless as a long-term global reference. However, the pose of a robot in the `odom` frame is guaranteed to be continuous, meaning that the pose of a mobile platform in the `odom` frame always evolves in a smooth way, without discrete jumps.

In a typical setup the `odom` frame is computed based on an odometry source, such as wheel odometry, visual odometry or an inertial measurement unit.

The `odom` frame is useful as an accurate, short-term local reference, but drift makes it a poor frame for long-term reference.



base_link
and
base_footprint



Coordinate frames

- **map** frame has its origin at some arbitrarily chosen point in the world.
This coordinate frame is fixed in the world.
- **odom** frame has its origin at the point where the robot is initialized.
This coordinate frame is fixed in the world.
- **base_footprint** has its origin directly under the center of the robot.
It is the 2D pose of the robot.
This coordinate frame moves as the robot moves.
- **base_link** has its origin directly at the pivot point or center of the robot.
This coordinate frame moves as the robot moves.

Coordinate frame conventions - [REP 103](#)

All coordinate frames should follow these conventions.

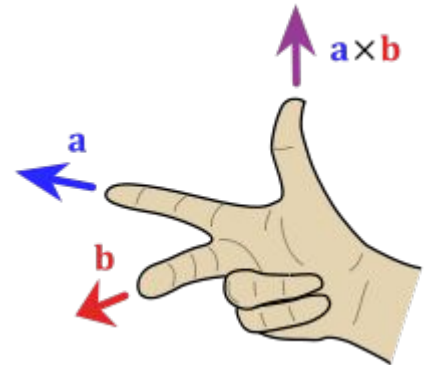
Chirality

All systems are right handed. This means they comply with the [right hand rule](#) [4].

Axis Orientation

In relation to a body the standard is:

- x forward
- y left
- z up



Coordinate frame conventions - [REP 103](#)

Rotation Representation

There are many ways to represent rotations. The preferred order is listed below, along with rationale.

1. quaternion

- Compact representation
- No singularities

2. rotation matrix

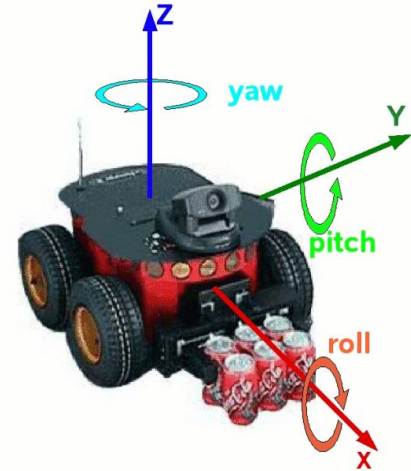
- No singularities

3. fixed axis roll, pitch, yaw about X, Y, Z axes respectively

- No ambiguity on order
- Used for angular velocities

4. euler angles yaw, pitch, and roll about Z, Y, X axes respectively

- Euler angles are generally discouraged due to having 24 'valid' conventions with different domains using different conventions by default.

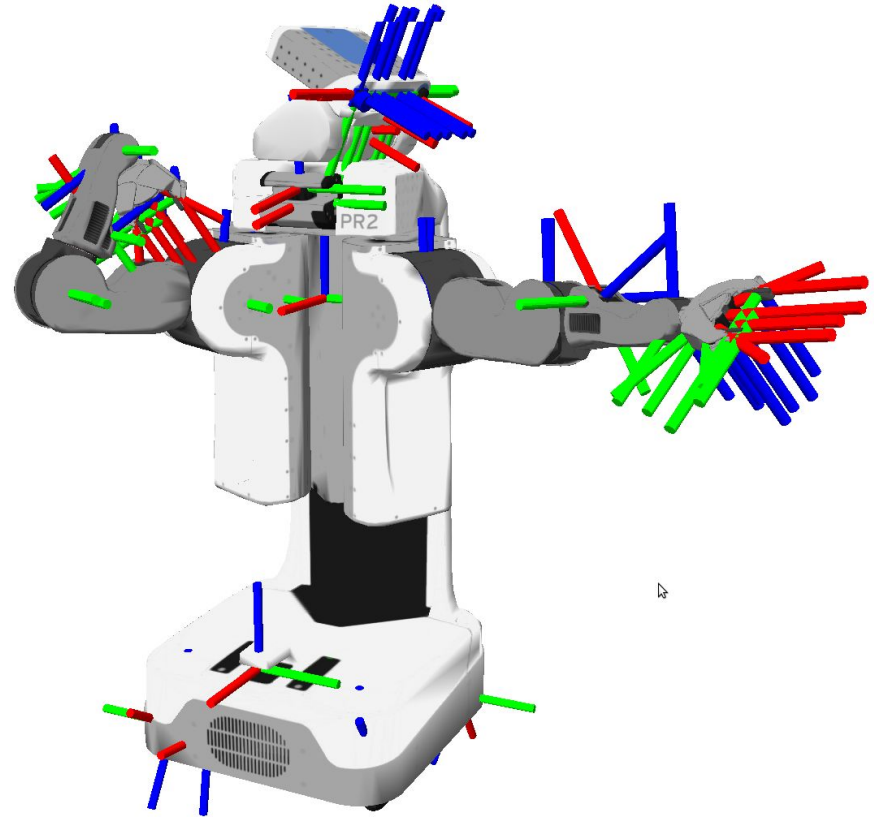


ROS tf2 library and tools

A robotic system typically has many 3D coordinate frames that change over time, such as a world frame, base frame, gripper frame, head frame, etc.

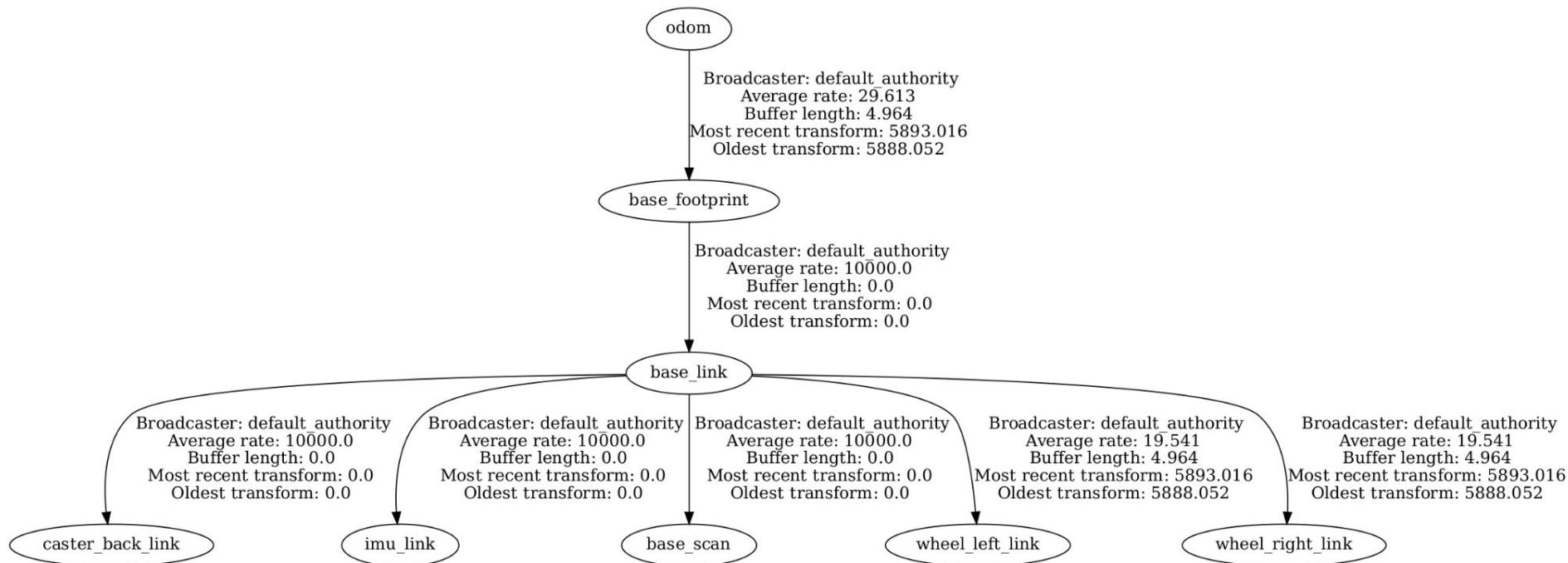
tf2 keeps track of all these frames over time, and allows you to ask questions like:

- Where was the head frame relative to the world frame, 5 seconds ago?
- What is the pose of the object in my gripper relative to my base?
- What is the current pose of the base frame in the map frame?



ros2 run tf2_tools view_frames

view_frames Result
Recorded at time: 1694688033.0883667



```
ros2 run tf2_ros tf2_echo \  
    [source_frame] [target_frame]
```

tf2_echo reports the transform
between any two frames
broadcasted over ROS.

```
ros2 run tf2_ros tf2_echo odom base_footprint
```

At time 9545.330000000

- Translation: [-1.999, -0.500, 0.009]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

At time 9546.316000000

- Translation: [-1.999, -0.500, 0.009]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

At time 9547.302000000

- Translation: [-1.999, -0.500, 0.009]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

```
ros2 run tf2_ros tf2_echo odom base_link
```

At time 9532.274000000

- Translation: [-1.999, -0.500, 0.019]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

At time 9533.260000000

- Translation: [-1.999, -0.500, 0.019]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

At time 9534.246000000

- Translation: [-1.999, -0.500, 0.019]
- Rotation: in Quaternion [0.000, 0.003, 0.000, 1.000]

```
ros2 run tf2_ros tf2_echo \  
base_footprint base_link
```

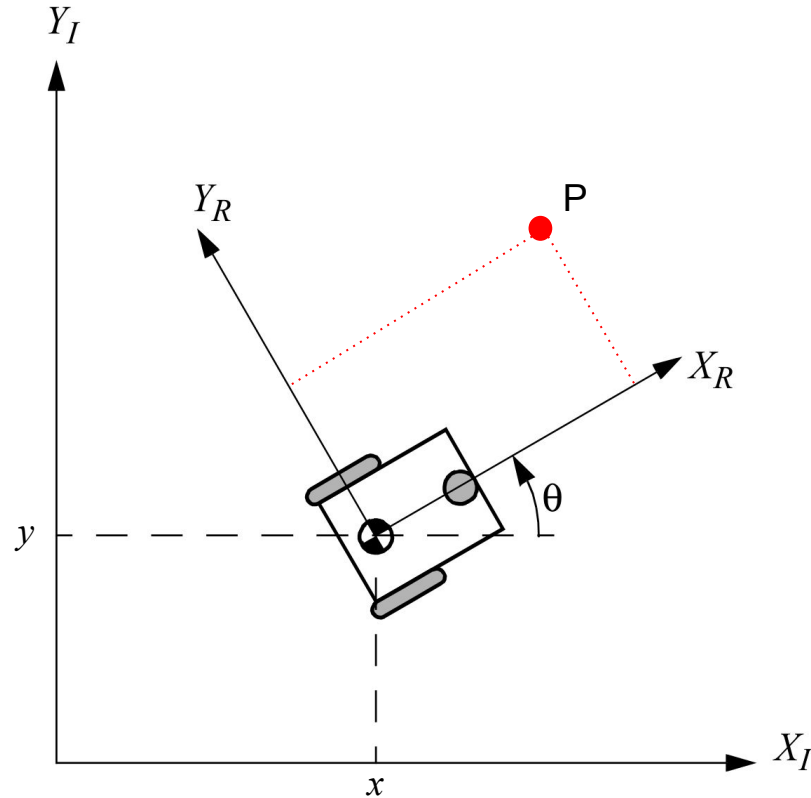
At time 0.0

- Translation: [0.000, 0.000, 0.010]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 0.000, 1.000]

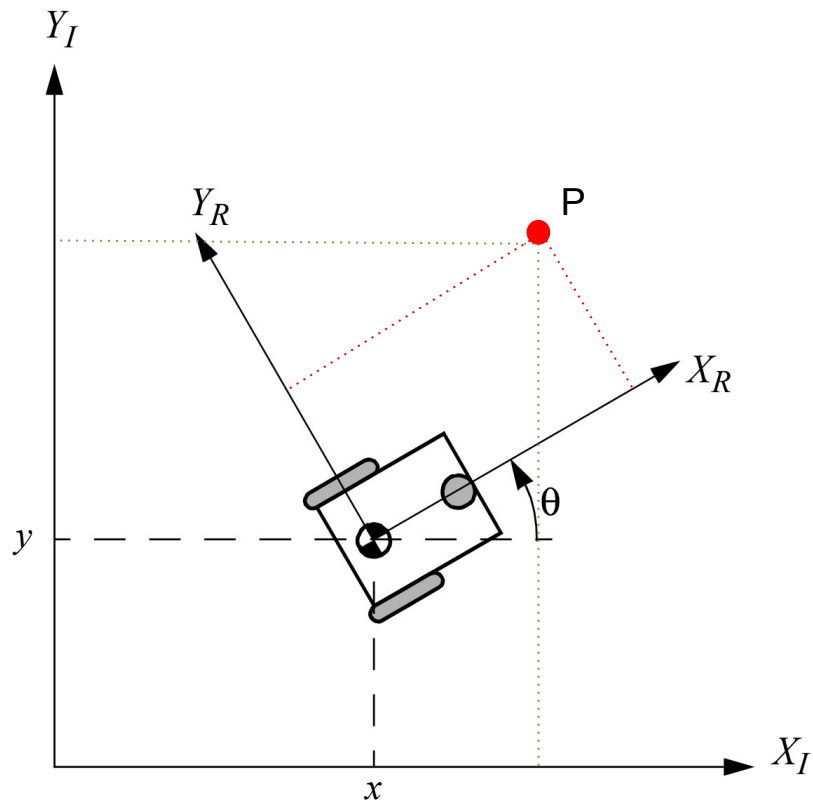
At time 0.0

- Translation: [0.000, 0.000, 0.010]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 0.000, 1.000]

Problem: coordinates of a point in the robot/fixed frames



$${}^I\mathbf{p} \leftarrow {}^R\mathbf{p}$$



$$I_P = I_{T_R} \cdot R_P$$

$I_P \rightarrow R_P \quad ?$

Statement of the problem

/odom topic gives the pose of the robot in the “odom” frame.

Given a map, an origin of coordinates in the map, and an initial robot pose:

- Compute the map -> odom transform
- Visualize the robot in the origin of the map

References

[Introduction to Autonomous Mobile Robots](#)

[Publishing Odometry Information Over ROS](#)

[REP 103 -- Standard Units of Measure and Coordinate Conventions \(ROS.org\)](#)

[REP 105 -- Coordinate Frames for Mobile Platforms \(ROS.org\)](#)

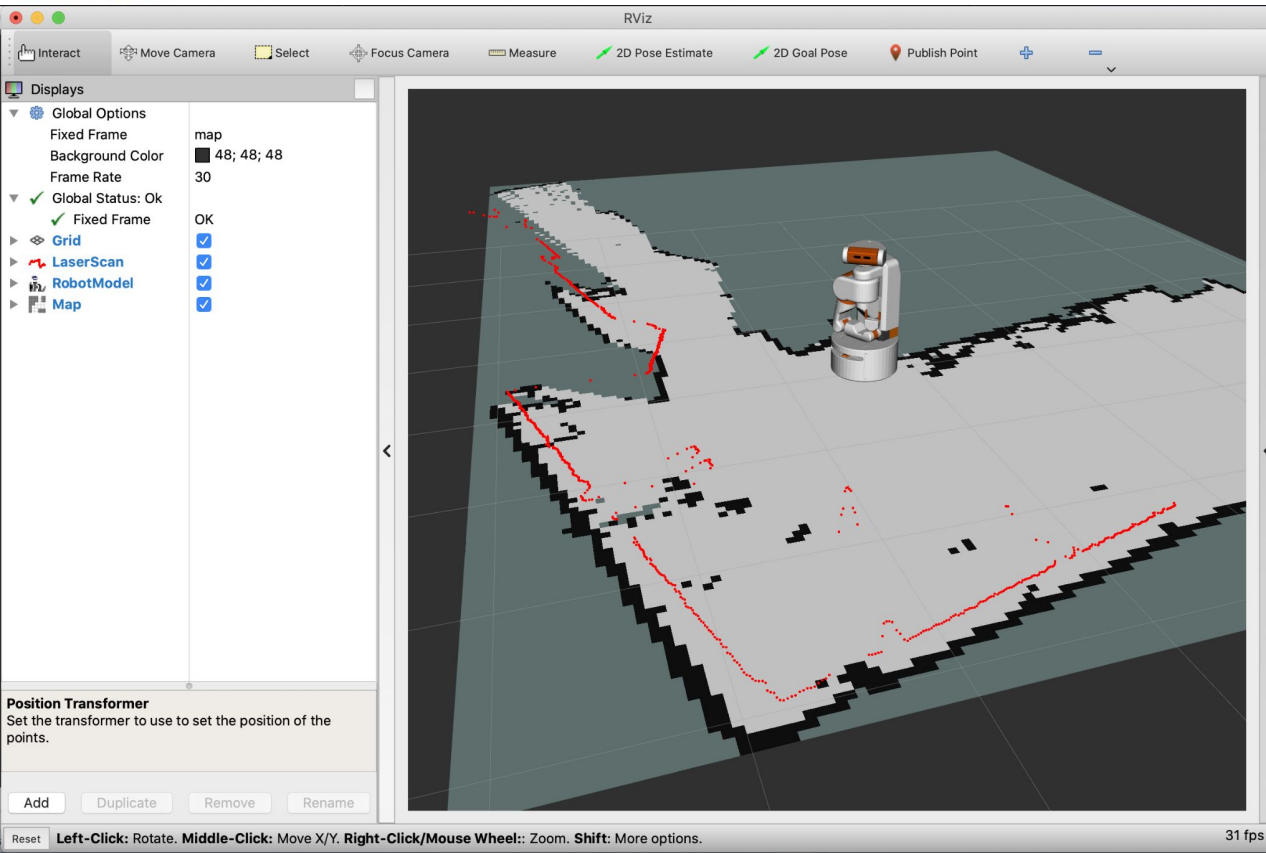
[Coordinate Frames and Transforms for ROS-based Mobile Robots](#)

[Introducing tf2 — ROS 2 Documentation: Humble documentation](#)

Mobile Robotics

Bachelor's
Degree in
Robotic
Intelligence

Maps



Maps as Occupancy grids

Occupancy grids are stored in a pair of files.

The YAML file describes the map meta-data,
and names the image file.

The image file encodes the occupancy data.

The image describes the occupancy state of each cell of the world in the color of the corresponding pixel. In the standard configuration, whiter pixels are free, blacker pixels are occupied, and pixels in between are unknown.

YAML format

Required fields:

- **image** : Path to the image file containing the occupancy data; can be absolute, or relative to the location of the YAML file
- **resolution** : Resolution of the map, meters / pixel
- **origin** : The 2-D pose of the lower-left pixel in the map, as (x, y, yaw), with yaw as counterclockwise rotation (yaw=0 means no rotation).
- **occupied_thresh** : Pixels with occupancy probability greater than this threshold are considered completely occupied.
- **free_thresh** : Pixels with occupancy probability less than this threshold are considered completely free.
- **negate** : Whether the white/black free/occupied semantics should be reversed (interpretation of thresholds is unaffected)

Optional parameter:

- **mode** : Can have one of three values: trinary, scale, or raw. Trinary is the default.

Value Interpretation (1)

Given a pixel that has a value x in the range $[0, 256)$, how should we interpret this value when put into the ROS message? First we convert integer x to a floating point number p depending on the interpretation of the `negate` flag from the `yaml`.

- If `negate` is false, $p = (255 - x) / 255.0$.
This means that black (0) now has the highest value (1.0) and white (255) has the lowest (0.0).
- If `negate` is true, $p = x / 255.0$. This is the non-standard interpretation of images, which is why it is called `negate`, even though the math indicates that x is not negated (nomenclature is hard).

Value Interpretation (2)

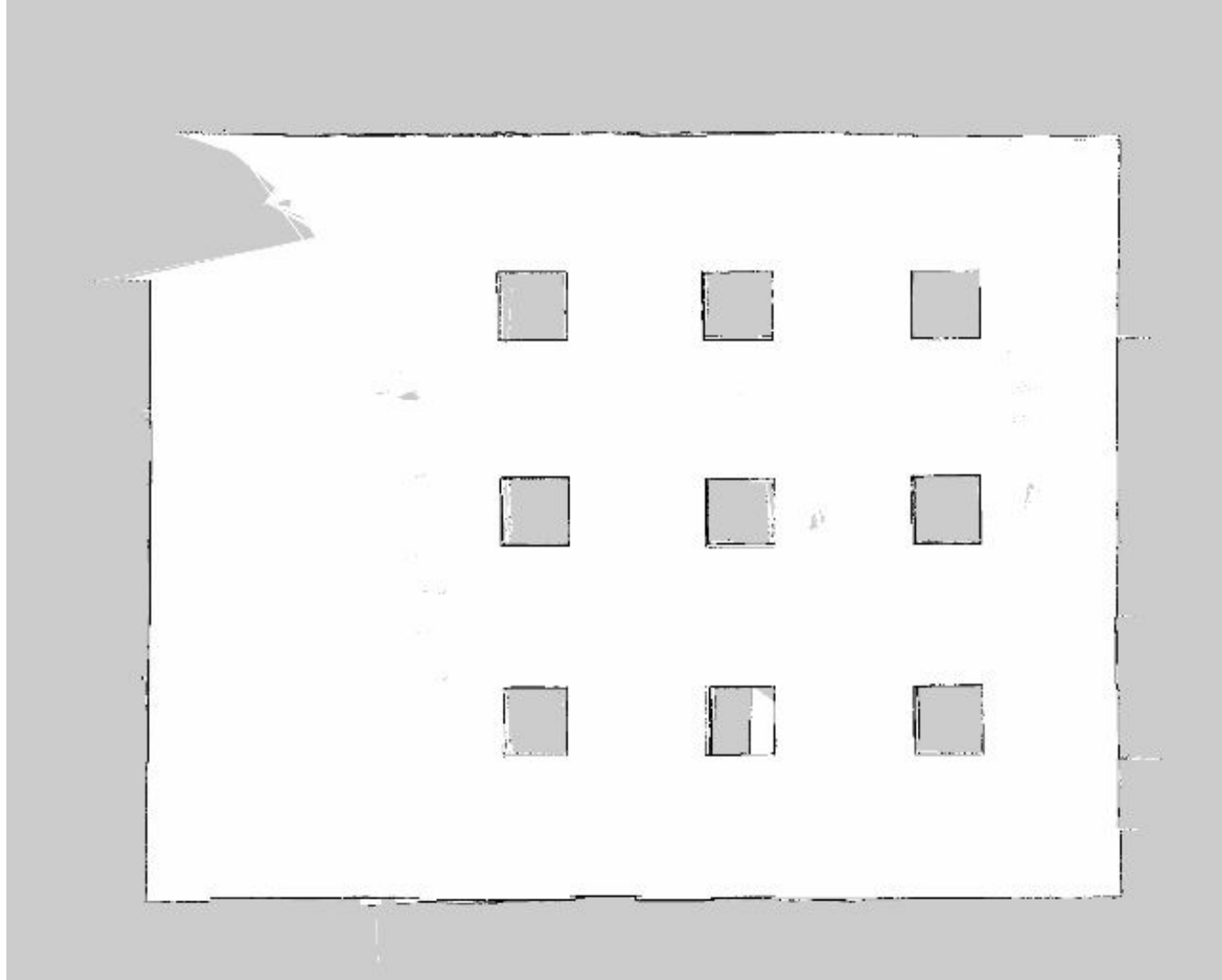
The standard interpretation is the trinary interpretation,
i.e. interpret all values so that the output ends up being one of three values:

- If $p > \text{occupied_thresh}$, output the value 100 to indicate the cell is **occupied**.
- If $p < \text{free_thresh}$, output the value 0 to indicate the cell is **free**.
- Otherwise, output -1 a.k.a. 255 (as an unsigned char), to indicate that the cell is **unknown**.

 occupied

 free

 unknown



Value Interpretation (3)

Scale mode

This tweaks the above interpretation to allow for more output values than trinary.

- As before if $p > \text{occupied_thresh}$, output the value 100.
If $p < \text{free_thresh}$, output the value 0.
- Otherwise, output $99 * (p - \text{free_thresh}) / (\text{occupied_thresh} - \text{free_thresh})$

This will allow you to output a full gradient of values ranging from $[0, 100]$. To output -1, simply use the alpha channel of a png, where any transparency will be interpreted as unknown.

Raw mode

This mode will output x for each pixel, so output values are $[0, 255]$.

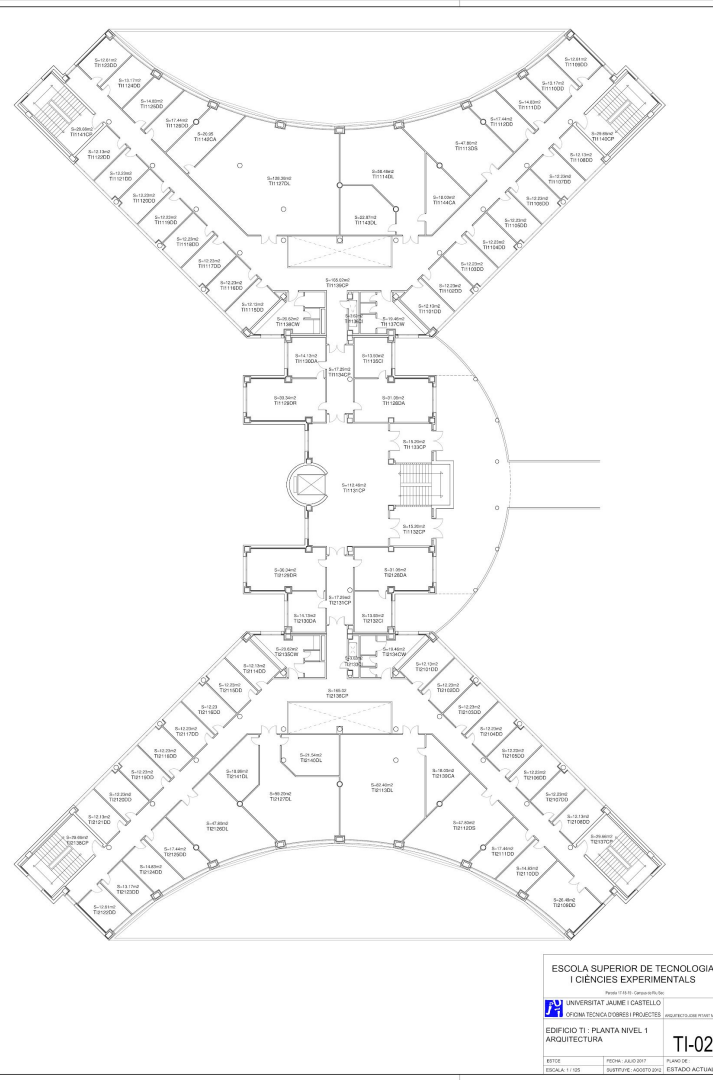
Task

Convert architectural map to ROS map:

1. Convert PDF to image
2. Adjust image size (optional)
3. Compute resolution
4. Edit image (delete names, doors, etc.)

TI_n1.pdf

Edificis de l'Escola Superior de Tecnologia i Ciències Experimentals





ESTADO ACTUAL

Tl_n1.pdf

Format: PDF-1.7

Paper size: A1, vertical (594 × 841 mm)

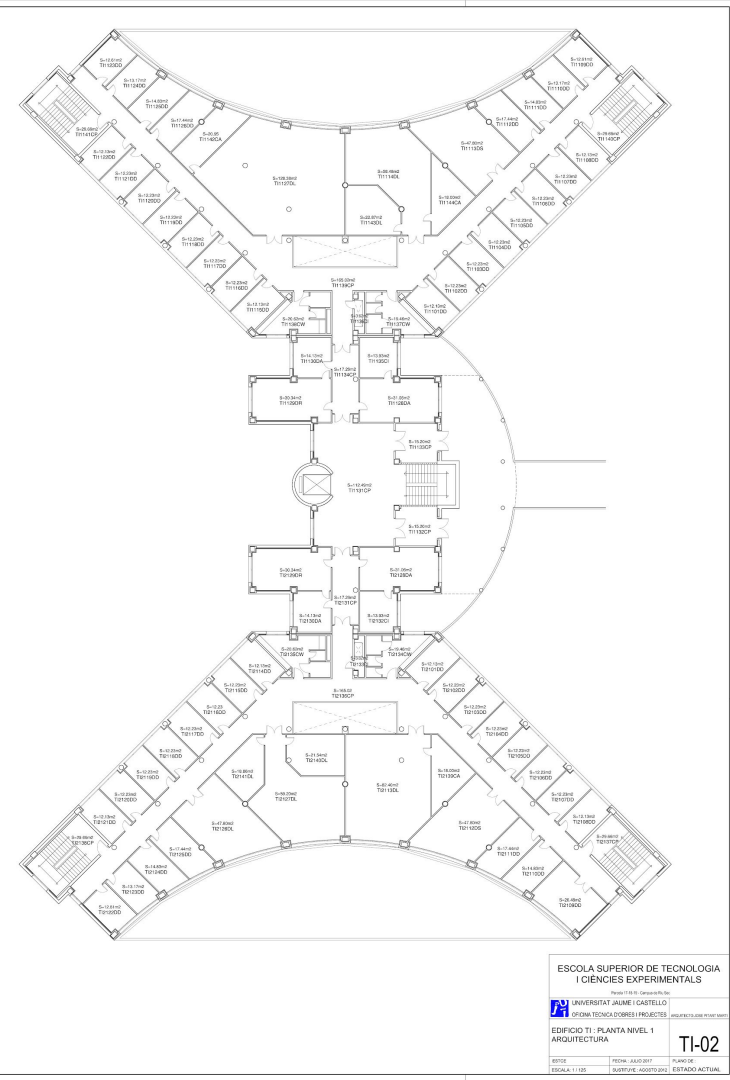
Scale: 1/125

PDF to PNG

Format: png (PNG)

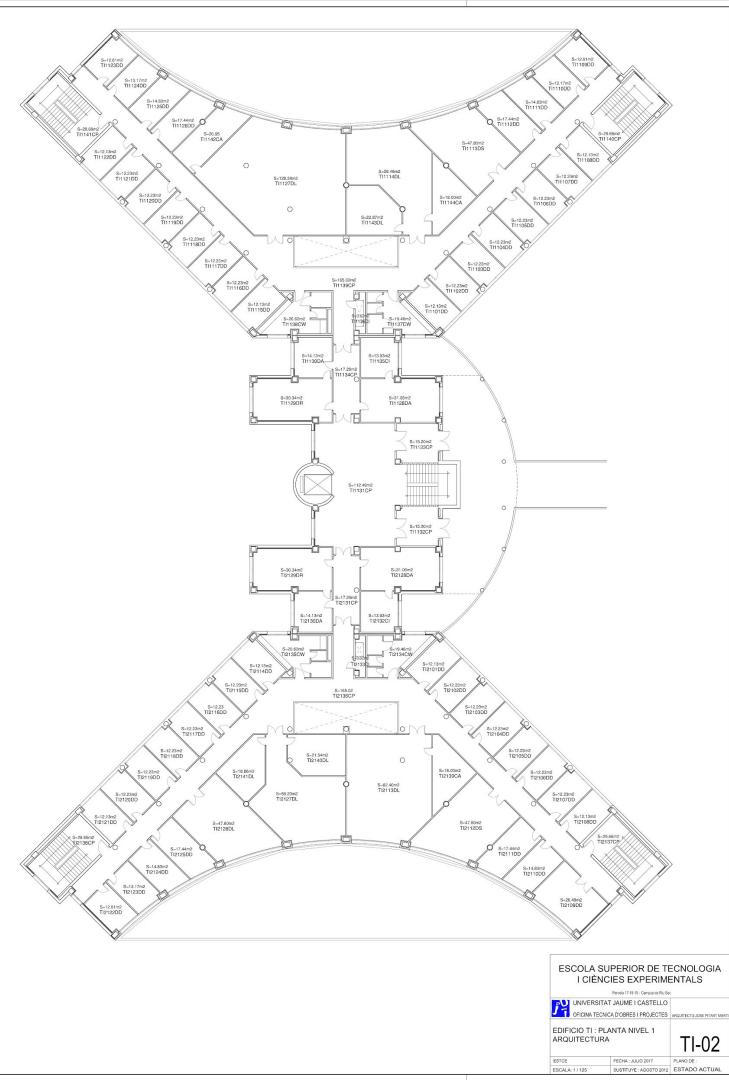
Width: 4678 px

Height: 6623 px



TI_n1.yaml

```
image: ./TI_n1.png
mode: trinary
resolution: meters / pixel ???
origin: [ , , 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```



TI_n1.yaml

image: ./TI_n1.png

mode: trinary

resolution:

horizontal: $0.594 * 125 / 4678$

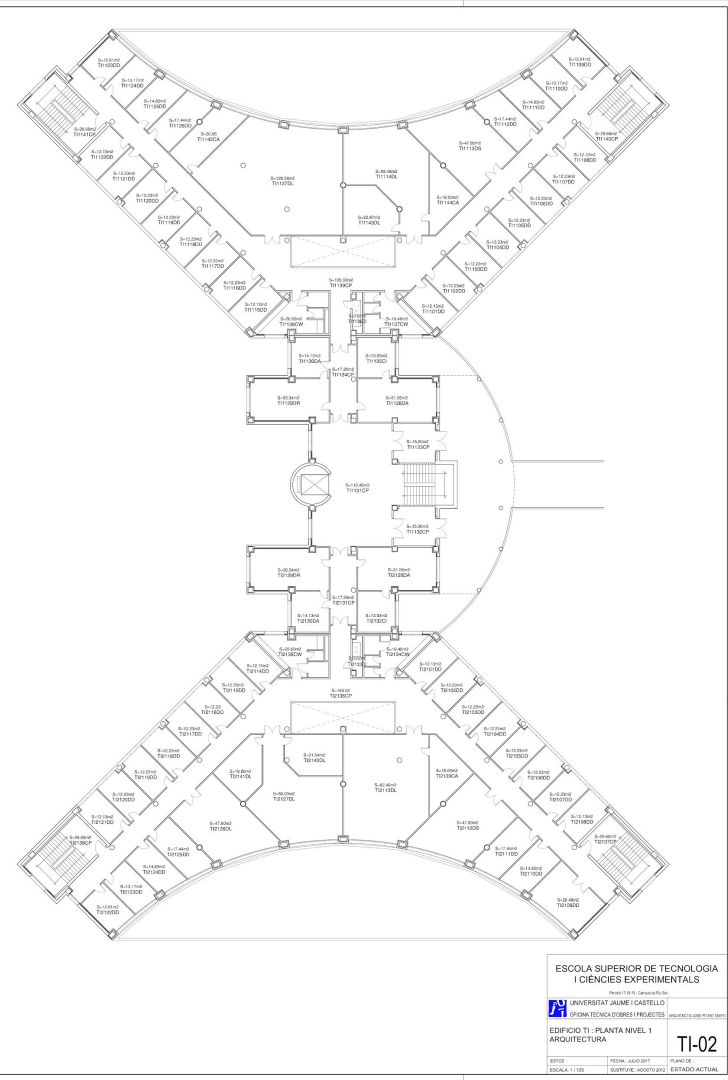
vertical: $0.841 * 125 / 6623$

origin: [, , 0]

negate: 0

occupied_thresh: 0.65

free_thresh: 0.25



TI_n1.yaml

image: ./TI_n1.png

mode: trinary

resolution:

horizontal: $0.594 * 125 / 4678 = 0.0159$

vertical: $0.841 * 125 / 6623 = 0.0159$

origin: [, , 0]

negate: 0

occupied_thresh: 0.65

free_thresh: 0.25

Parameters

Map Server

Implements the server for publishing a map topic.

yaml_filename

Type	Default
string	N/A

Description

Path to map yaml file.

topic_name

Type	Default
string	"map"

Description

Topic to publish loaded map to.

frame_id

Type	Default
string	"map"

Description

Frame to publish loaded map in.

Publish and visualize the map of building T1

Download archive [Worlds.zip](#)

Uncompress it in your IR2121 repository folder.

Open a terminal window, cd to `Worlds/scripts` and run:

```
./visualize_map.bash
```

Open a terminal window, cd to `Worlds/scripts` and run:

```
./publish_map.bash
```

Interact

Move Camera

Select

Focus Camera

Measure

2D Pose Estimate

2D Goal Pose

Publish Point

+

-

Displays



Global Options

Fixed Frame

Background Color

map

48; 48; 48

Frame Rate

30

Global Status: W...

Fixed Frame

No tf data. Actual erro...

Grid

Map

Status: Ok

Topic

/map

Update Topic

/map_updates

Alpha

0,7

Color Scheme

map

Draw Behind

☐

Resolution

0,0159

Width

4678

Height

6623

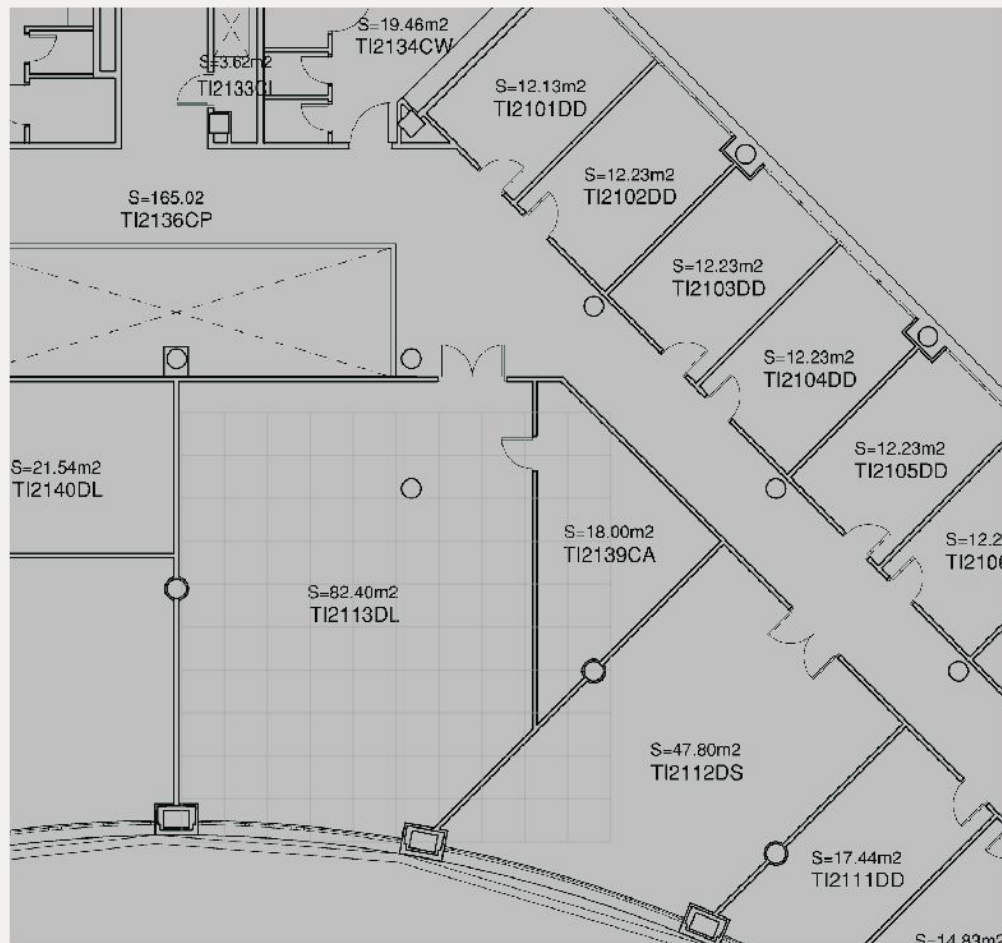
Position

-41.5; -28; 0

Orientation

0; 0; 0; 1

Use Timestamp

☐

Add

Duplicate

Remove

Rename

Reset

30 fps

Gazebo world

Exit the previous ROS scripts, and close those terminals.

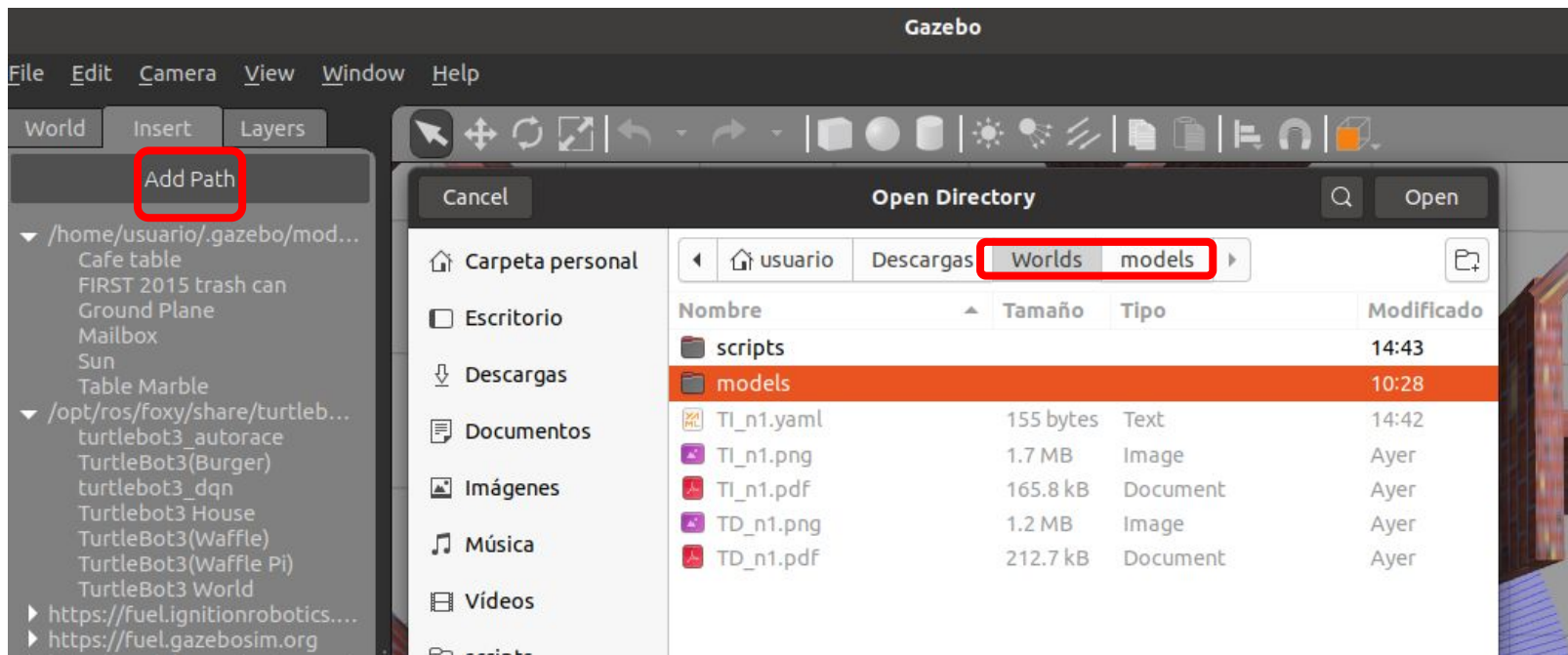
Open a new terminal window,

cd to `Worlds/scripts`

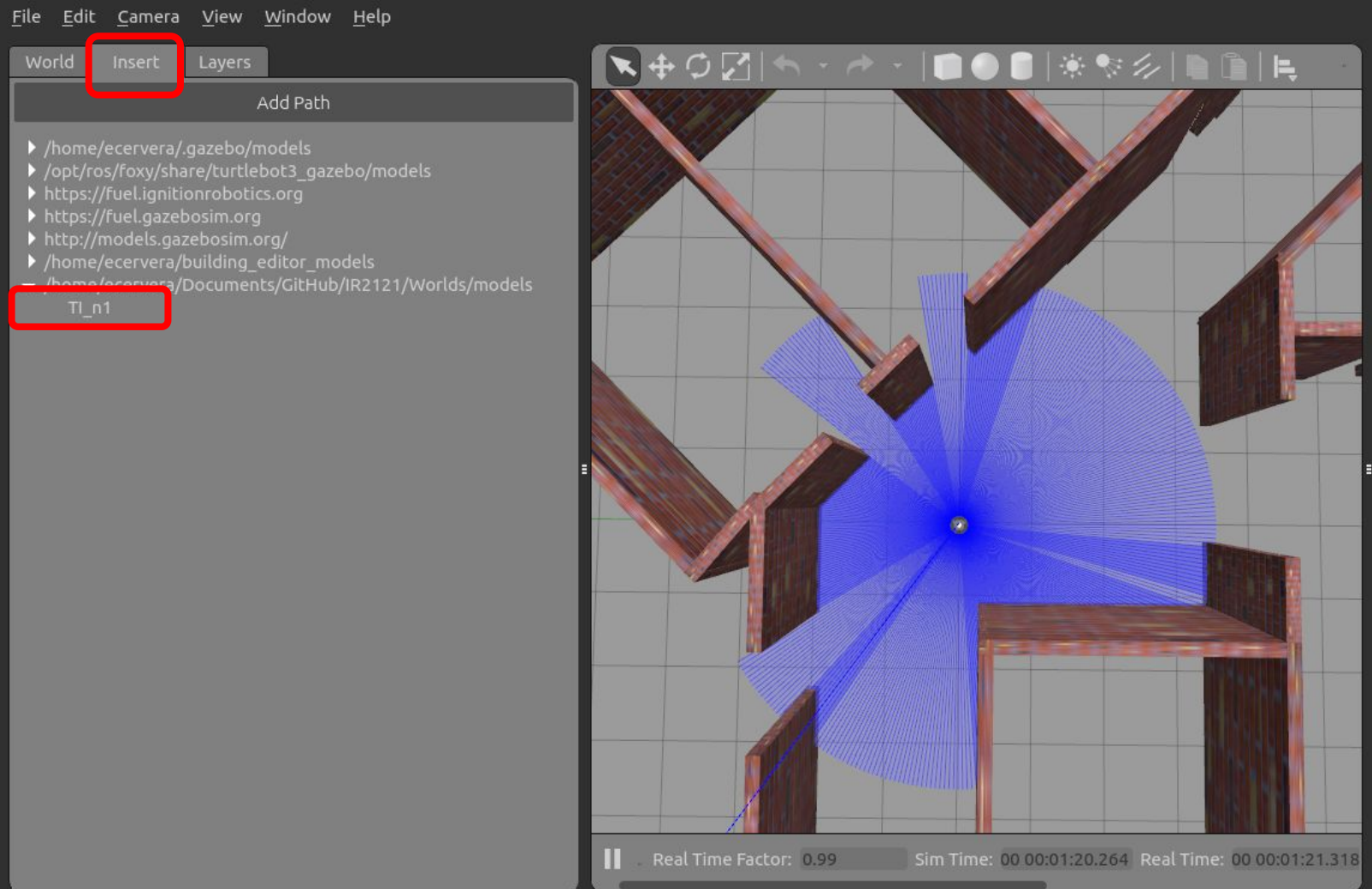
and run:

```
./launch_tb3_empty_sim.bash
```

Add
path
to
model



Insert model



Visualize robot and map

Open a terminal window, cd to IR2121/Worlds/scripts and run:

```
./visualize_robot.bash
```

Open a terminal window, cd to IR2121/Worlds/scripts and run:

```
./publish_map.bash
```

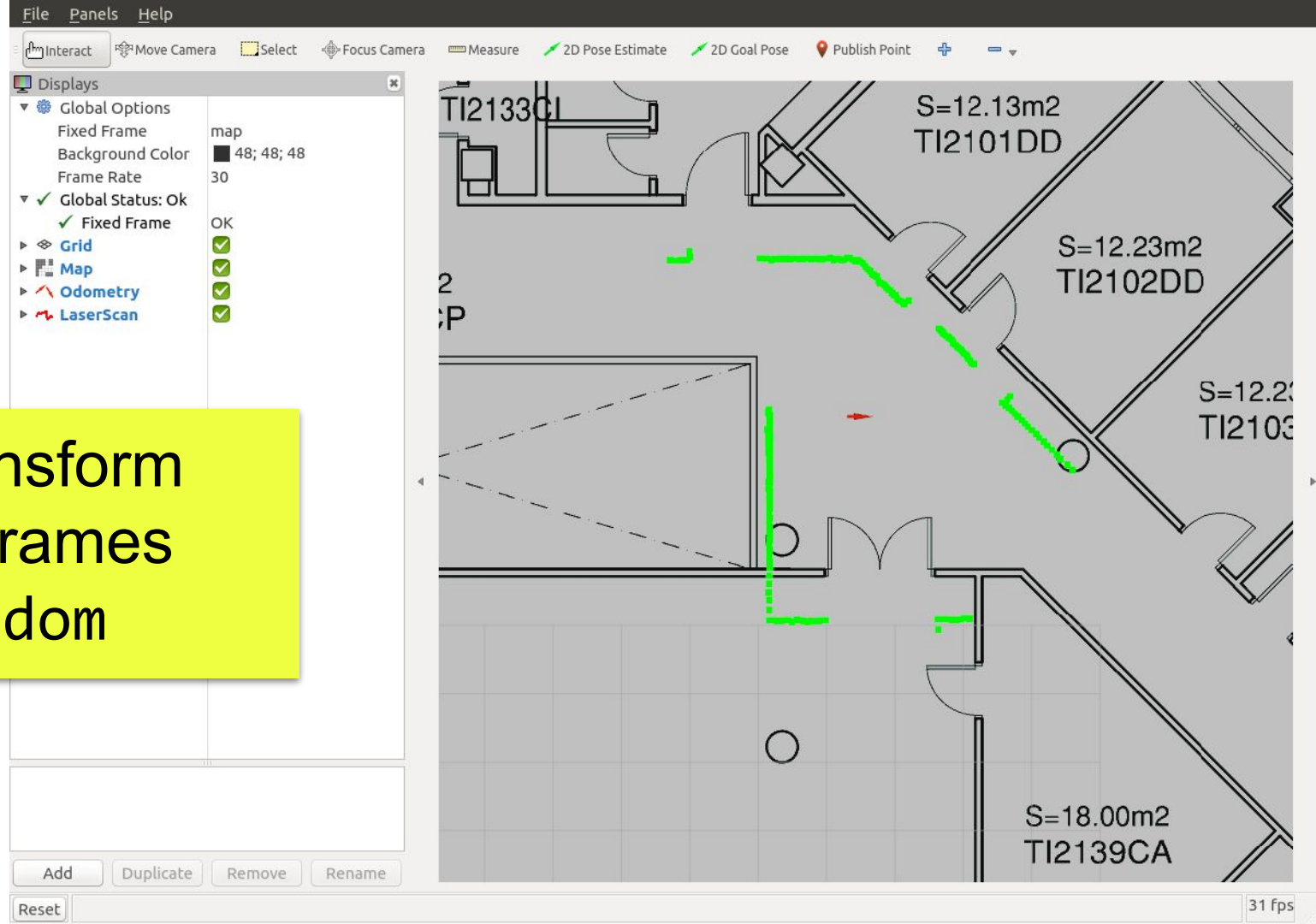
Open a terminal window, cd to IR2121/Worlds/scripts and run:

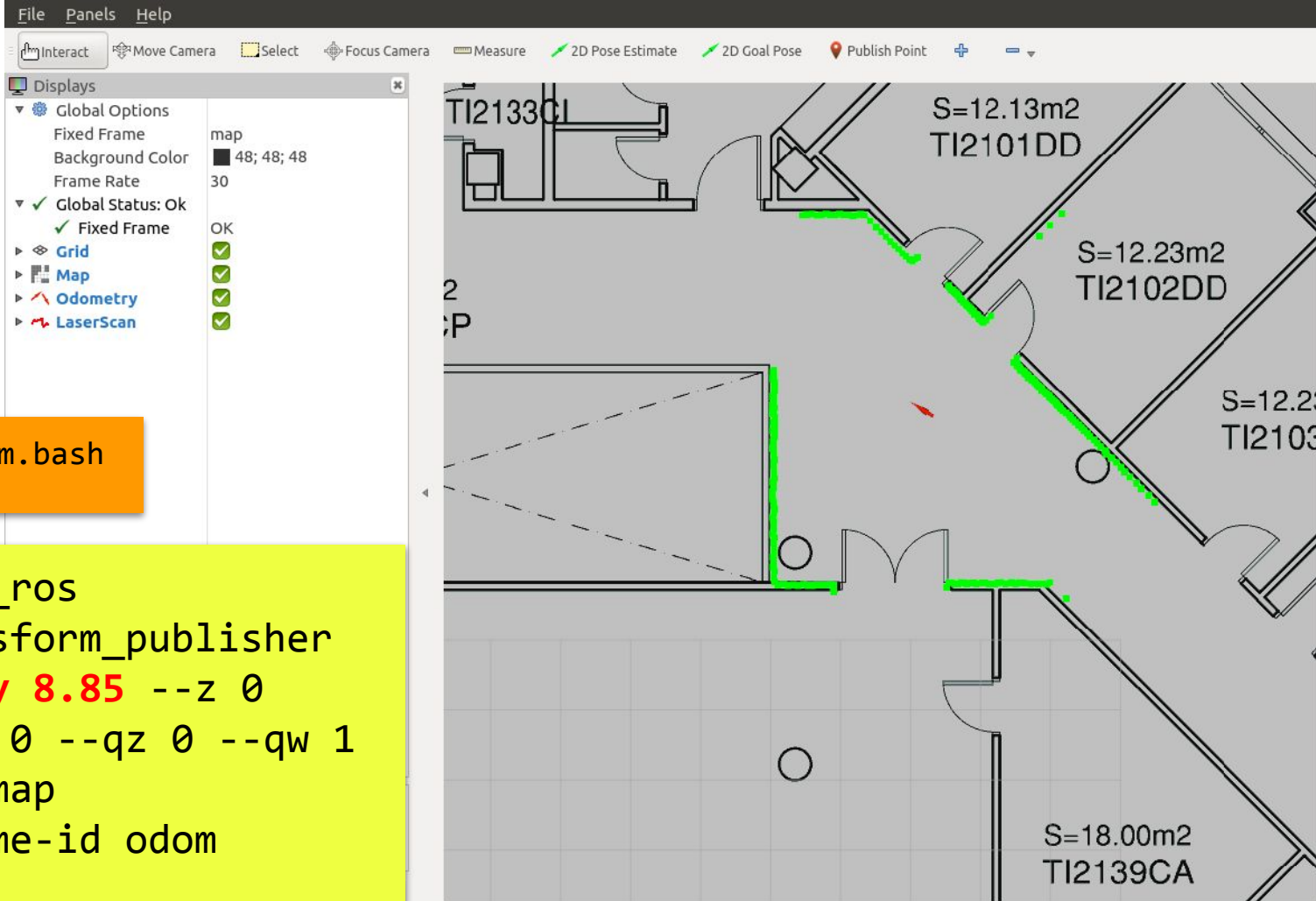
```
./publish_transform.bash
```

static_transform_publisher

```
ros2 run tf2_ros static_transform_publisher \  
  --x 1 --y 8 --z 0 \  
  --qx 0 --qy 0 --qz 0 --qw 1 \  
  --frame-id map \  
  --child-frame-id odom
```

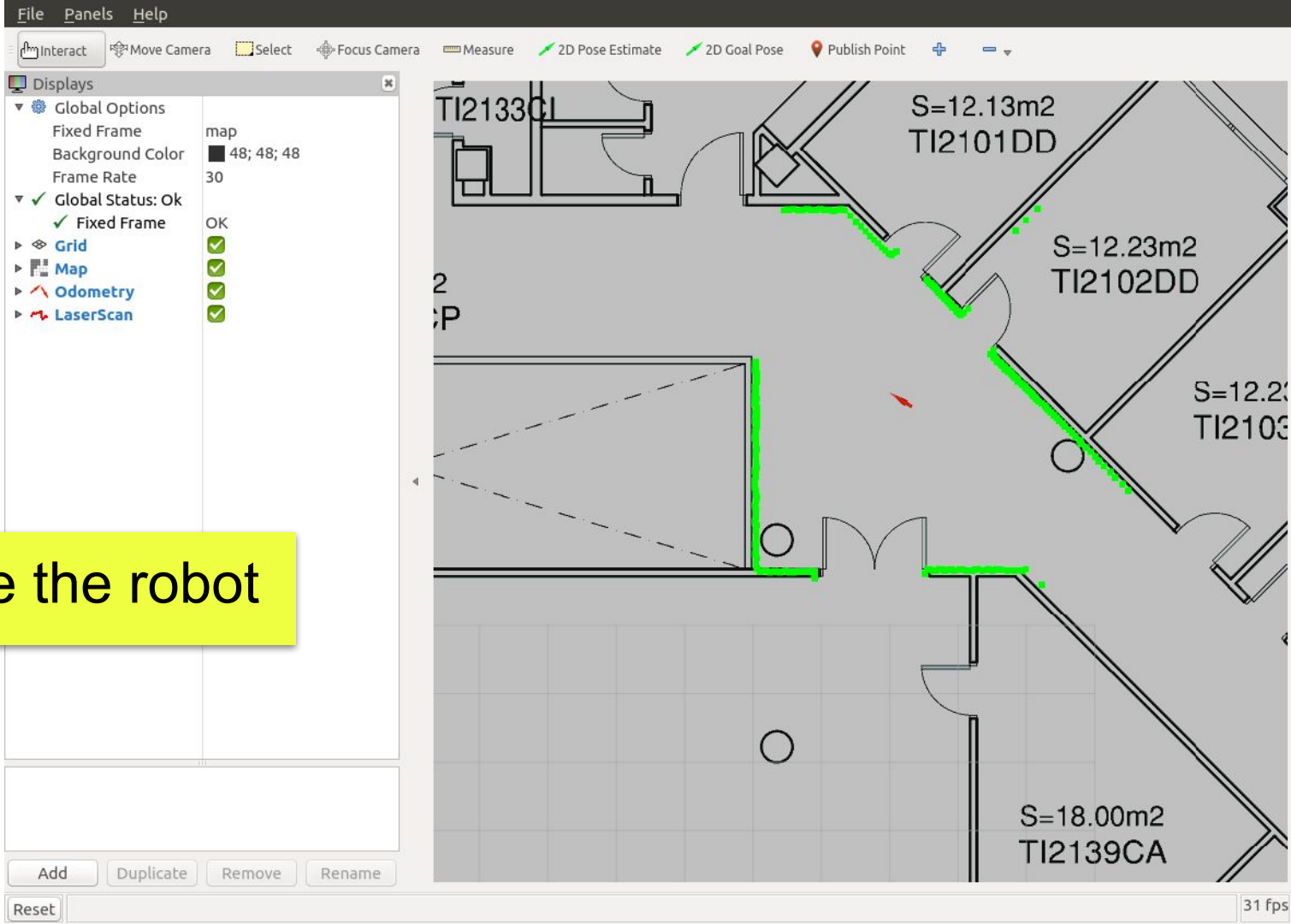
Adjust transform
between frames
map and odom



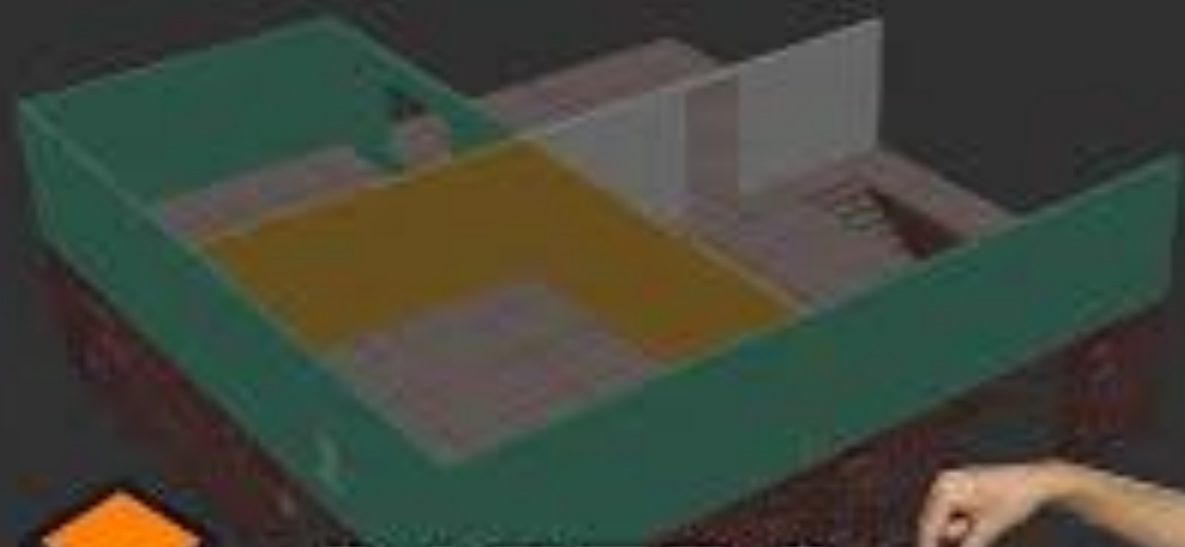


`publish_transform.bash`

```
ros2 run tf2_ros  
static_transform_publisher  
--x 1.15 --y 8.85 --z 0  
--qx 0 --qy 0 --qz 0 --qw 1  
--frame-id map  
--child-frame-id odom
```



Teleoperate the robot



GAZEBO



BUILDING EDITOR



ESCOLA SUPERIOR DE TECNOLOGIA
I CIÈNCIES EXPERIMENTALS

Paralela 17-18-19 - Campus de Río San...



UNIVERSITAT JAUME I CASTELLO
OFICINA TÈCNICA D'OBRES I PROJECTES

ES | AQUÍ FECTO JOSÉ PÉREZ BARRO

EDIFICIO TD : PLANTA NIVEL 1
ARQUITECTURA

TD-02

ESTCE

ESCALA: 1 / 200

FECHA : SEPTIEMBRE 2022
SUSTITUYE : JULIO 2018

22	PLANO DE :
	ESTADO ACTUAL

TD_n1.pdf

Format: PDF-1.7

Paper size: A1, landscape (841 × 594 mm)

Scale: 1/200

[PDF to PNG](#)

Format: png (PNG)

Width: 6623 px

Height: 4678 px

TD_n1.yaml

```
image: ./TD_n1.png
mode: trinary
resolution: meters / pixel ???
origin: [ , , 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Task 1: simulation

Visualize the map of building TD, floor 1, in RViz.

Build a Gazebo model of the laboratory TD1107AL and the surrounding area.

Launch a Gazebo simulation of a TurtleBot 3 and the model of the laboratory.

Teleoperate the robot, and verify that the laser scan fits the walls in the map.

Move the robot and record the following topics in a bagfile for 60-90 seconds:

```
/clock /map /odom /scan /tf /tf_static
```

Submit the requested files in the corresponding task of Aula Virtual:

- `rosbag2_2026_...` (compressed bagfile)
- a screenshot of `rviz2` showing the map, the odometry and the laser signal

Task 2: physical robot

Visualize the map of building TD, floor 1, in RViz.

Launch the TurtleBot 3 physical robot.

Teleoperate the robot, and verify that the laser scan fits the walls in the map.

Submit the requested files in the corresponding task of Aula Virtual.

Move the robot and record the following topics in a bagfile for 60-90 seconds:

`/map /odom /scan /tf /tf_static`

Submit the requested files in the corresponding task of Aula Virtual:

- `rosbag2_2026_...` (compressed bagfile)
- a screenshot of rviz2 showing the map, the odometry and the laser signal

Tips for working with the physical robot

Use one of the laptops of the lab (WiFi or Ethernet)

Or, use the desktop PC with the Wifi PIROBOTNET6

Or, use your own laptop with a USB WiFi dongle / cable

Do **NOT** define `ROS_LOCALHOST_ONLY` (“unset”)

Do define `ROS_DOMAIN_ID`

Always check the values of the /odom topic

```
(ros2 topic echo /odom --no-arr)
```

Tips for working with the physical robot

Check the /scan topic in RViz

Reliability policy: 'Best effort'

Durability policy: 'Volatile'

Homework: model of building TD

[Build a more complete Gazebo model](#) of the TD central building, level 1.

For a satisfactory grade: include TD1107AL and TD1118CP.

For an excellent grade: also include TD2116CP.

Record a bagfile with the topics during a teleoperation of the robot across the building.

Submit these files:

- TD_n1.yaml
- TD_n1.zip (Gazebo model, including model.config and model.sdf)
- rosbag2_2026_xxx.zip (compressed bagfile)

References

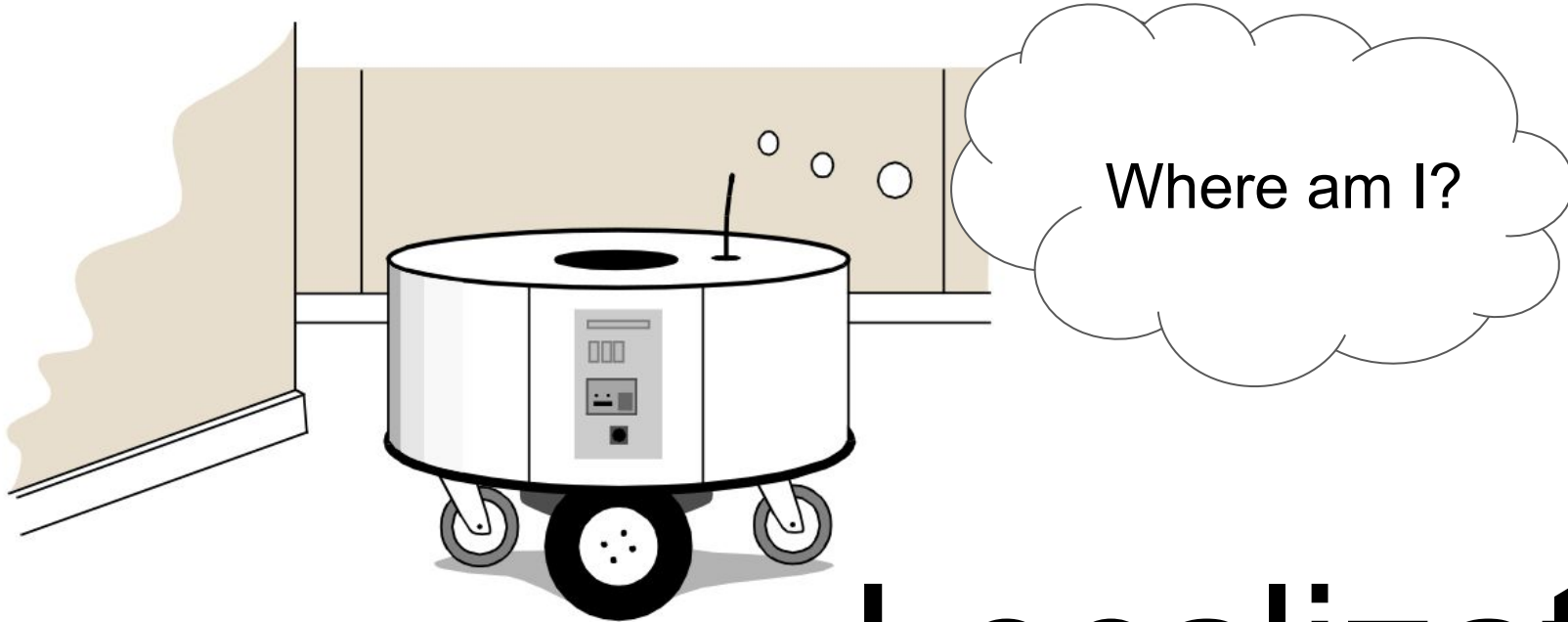
[Using tf2 with ROS 2](#)

[map_server - ROS Wiki](#)

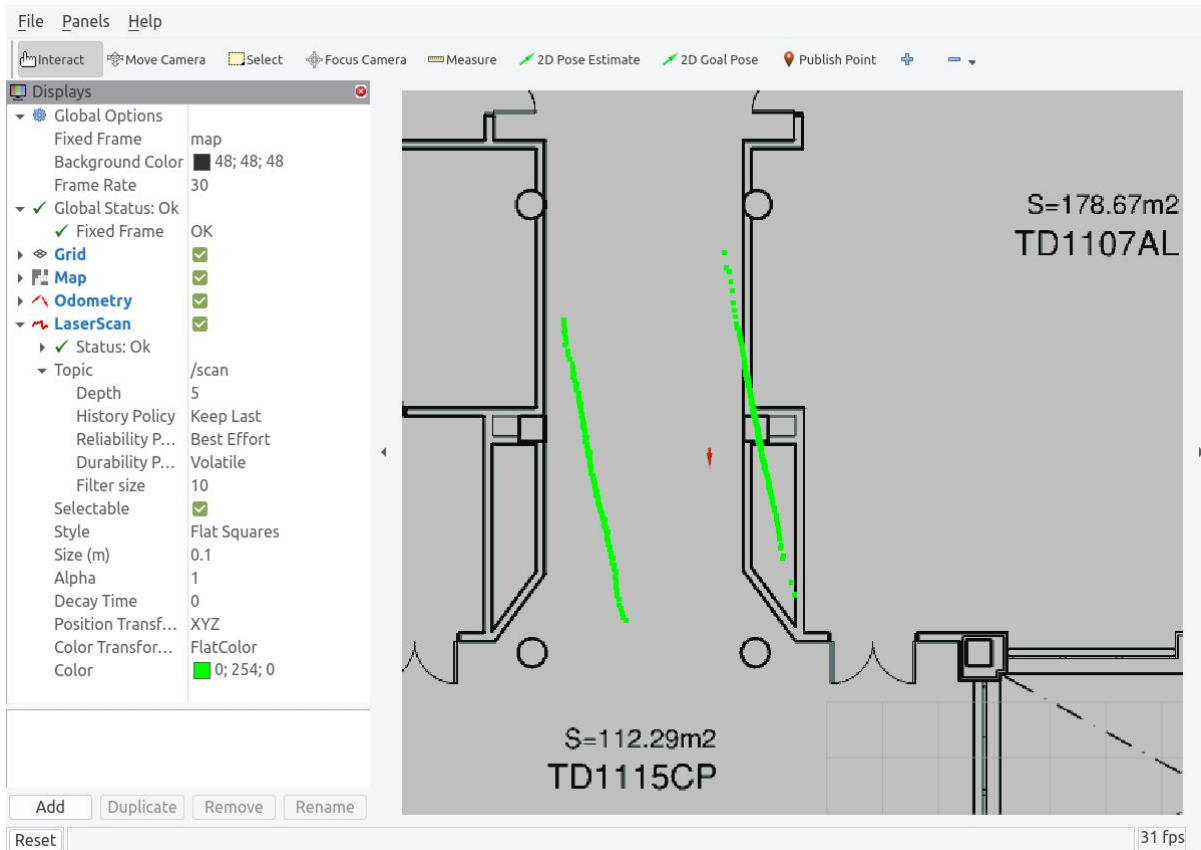
[Map Server / Saver — Nav2 1.0.0 documentation](#)

[Gazebo : Tutorial : Building Editor](#)

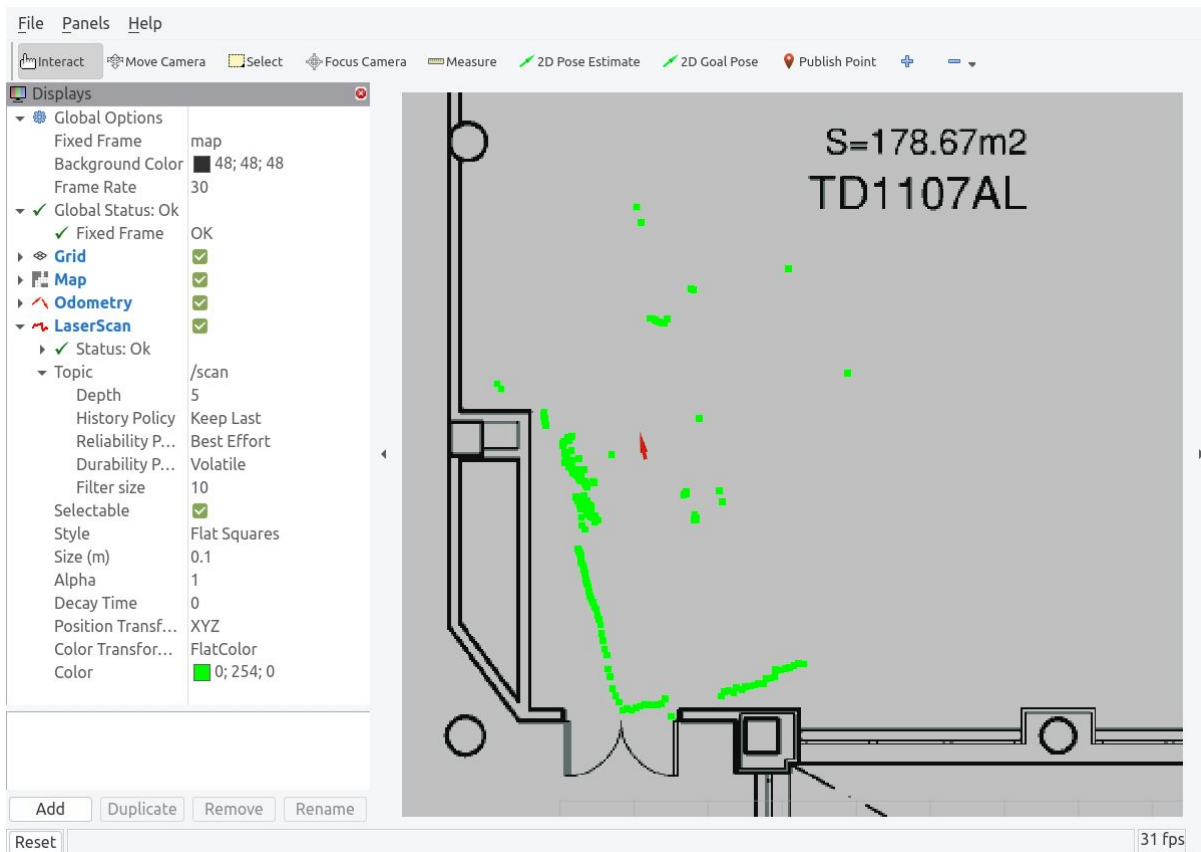
Mobile Robotics



Localization



The problem



The problem

Specification -

Coordinate Frames

base_link

The coordinate frame called `base_link` specifies position or orientation; for every hardware 103 [1] specifies a preferred orientation for

odom

The coordinate frame called `odom` is a world frame. Drift makes the `odom` frame useless as a long-term reference, meaning that the pose of a mobile platform

In a typical setup the `odom` frame is computed by integrating the velocity data from the

drift

noun [countable]

/drift/

a slow change, often toward sth bad

[decaimiento](#) [masculine]

- *the drift toward war*
[el decaimiento hacia la guerra](#)
- *the Democrats' drift toward the center*
[el decaimiento de los demócratas hacia el centro](#)

a slow movement

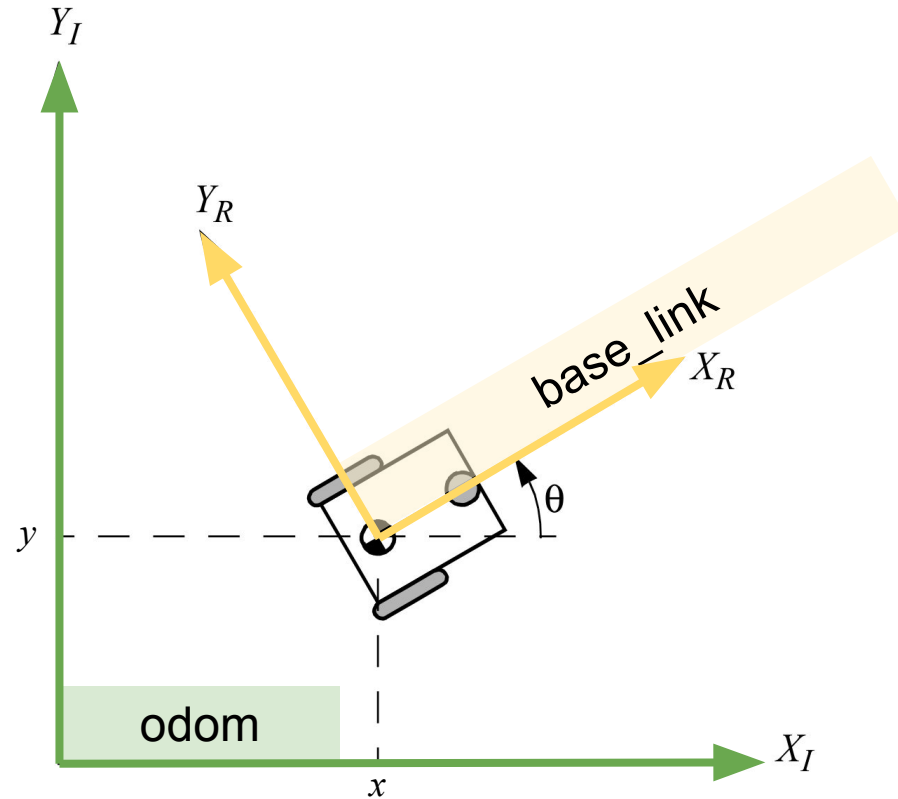
[tendencia](#) [feminine]

[movimiento](#) [masculine]

- *the drift of the continents*
[el movimiento de los continentes](#)

The `odom` frame is useful as an accurate, short-term local reference, but drift makes it a poor frame for long-term reference.

ROS frames of reference



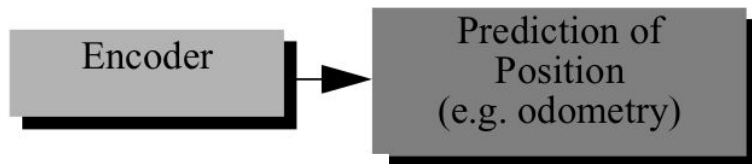
The coordinate frame called odom is a world-fixed frame.

The pose of a mobile platform in the odom frame can drift over time, without any bounds. This drift makes the odom frame useless as a long-term global reference. However, the pose of a robot in the odom frame is guaranteed to be continuous, meaning that the pose of a mobile platform in the odom frame always evolves in a smooth way, without discrete jumps.

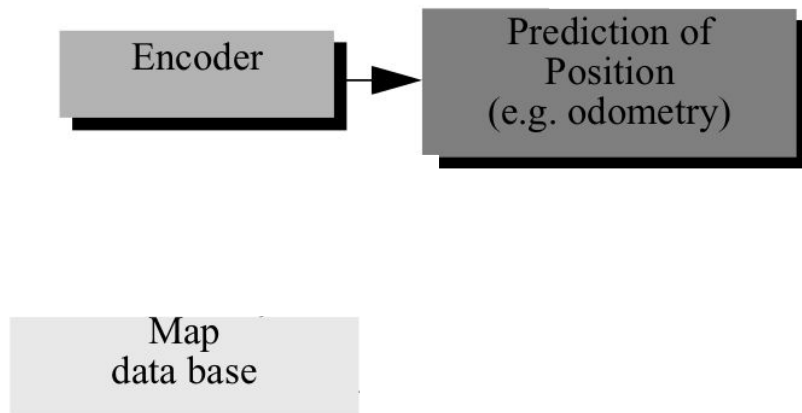
In a typical setup the odom frame is computed based on an **odometry source**, such as **wheel** odometry, **visual** odometry or an **inertial measurement unit**.

The odom frame is useful as an **accurate, short-term** local reference, but drift makes it a **poor frame for long-term** reference.

The solution



The solution

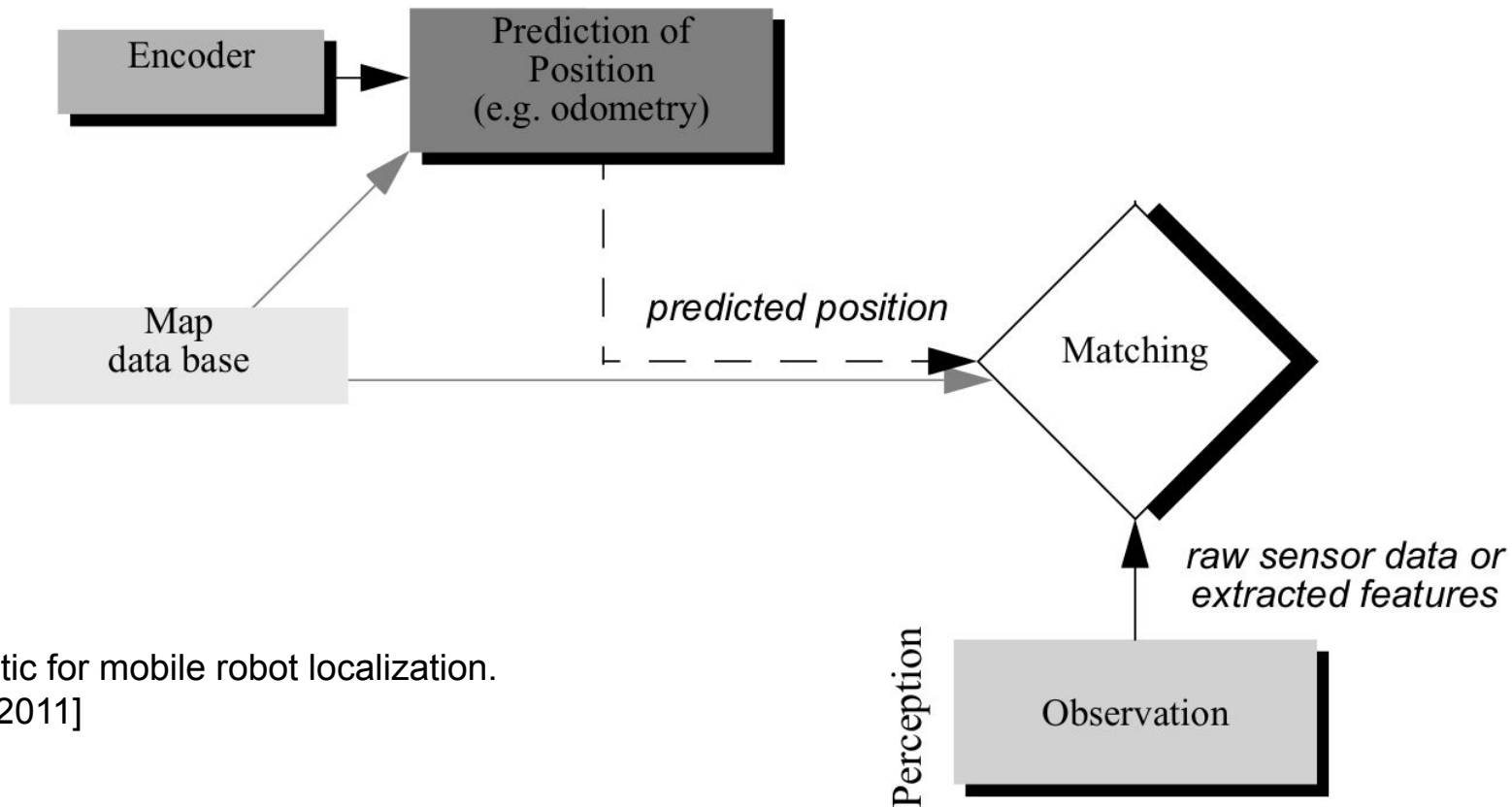


General schematic for mobile robot localization.
[Siegwart et al., 2011]

Perception

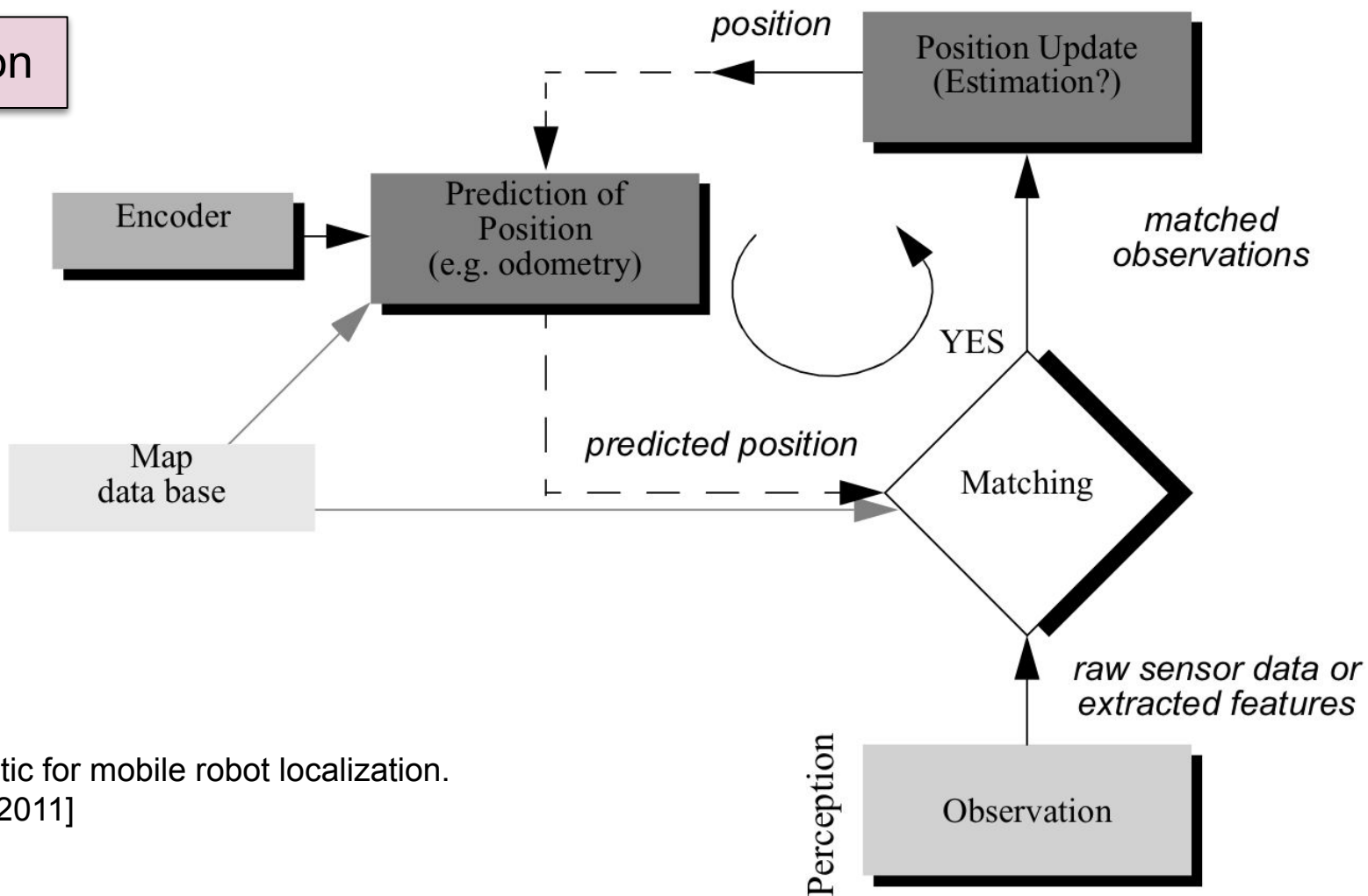
Observation

The solution



General schematic for mobile robot localization.
[Siegwart et al., 2011]

The solution



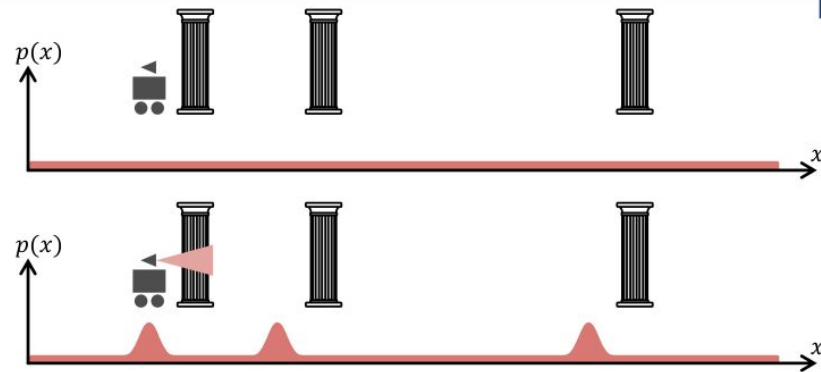
General schematic for mobile robot localization.
[Siegwart et al., 2011]

ACT - SEE Cycle for Localization



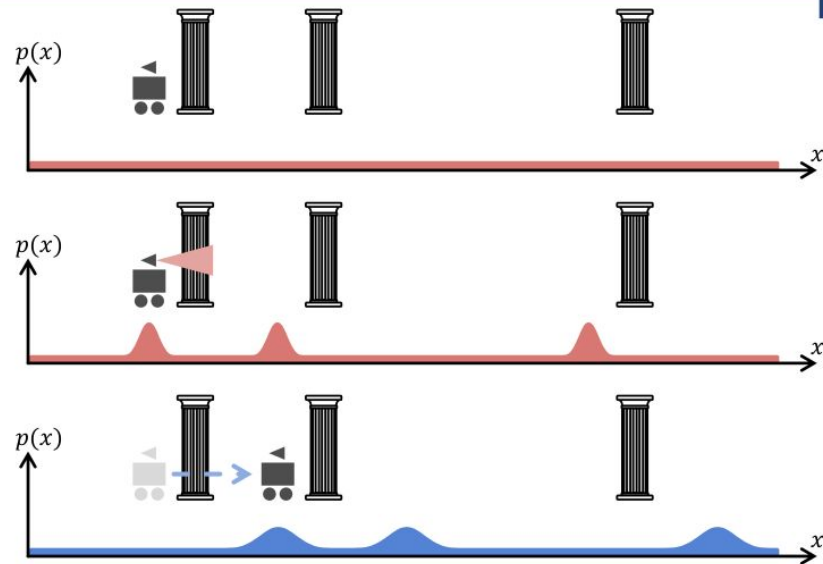
ACT - SEE Cycle for Localization

- **SEE:** The robot queries its sensors
→ finds itself next to a pillar



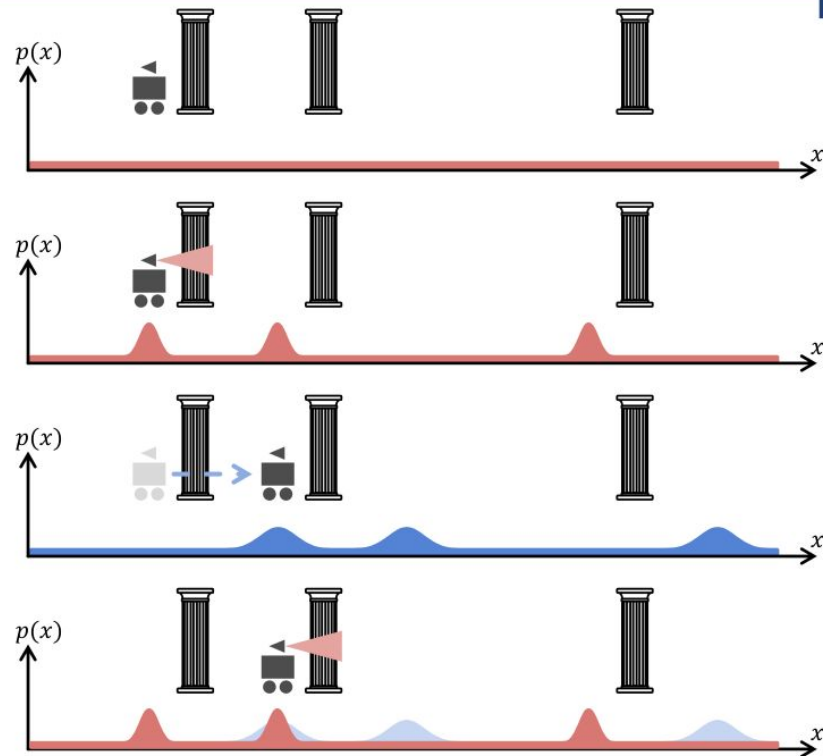
ACT - SEE Cycle for Localization

- **SEE:** The robot queries its sensors
→ finds itself next to a pillar
- **ACT:** Robot moves one meter forward
 - motion estimated by wheel encoders
 - accumulation of uncertainty



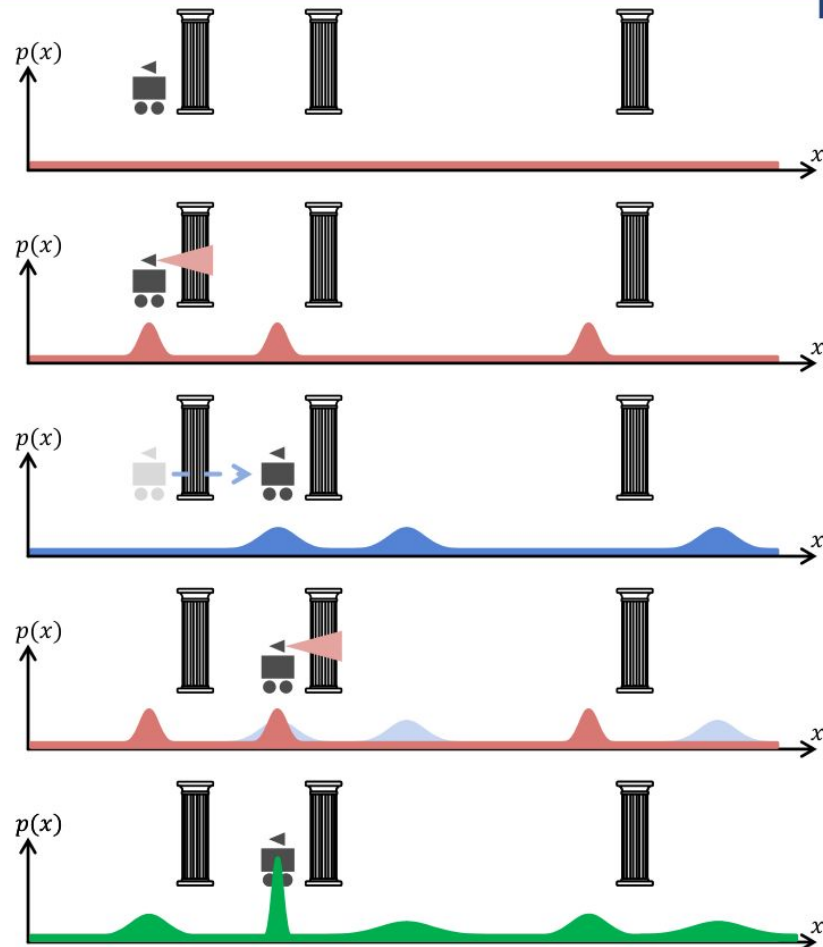
ACT - SEE Cycle for Localization

- **SEE:** The robot queries its sensors
→ finds itself next to a pillar
- **ACT:** Robot moves one meter forward
 - motion estimated by wheel encoders
 - accumulation of uncertainty
- **SEE:** The robot queries its sensors
again → finds itself next to a pillar

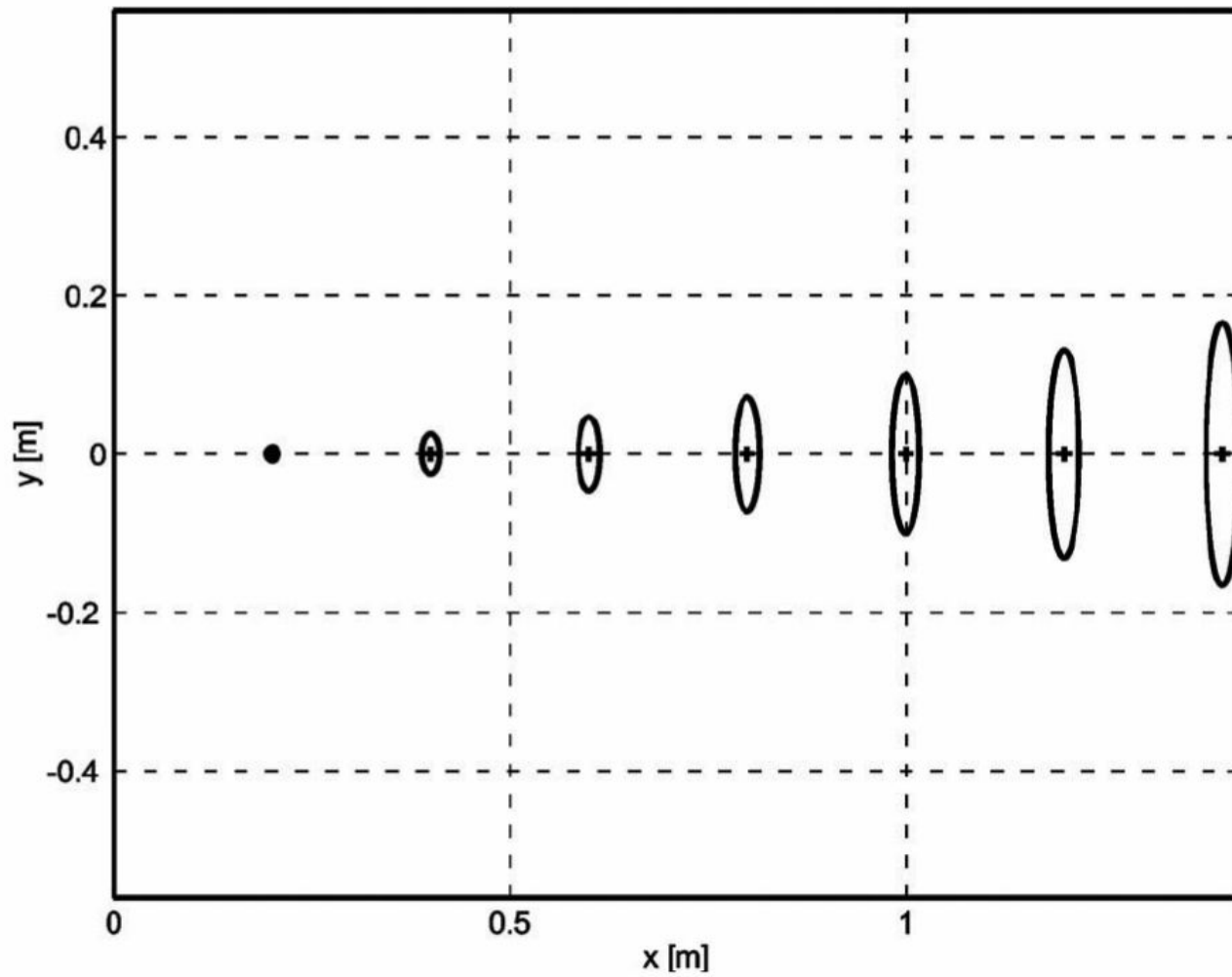


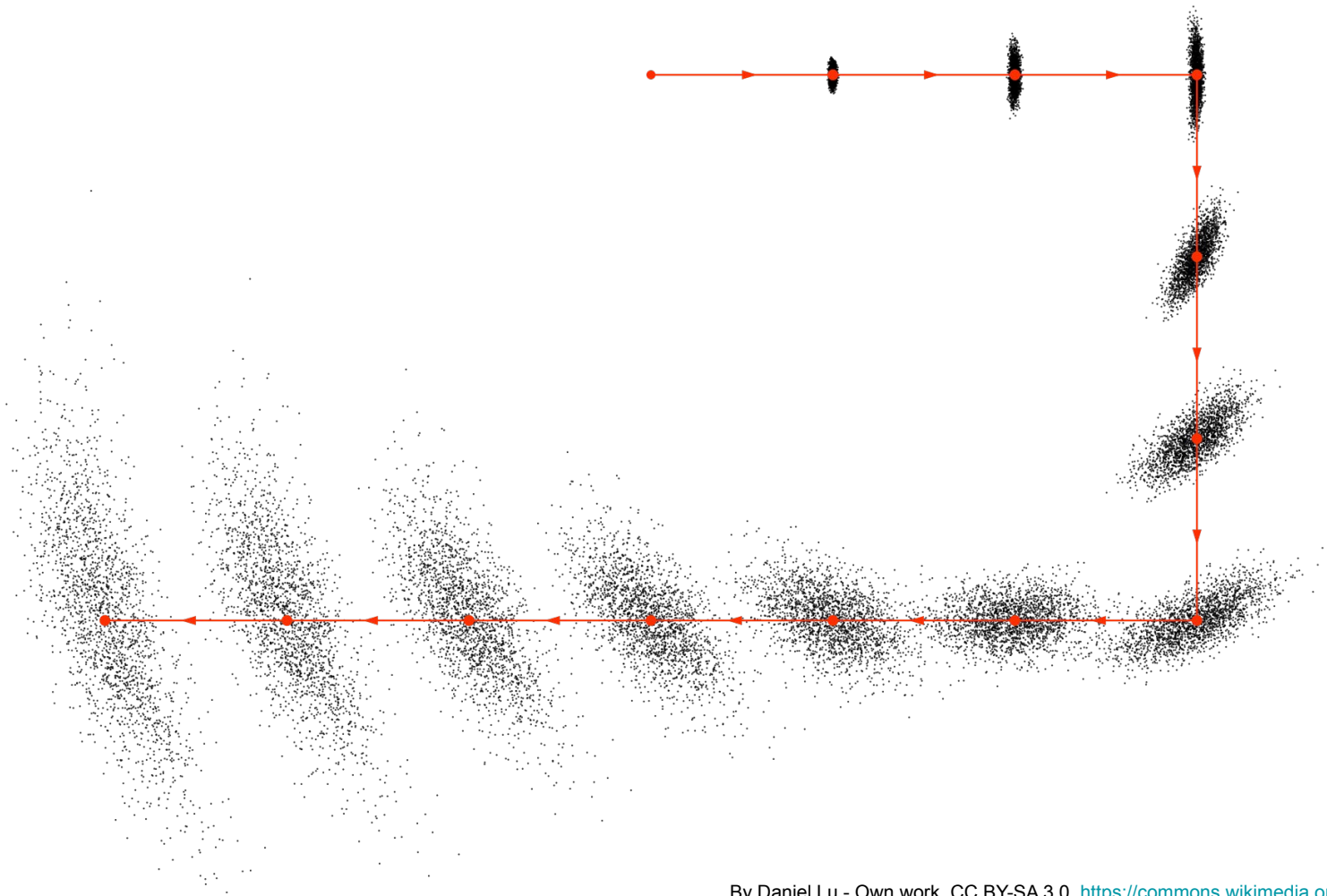
ACT - SEE Cycle for Localization

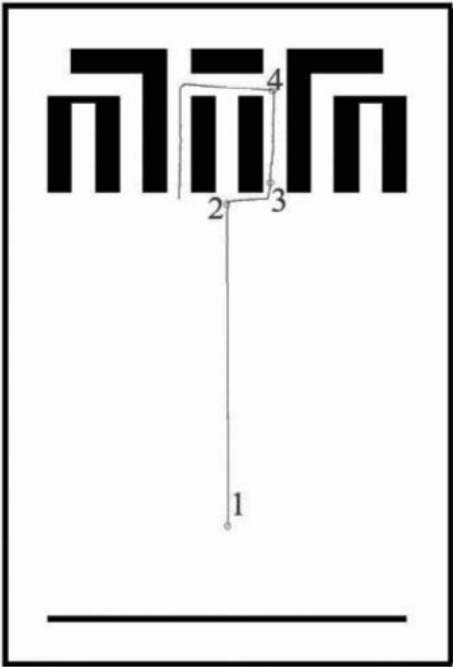
- **SEE:** The robot queries its sensors
→ finds itself next to a pillar
- **ACT:** Robot moves one meter forward
 - motion estimated by wheel encoders
 - accumulation of uncertainty
- **SEE:** The robot queries its sensors
again → finds itself next to a pillar
- **Belief update (information fusion)**



Error Propagation in Odometry

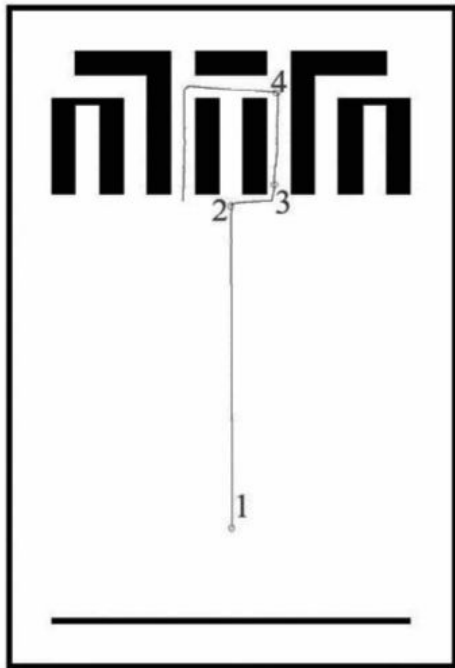




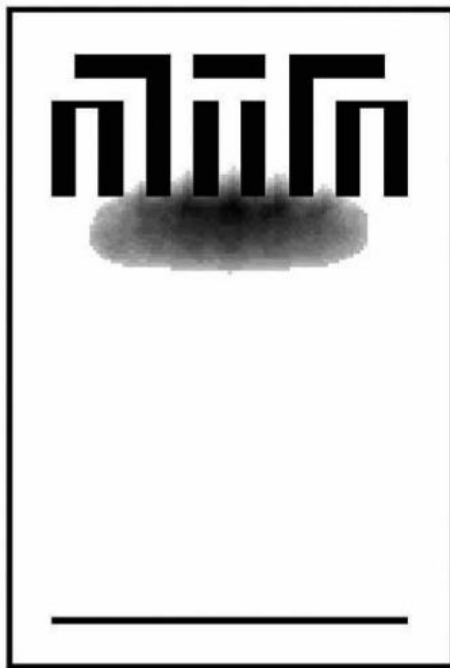


Path of the robot

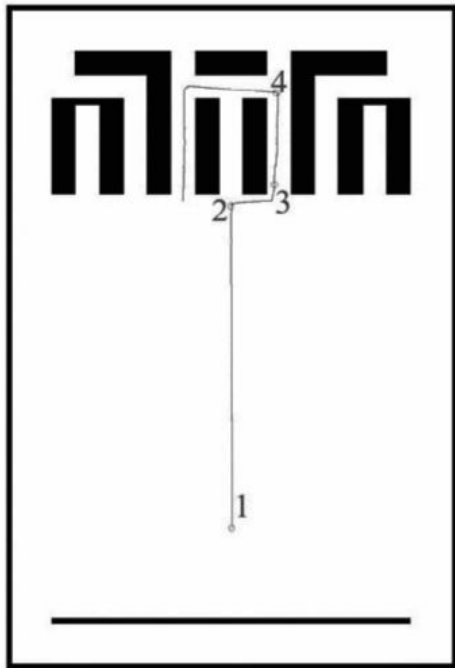
Belief states at positions 2, 3, and 4



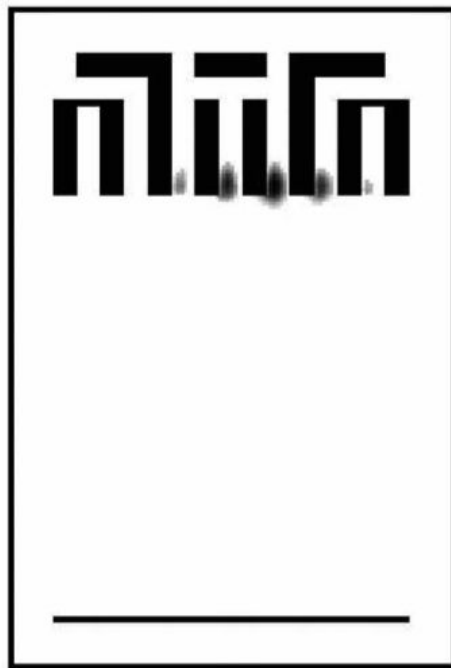
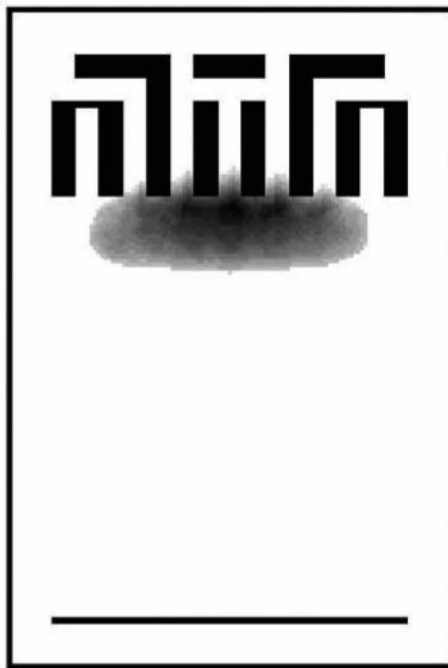
Path of the robot



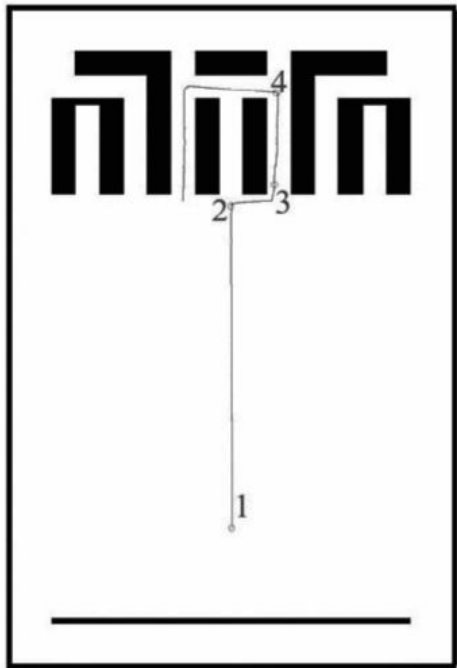
Belief states at positions 2, 3, and 4



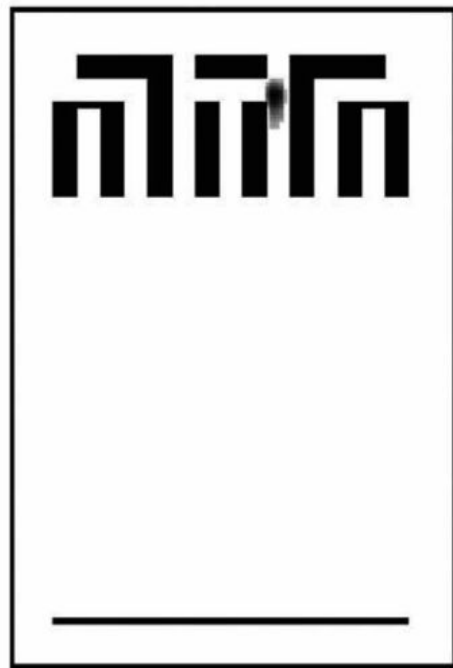
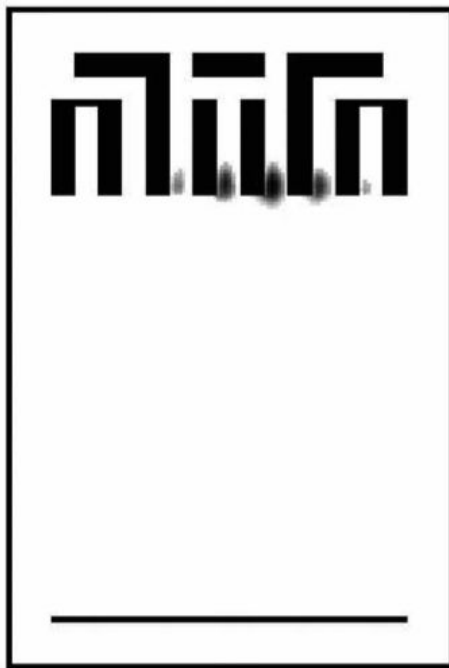
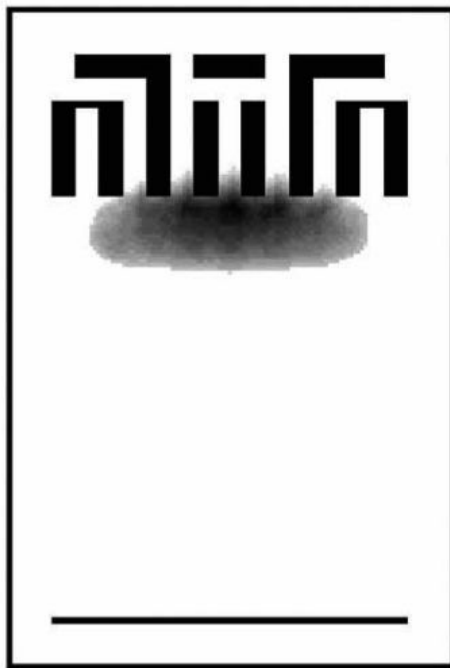
Path of the robot



Belief states at positions 2, 3, and 4



Path of the robot



Belief states at positions 2, 3, and 4

Localization in simulation

Download archive [Worlds.zip](#) (updated version).

Uncompress it in your IR2121 repository folder.

Launch an empty Gazebo simulation of the TurtleBot 3.

Insert the model of the TI building in the simulation.

Localization in simulation

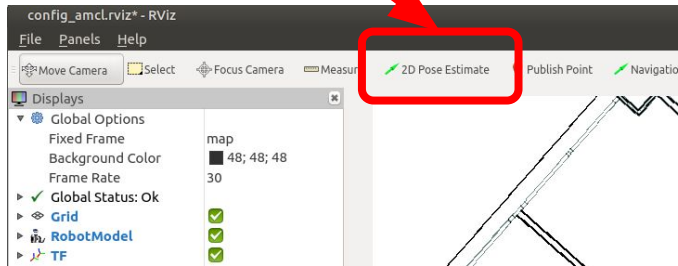
Launch AMCL with:

```
cd Worlds/scripts/ && ./launch_amcl.bash
```

Launch RViz2 with:

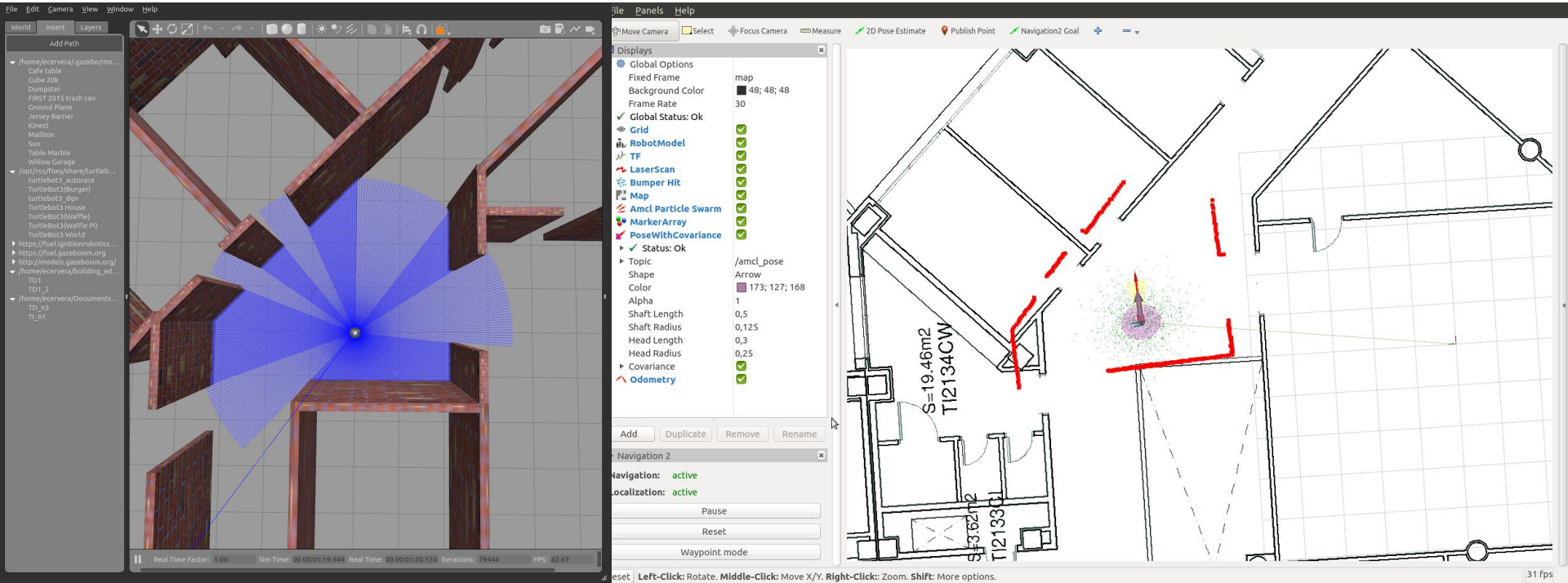
```
cd Worlds/scripts/ && ./visualize_localization.bash
```

Estimate the 2D pose of the robot



```
sudo apt install  
ros-humble-turtlebot3-navigation2
```

```
1 #!/bin/bash  
2 source /opt/ros/humble/setup.bash  
3 export ROS_LOCALHOST_ONLY=1  
4 export TURTLEBOT3_MODEL=burger  
5  
6 ros2 launch amcl.launch.py \  
7   use_sim_time:=True \  
8   map:=../TI_n1_edited.yaml
```



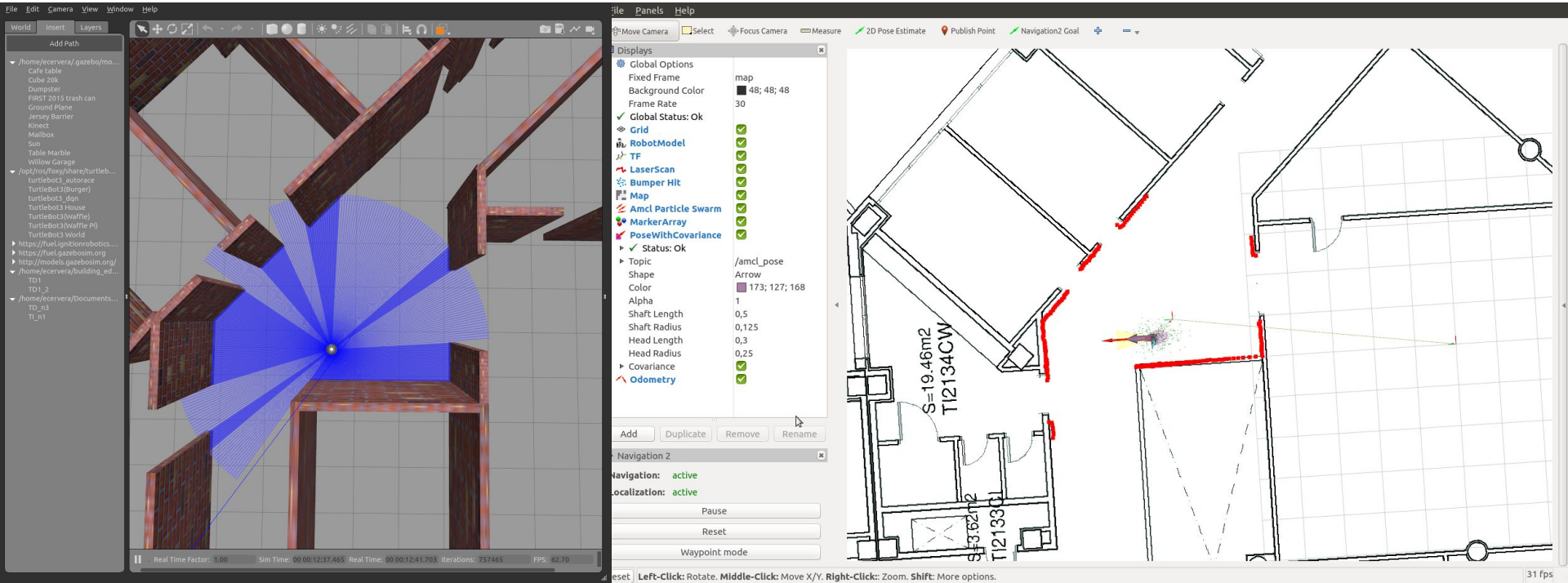
Localization in simulation

Move the robot around with the keyboard.

The AMCL algorithm should automatically adjust the localization of the robot with respect to the map.

Record the following topics in a bagfile for 60 seconds:

```
/clock /map /odom /scan /tf /tf_static  
/amcl_pose /particle_cloud /robot_description
```



Verification of the recorded bagfile

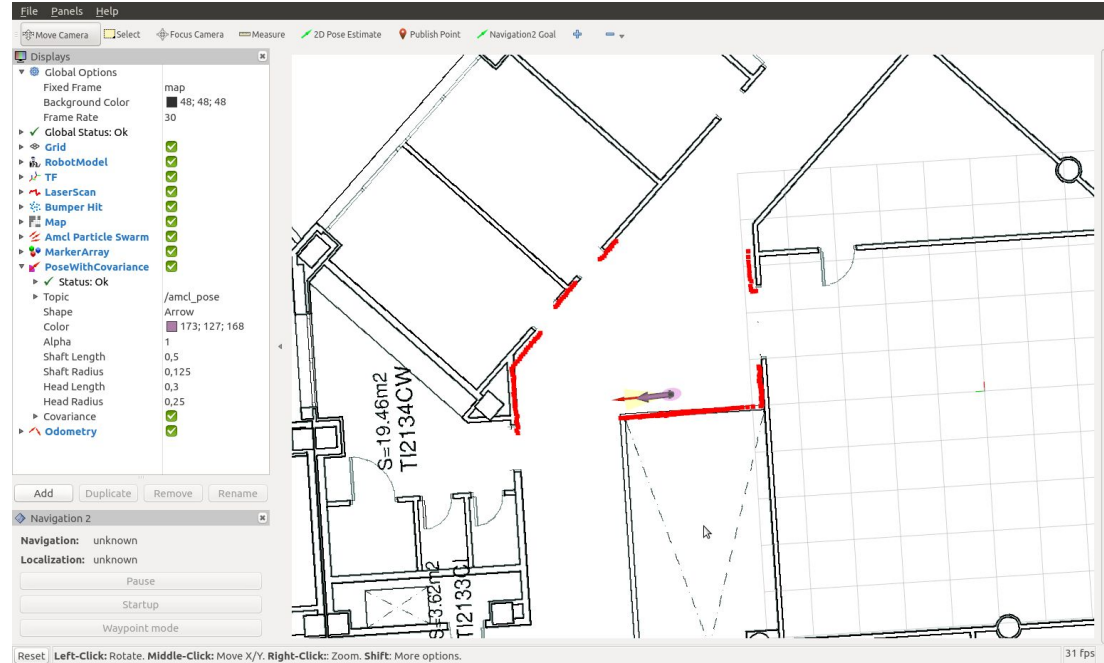
Stop the simulator

Stop RViz

Stop teleoperation

Launch RViz (`./visualize_localization.bash`)

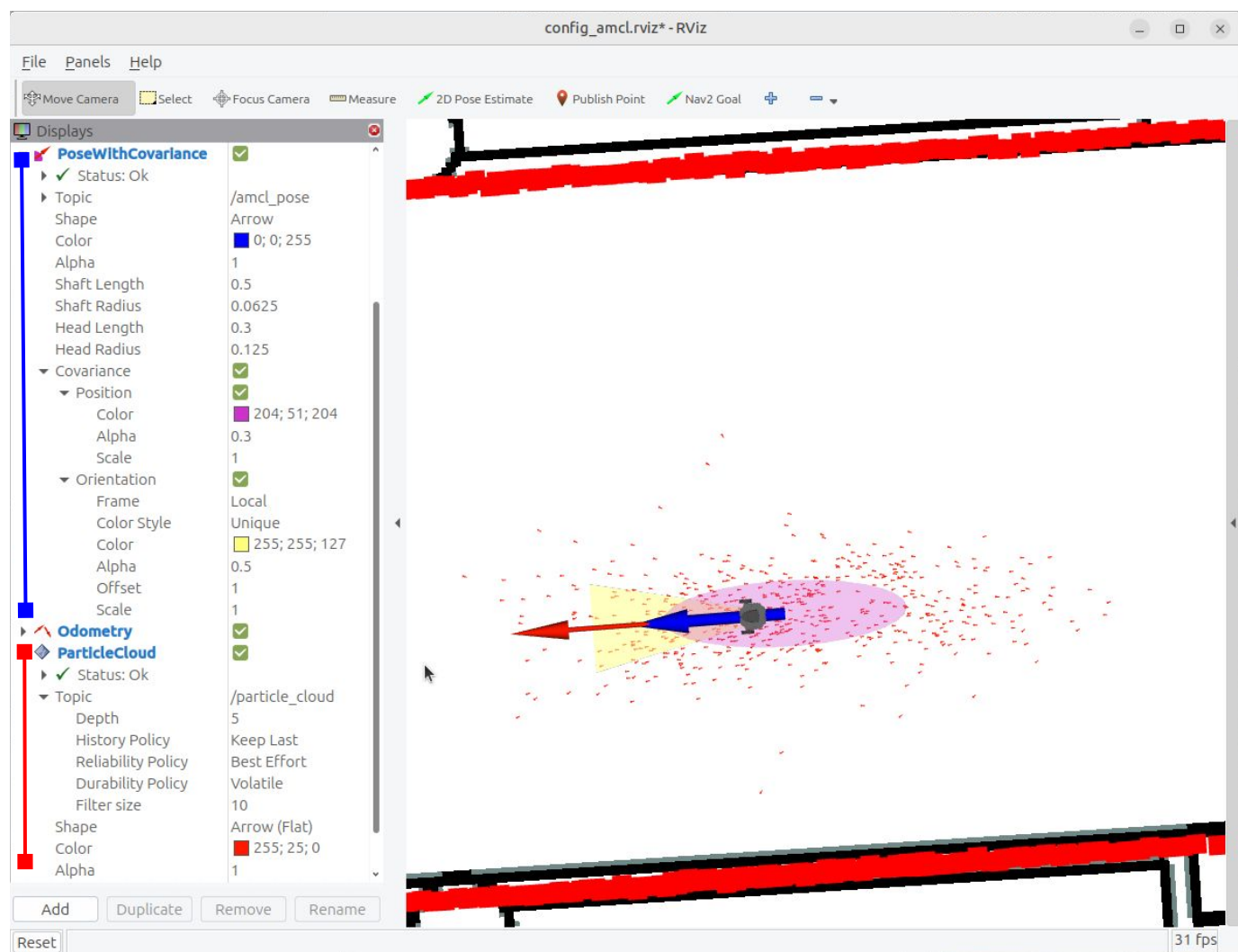
Play the bagfile



Visualization of AMCL pose and particle cloud

AMCL pose
(with covariance)

Particle Cloud



Task 1: localization in simulation

Launch a Gazebo simulation of the TurtleBot 3

Insert your model of the TD building in the simulation

Launch AMCL

Launch RViz2

Estimate the 2D pose of the robot

Move the robot around with the keyboard

Record 60-90 seconds of the following topics in a bagfile:

```
/clock /map /odom /scan /tf /tf_static  
/amcl_pose /particle_cloud /robot_description
```

Verification

Stop everything and relaunch rviz

Visualize the recorded bagfile prior to submission to Aula Virtual

If some topics are not displayed:

Launch `ros2 bag record before` any other ROS script

Task 2: localization with the TurtleBot 3

Launch the TurtleBot 3 physical robot

Launch AMCL in the desktop PC

Launch RViz2 in the desktop PC

Estimate the 2D pose of the robot

Move the robot around with the keyboard

Record between 1 and 3 minutes of the following topics in a bagfile:

`/map /odom /scan /tf /tf_static`

`/amcl_pose /particle_cloud /robot_description`

Verification

Stop everything and relaunch rviz

Visualize the recorded bagfile prior to submission to Aula Virtual

If some topics are not displayed:

Launch `ros2 bag record before` any other ROS script

Task 2: localization with the TurtleBot 2

Launch the [TurtleBot 2 physical robot](#)

Launch AMCL in the desktop PC

Launch RViz2 in the desktop PC

Estimate the 2D pose of the robot

Move the robot around with the keyboard

Record between 1 and 3 minutes of the following topics in a bagfile:

`/map /odom /scan /tf /tf_static`

`/amcl_pose /particle_cloud /robot_description`

References

[Siegwart et al., 2011] Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots. MIT press. [Catàleg UJI](#).

[AMCL - ROS Wiki](#)

[AMCL — Nav2 1.0.0 documentation](#)

[Monte Carlo localization - Wikipedia](#)

[Robust Monte Carlo localization for mobile robots - ScienceDirect](#)